DIGITAL FORENSICS

Fall 2025

Montgomery College

PORTFOLIO

A Collection of Labs & Projects

Submitted by:

Marqell Broxton

Email:

marqellbroxton@gmail.com

LinkedIn:

linkedin.com/in/marqellbroxton

Table of Contents

Course: NWIT 263 – Digital Forensics
Lab Number: 01
Lab Title: Recovering Deleted Windows Files Using Recuva

Professor: Reza Mirabrishami
Date: 9/11/2025

1. Introduction

The purpose of this lab was to learn how deleted files can be recovered from a Windows system using the Recuva recovery tool. File deletion does not immediately remove data from storage; instead, the space is marked as available for reuse. This lab demonstrated how forensic tools can recover deleted data and why this process is important in digital investigations.

2. Objectives

- Create and permanently delete a test file
- Install and run Recuva
- Recover a deleted file from the system
- Evaluate the condition of the recovered file
- Understand the forensic significance of file recovery tools

3. Tools Used

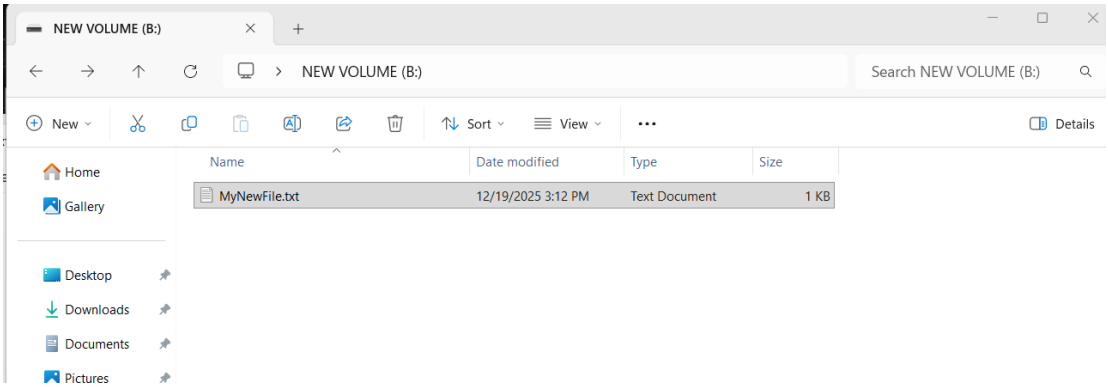
- Recuva (Free Version)
- Windows 10
- Notepad

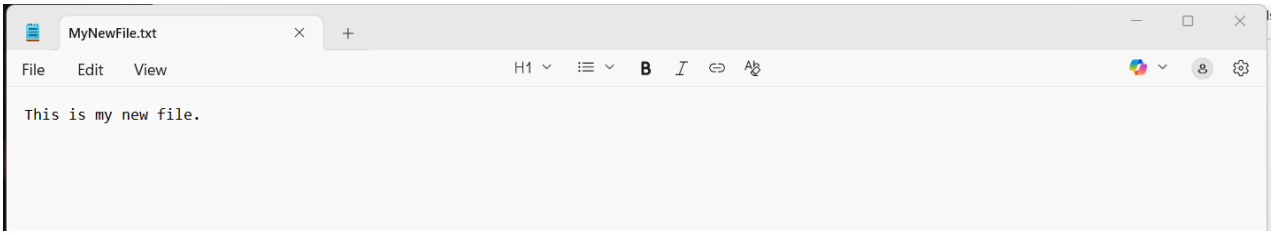
4. Lab Procedure

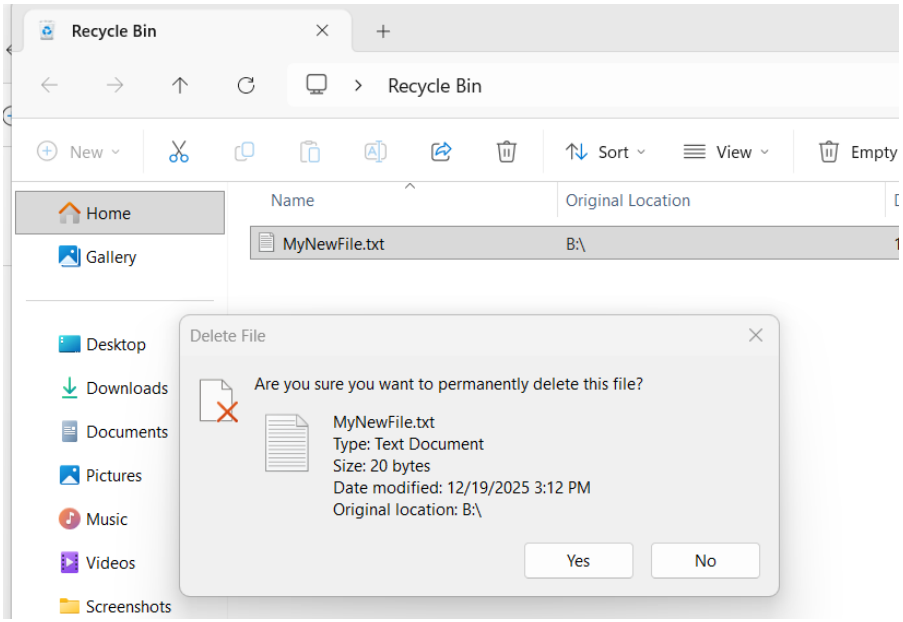
Part 1: File Creation and Deletion

A text file named **MyNewFile.txt** was created on the desktop using Notepad with the contents: “This is a test file for Recuva recovery.”

The file was then deleted and permanently removed by emptying the Recycle Bin.

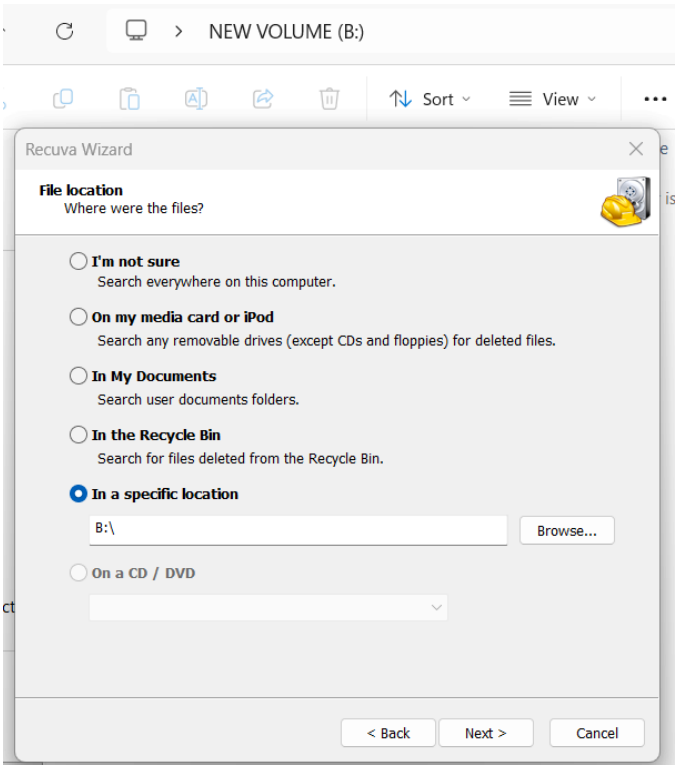






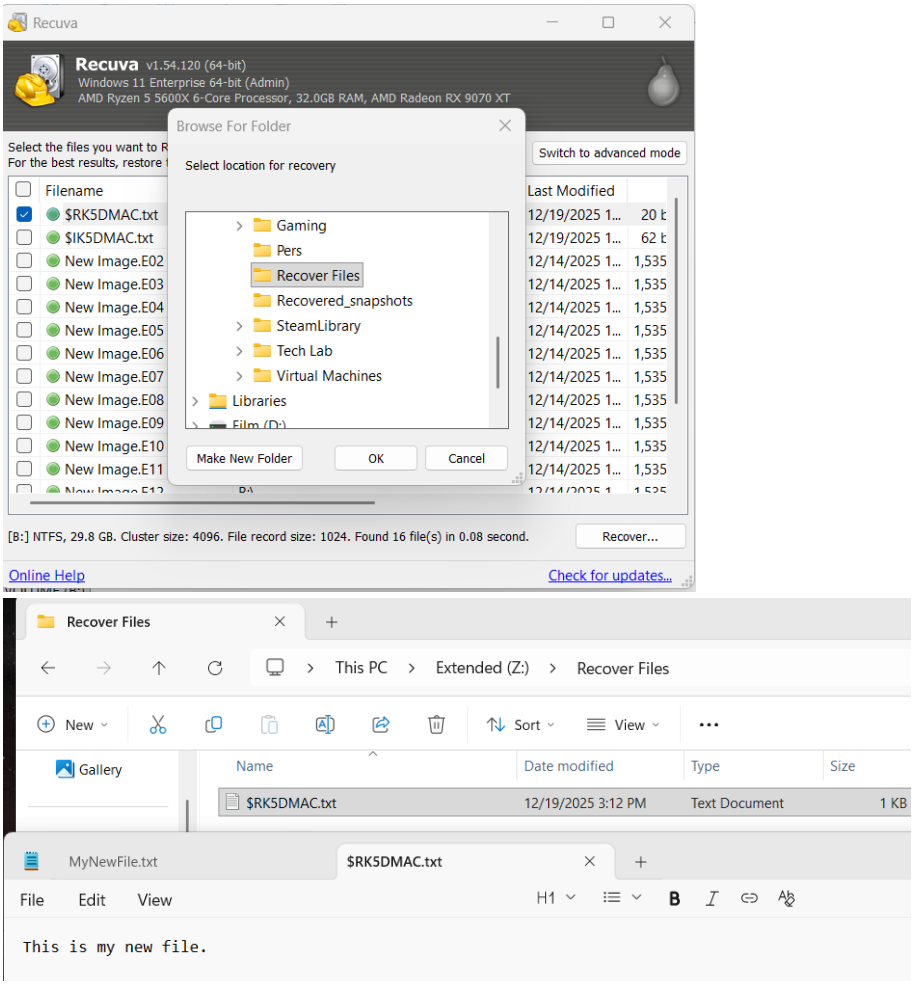
Part 2: Installing and Running Recuva

Recuva was downloaded from the official website and installed using default settings. The program was then launched from the Start Menu.



Part 3: File Recovery Process

- The file type **All Files** was selected.
- The scan location was set to a directory on an external drive.
- A scan was initiated to search for deleted files.
- The deleted file **MyNewFile.txt** was located in the results.
- The file was recovered and saved to a separate folder named **Recover Files**.



5. Analysis and Results

1. What file did you recover?

The recovered file was **MyNewFile.txt**, a plain text (.txt) document.

2. Was a Quick Scan sufficient, or did you need a Deep Scan?

A Quick Scan was sufficient to locate and recover the deleted file because it had not yet been overwritten.

3. Did the recovered file open correctly? If not, why?

Yes, the recovered file opened correctly. This occurred because the data clusters had not been overwritten after deletion.

4. Did the file name remain the same after recovery? If not, why?

No, the file name was not the same. Recuva renamed the file during recovery.

5. Why is Recuva important in digital forensics? Provide an example scenario.

Recuva is important in digital forensics because it allows investigators to recover deleted files that may contain evidence. For example, a suspect may delete documents or logs to hide activity, but those files can often be recovered and analyzed using tools like Recuva.

6. Conclusion

This lab demonstrated that deleting a file does not permanently erase it from a system. Using Recuva, a deleted file was successfully recovered, opened, and verified. The lab reinforced the importance of forensic recovery tools in investigations and highlighted how deleted data can still provide valuable evidence.

7. Challenges and Observations

- Files that have not been overwritten are easier to recover.
- Saving recovered files to a different location is critical to avoid data corruption.
- Even simple tools like Recuva can be effective in forensic investigations.

8. References

- Piriform. *Recuva File Recovery Software*.
- NIST SP 800-86 – Guide to Integrating Forensic Techniques into Incident Response
- Course materials and lab instructions provided by the instructor

Course: NWIT 263 - Digital Forensics

Lab Number: 02

Lab Title: Kali Linux Installation

Professor: Reza Mirabrishami

Date: 9/11/2025

1. Introduction

This lab focuses on installing a Kali Linux virtual machine

2. Lab Objectives

By completing this lab, I was able to:

Install Kali Linux in a virtual environment and verify the installation with basic commands. By the end of this exercise, I will have a Kali Linux system working inside VirtualBox (or VMware).

3. Tools and Requirements

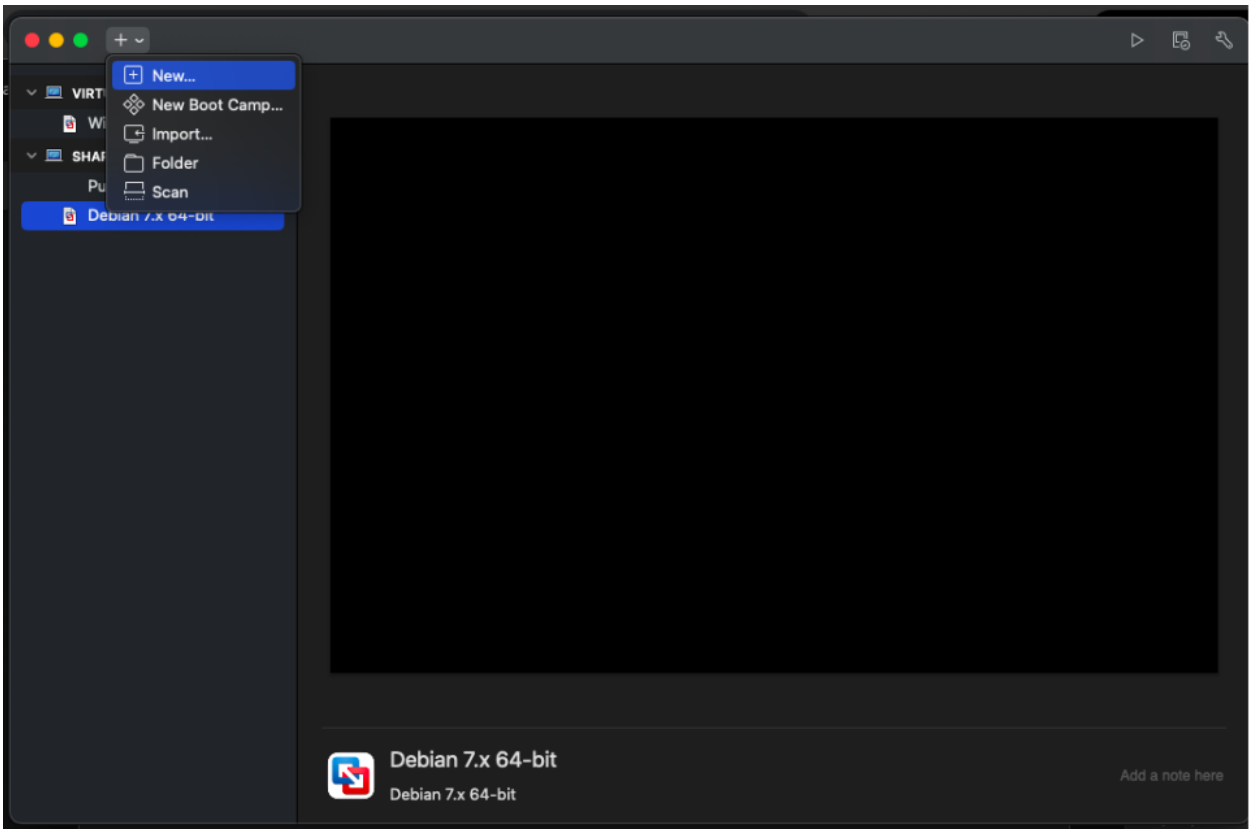
Download the 64-bit Installer ISO - Visit: https://www.kali.org/get-kali/
Install VMware Fusion - https://www.vmware.com/products/desktop-hypervisor/workstation-and-fusion

4. Lab Steps and Execution

Step 1: Downloaded the 64-bit Installer ISO and Installed VMware Fusion

Step 2: Created a Virtual Machine

- Opened VMware Fusion → Clicked New.
- Configured the VM as follows:
- Name: Kali Linux
- Type: Linux
- Version: Debian (64-bit)
- Memory (RAM): 4 GB
- Hard Disk: 20 GB



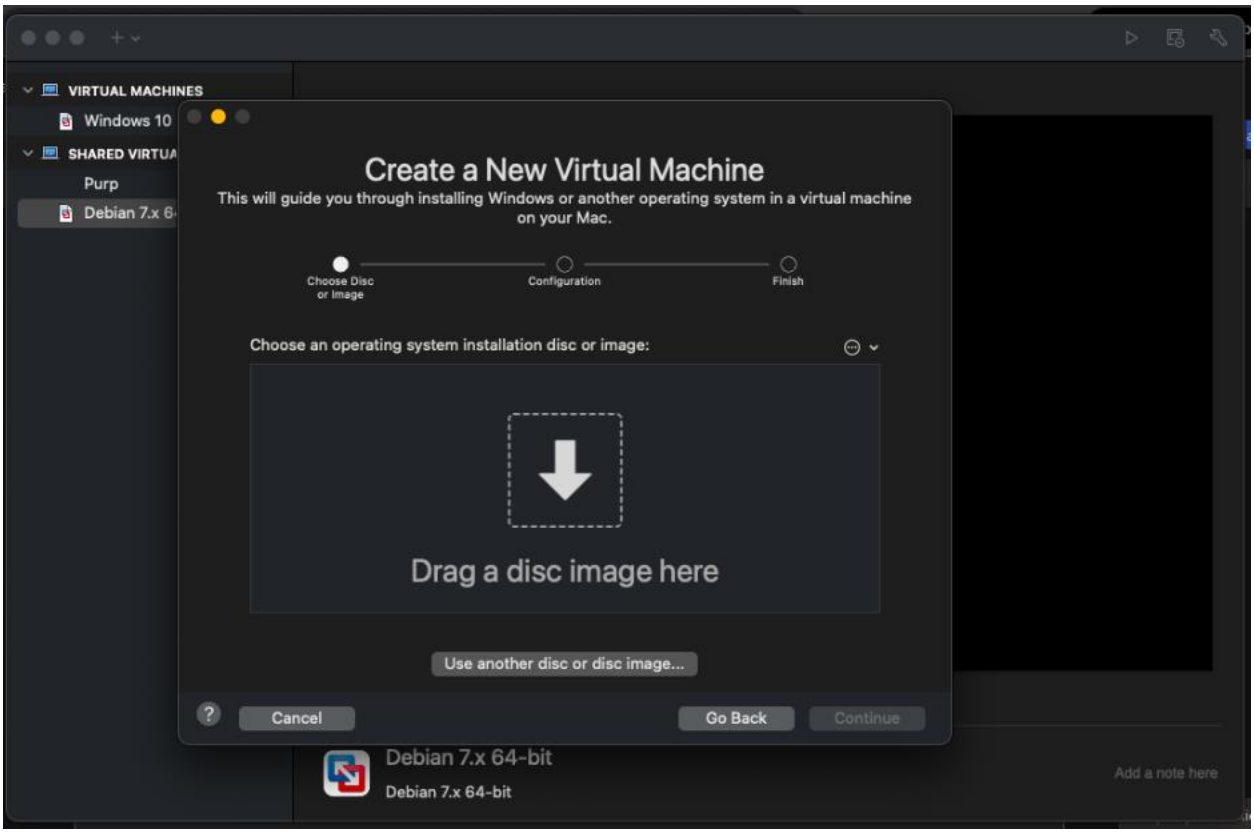
Step 3: Install Kali Linux

Started the VM and selected the downloaded ISO file as startup disk.

Choose Graphical Install.

Followed the on-screen instructions:

- Selected language, location, and keyboard.
- Created username and password.
- Partitioned disk → Choose Guided – Use Entire Disk.
- Waited for the installation to finish.

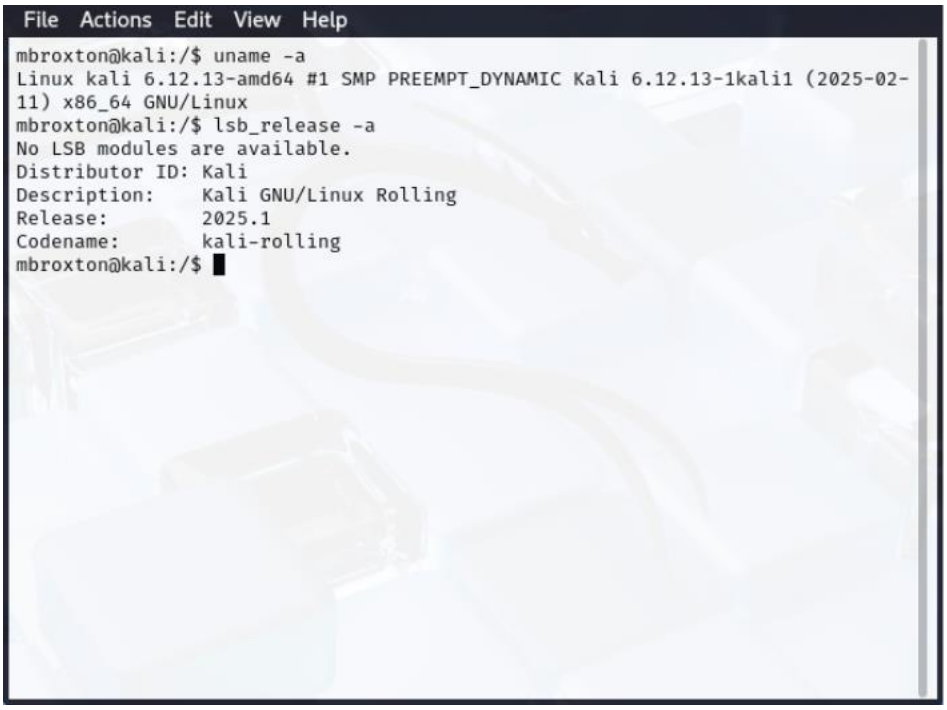


Step 4: Verify Installation

Logged in with my username and password.

Opened the terminal and typed:

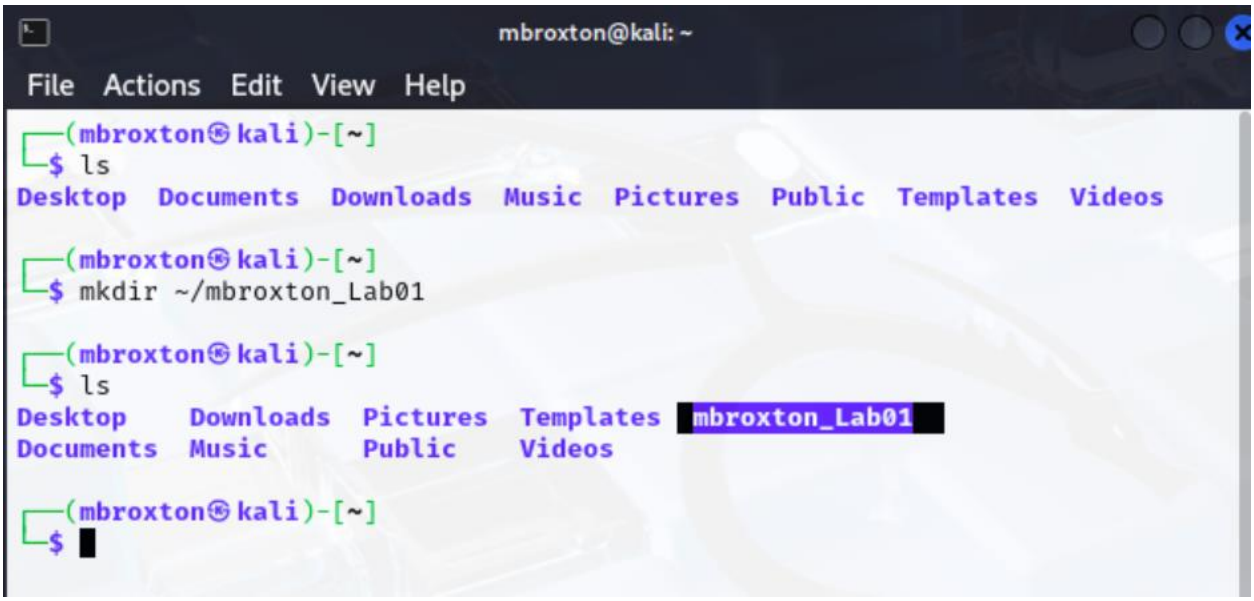
- uname –a
- lsb_release –a



Step 5: Personalize Your System

In the terminal, created a folder with my name:

- mkdir ~/mbroxton_Lab01



5. Conclusion

This lab successfully demonstrated the process of installing Kali Linux in a virtual environment using VMware Fusion. I was able to download the installer ISO, create a virtual machine, and complete the installation with the recommended system configurations. Next, I verified the installation with terminal commands and personalized the system. I confirmed that the Kali Linux environment was working properly. This lab was a foundational step in preparing a workspace for digital forensics.

6. Challenges and Observations

- Running uname -a and lsb_release -a confirmed the system was installed correctly. It was a good observation to see the system information
- Partitioning the disk and completing the graphical installation required careful attention
- Adjusting VM resources such as RAM and disk space was important

7. References

Kali ISO - Visit: https://www.kali.org/get-kali/

VMware Fusion - https://www.vmware.com/products/desktop-hypervisor/workstation-and-fusion

Course: NWIT 263 – Digital Forensics

Lab Number: 03

Lab Title: Hashing Lab – Windows & Linux

Professor: Reza Mirabrishami

Date: 9/19/2025

1. Introduction

This lab focuses on hashing files within Windows and Linux. The purpose is to understand how hashing ensures file integrity, recognize the differences between hash algorithms, and see how they are applied in digital forensics to confirm authenticity of evidence.

2. Lab Objectives

- Generate hashes of files in both Windows and Linux.
- Compare different types of hash functions (MD5, SHA-1, SHA-256).
- Verify that identical files produce the same hash and altered files do not.
- Document findings for forensic reporting.

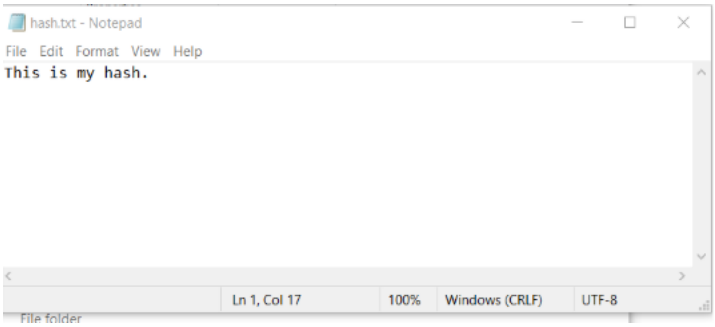
3. Tools and Requirements

- **Operating Systems:** Windows 10, Linux (Kali)
- **Windows Tool:** CertUtil (built-in utility)
- **Linux Tool:** sha256sum, md5sum, sha1sum

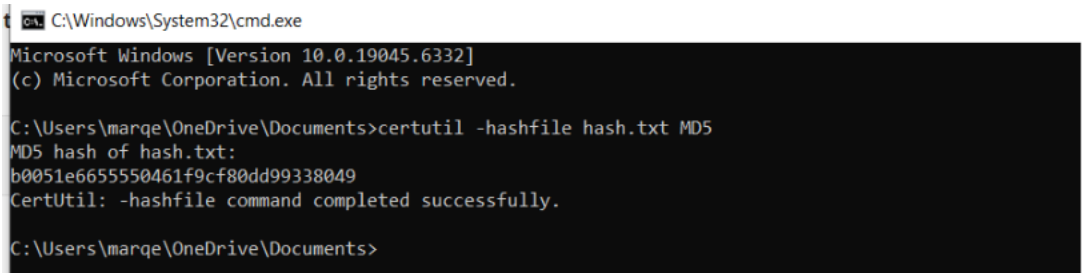
4. Lab Steps and Execution

Part 1 – Windows (Using CertUtil):

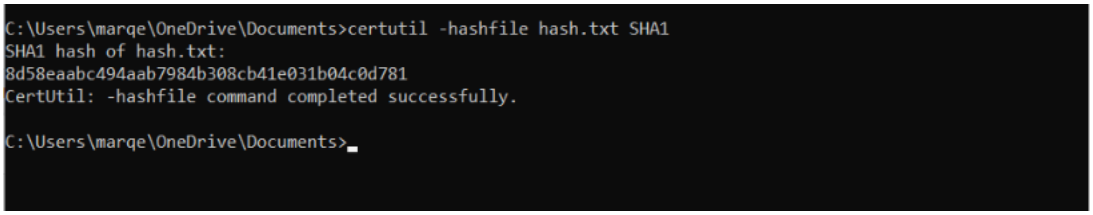
1. Created and saved a test file named hash.txt in Documents.



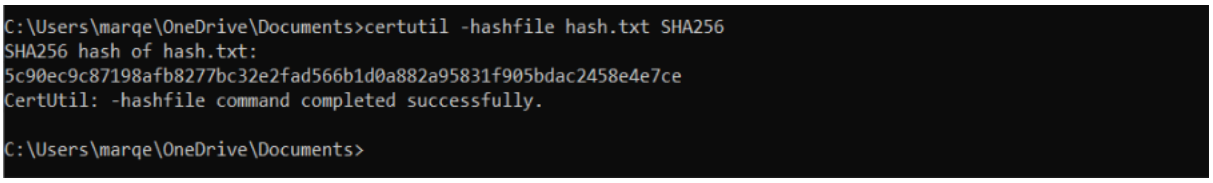
2. Opened Command Prompt and navigated to the Documents directory.
3. Ran the following commands:
   - certutil -hashfile hash.txt MD5



   - certutil -hashfile hash.txt SHA1



   - certutil -hashfile hash.txt SHA256



- certutil -hashfile hash.txt SHA512

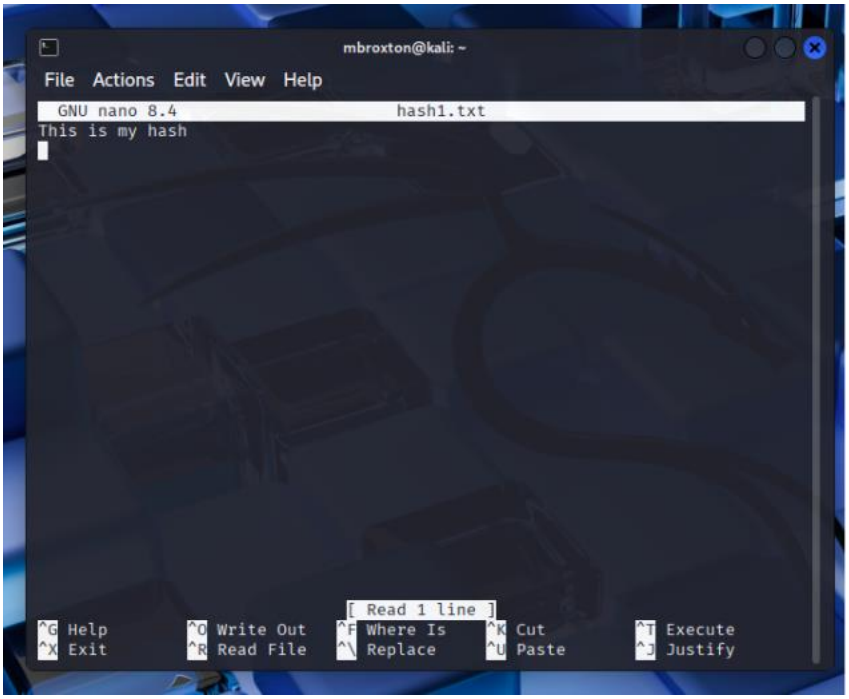
```
C:\Users\marqe\OneDrive\Documents>certutil -hashfile hash.txt SHA512
SHA512 hash of hash.txt:
8eca285136f51cbbbc084da954b50a7e06a7f8a085c89c6a340606010d5ff2eb016e3e1a96e90af0380edc26d50c6d660cc8ac351035159e727c06d4
53a8324f
CertUtil: -hashfile command completed successfully.

C:\Users\marqe\OneDrive\Documents>
```

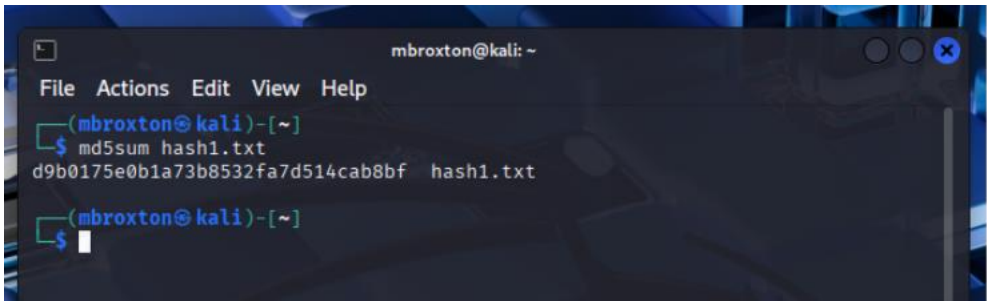
4. Observed that each command generated a unique hash value for the file depending on the algorithm.

Step 2: Linux (Using Built-in Hash Commands)

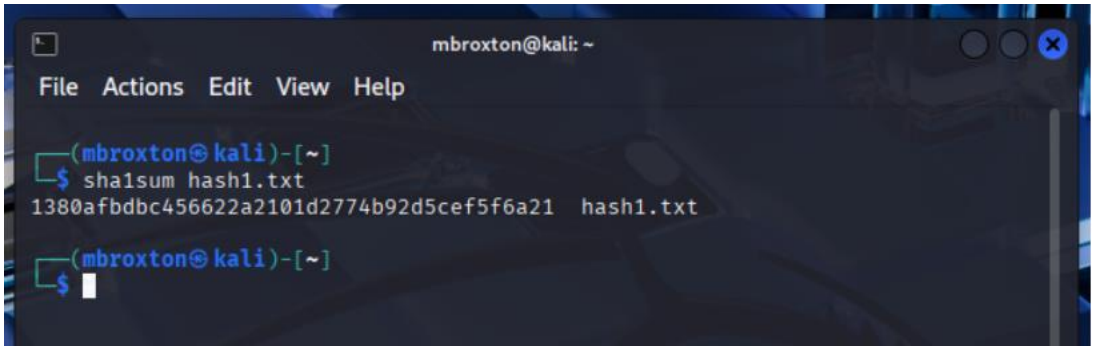
1. Created a file named hash1.txt in the Linux home directory



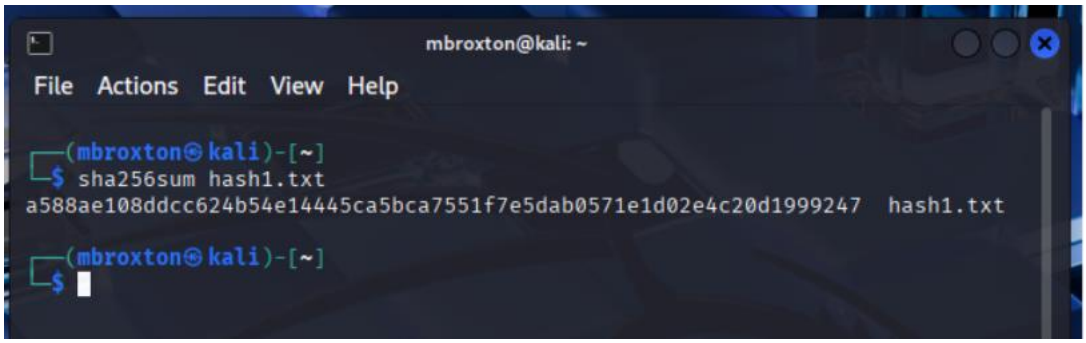
2. Opened terminal and ran:
   - md5sum hash1.txt



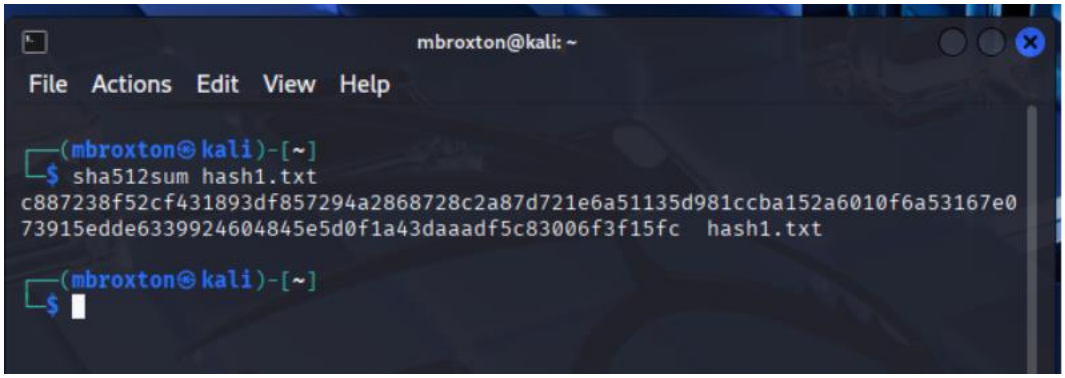
- sha1sum hash.txt



- sha256sum hash.txt



- sha512sum hash.txt



3. Edited the file slightly (removed period at the end). The new hash values were completely different, demonstrating the sensitivity of hashing to file changes.

4. Verified the integrity of the original file.

5. Conclusion

This lab demonstrated how hashing is used to ensure data integrity across Windows and Linux. Identical files consistently produced the same hash, while even minor alterations resulted in entirely different hashes. In digital forensics, hashing is crucial for validating evidence, ensuring files have not been altered during investigations or transfers.

6. Challenges and Observations

Remembering the exact syntax for CertUtil required careful attention.
Linux commands were straightforward but required checking file path accuracy.
Even a small edit (adding one character) drastically changed the hash, proving the reliability of hashing for tamper detection.
Using multiple algorithms reinforced the idea that while MD5 and SHA-1 are fast, SHA-256 is more secure.

7. References

Microsoft Docs – CertUtil: https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/certutil
Linux man pages (man sha256sum, man md5sum)

Course: NWIT 263 – Digital Forensics
Lab Number: 04

Lab Title: Using FTK Imager to Create, Mount, and Dismount a Forensic Image

Professor: Reza Mirabrishami

Date: 9/19/2025

1. Introduction

This lab focuses on using FTK Imager to create a forensic image of a storage device, mount the image for analysis, and properly dismount it afterward. Imaging ensures that investigators work with a verified copy of evidence rather than altering the original. Mounting allows forensic analysts to browse and analyze files in a controlled environment.

2. Lab Objectives

By completing this lab, I was able to:

- Create a forensic image of a USB drive using FTK Imager.
- Save the image in E01 (Expert Witness Format) with proper fragment size.
- Generate MD5/SHA1 hashes to verify image integrity.
- Mount the forensic image in read-only mode.
- Browse and analyze the mounted image.
- Properly dismount the image to maintain forensic soundness.

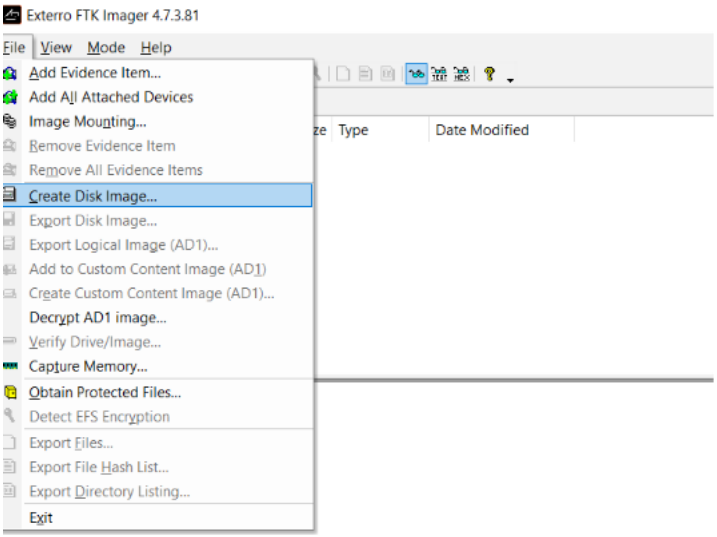
3. Tools and Requirements

- **Operating System:** Windows 10
- **Software:** FTK Imager (AccessData)
- **Storage Device:** USB drive (used as source)
- **Destination Folder:** B:\Forensic_Images\

4. Lab Steps and Execution

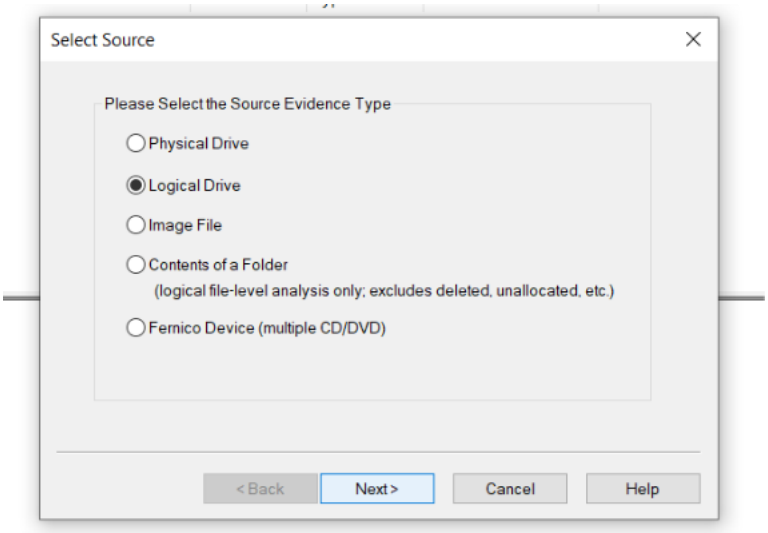
Step 1: Launch FTK Imager

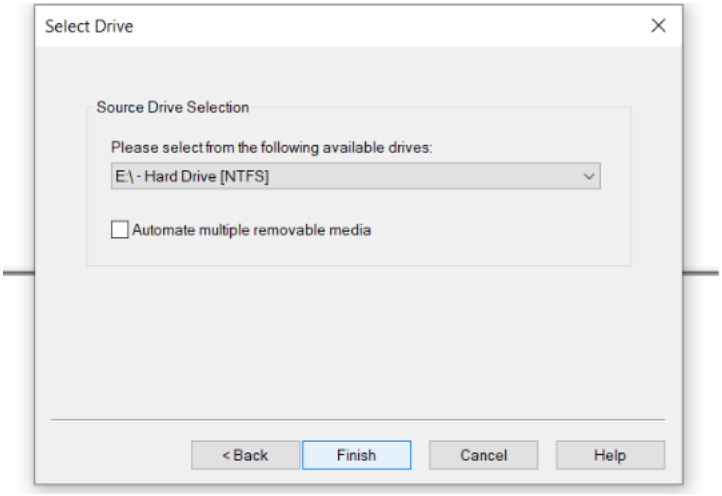
- Opened FTK Imager as Administrator.
- Selected File → Create Disk Image.



Step 2: Select Source Drive

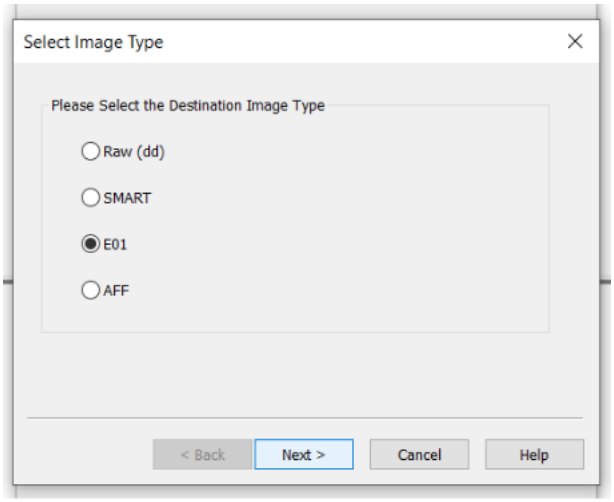
- Chose Logical Drive and selected my USB drive.
- Clicked Finish to confirm the source.



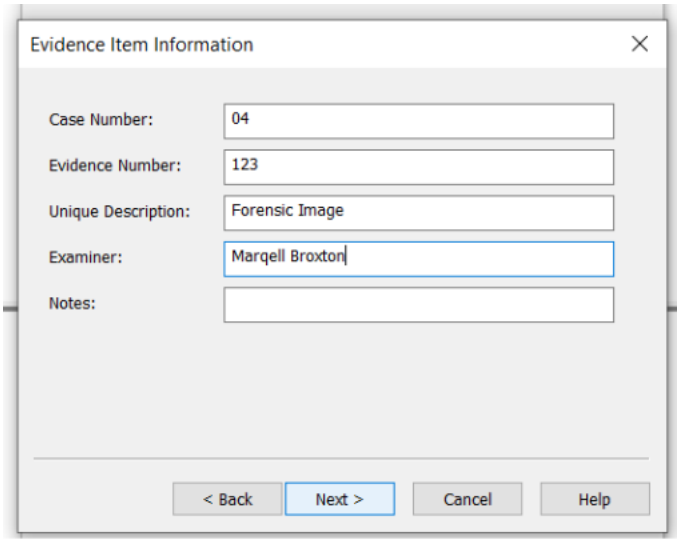


Step 3: Select Image Type and Destination

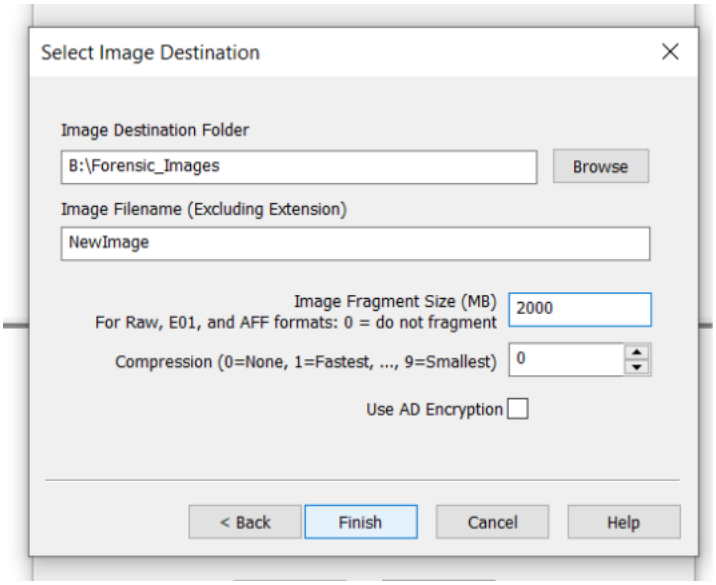
- Selected E01 (EWF) format.



- Entered case information (Case Number: 04, Examiner: Marqell Broxton).

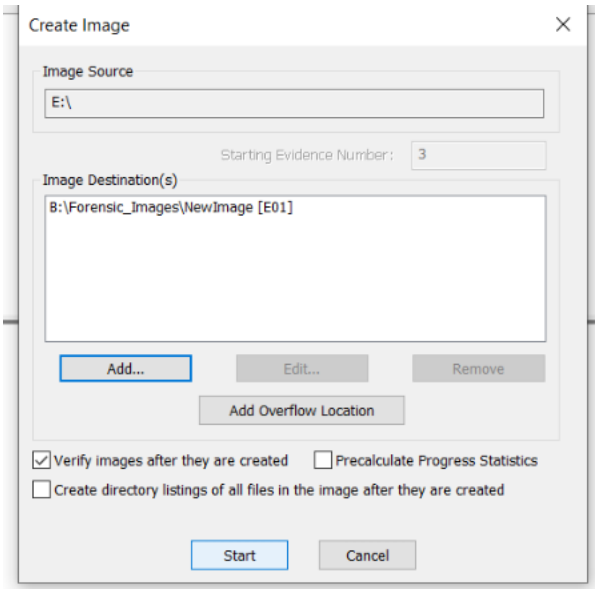


- Saved the image as NewImage.E01 in B:\Forensic_Images.
- Set fragment size to 2000 MB.
- Enabled MD5 and SHA1 hashing for verification.

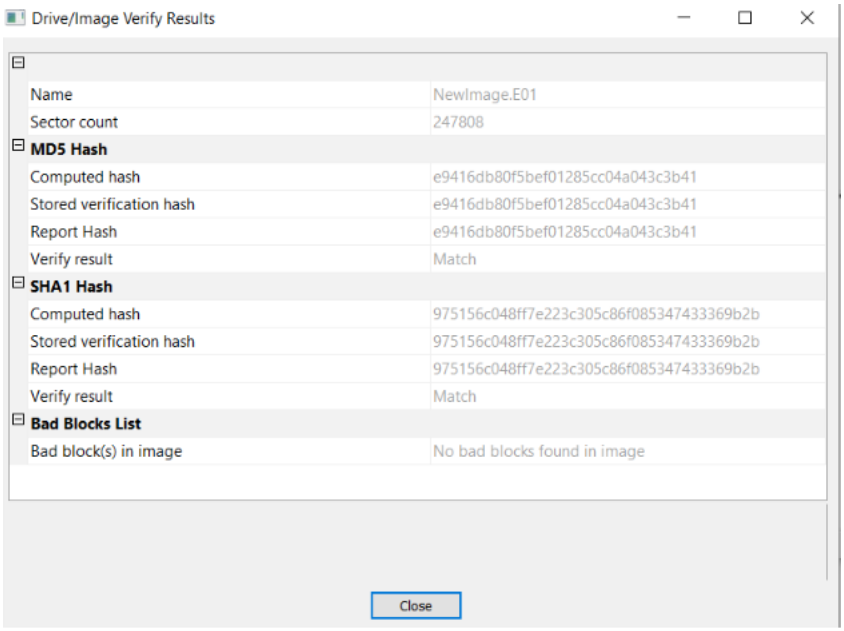


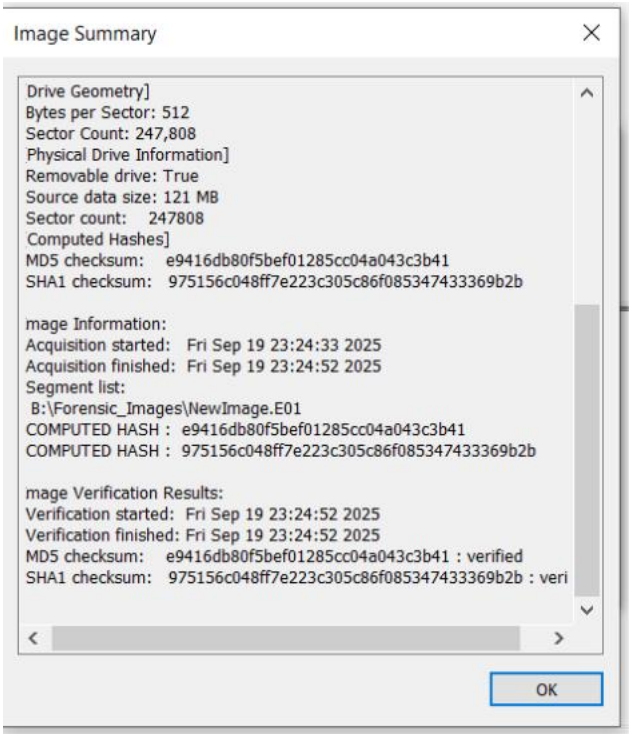
Step 4: Start the Imaging Process

- Clicked Start. FTK imaged the USB drive and generated MD5/SHA1 hashes.



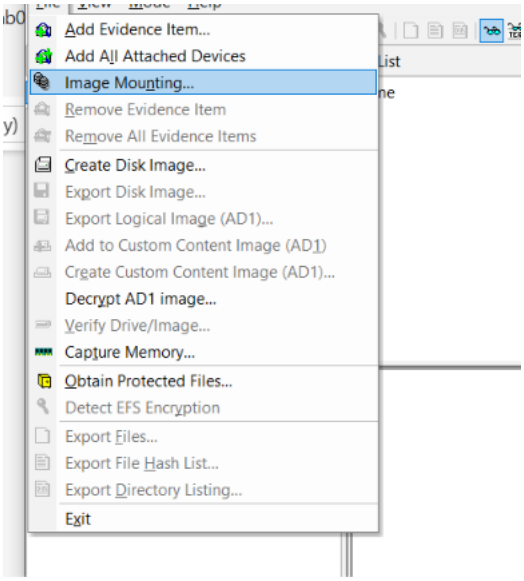
- Verified that the source hash and image hash matched.



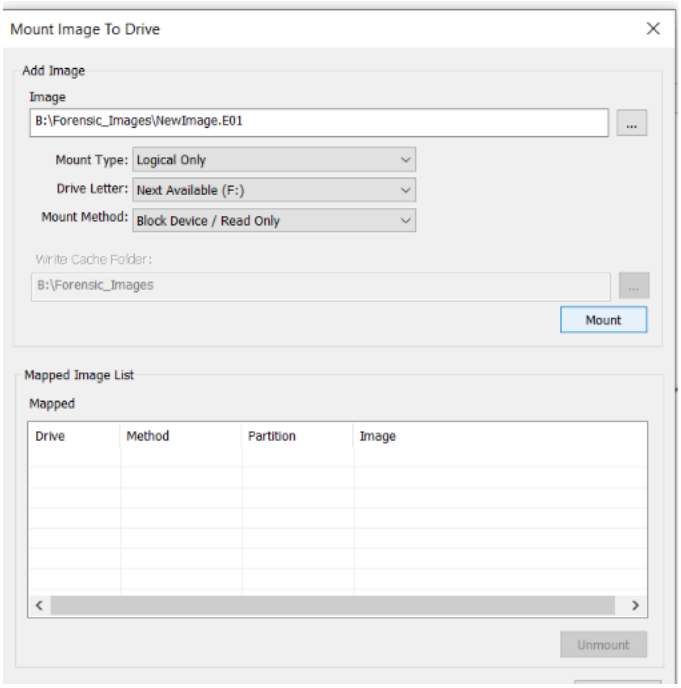


Step 5: Mount the Image

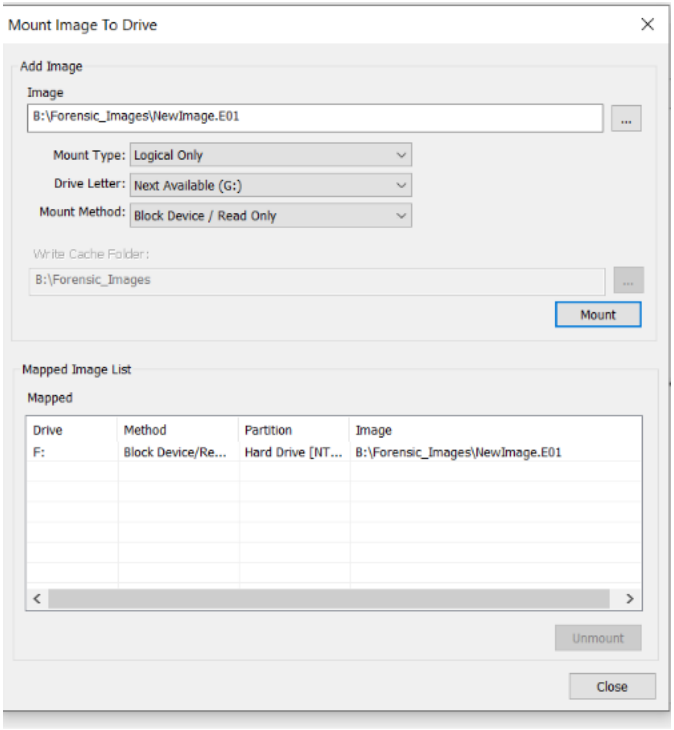
- Selected File → Image Mounting.



- Chose the newly created forensic_image.e01.



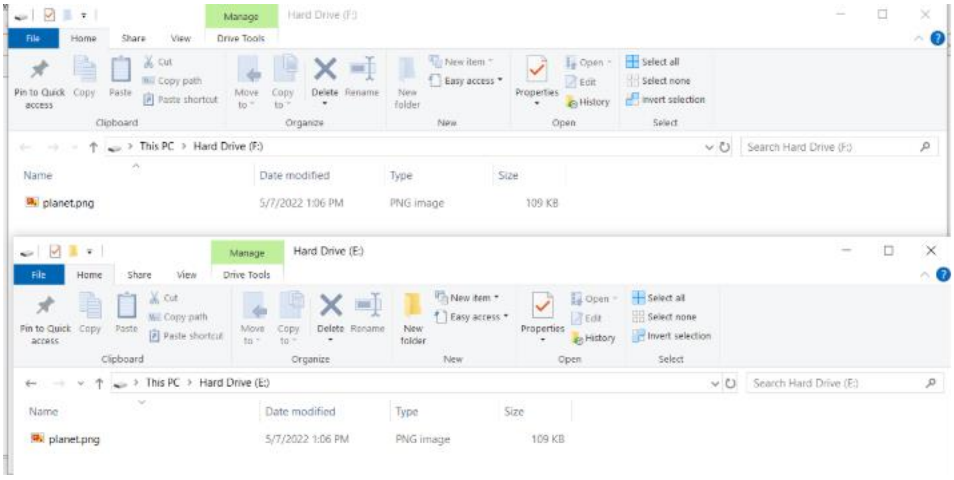
- Mounted the image as Read-Only with the drive letter F:.



- Opened File Explorer and confirmed the mounted drive was visible.

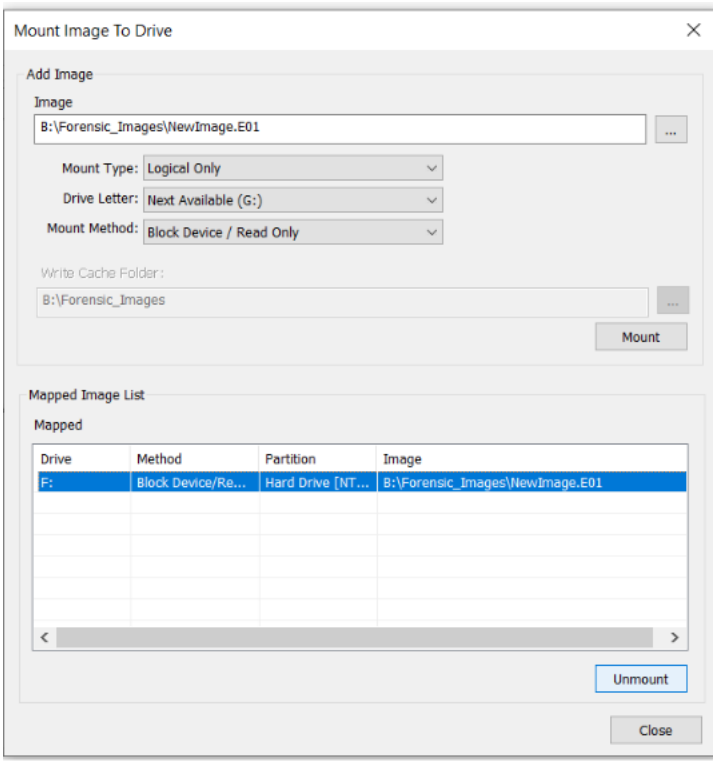
Step 6: Access the Mounted Image

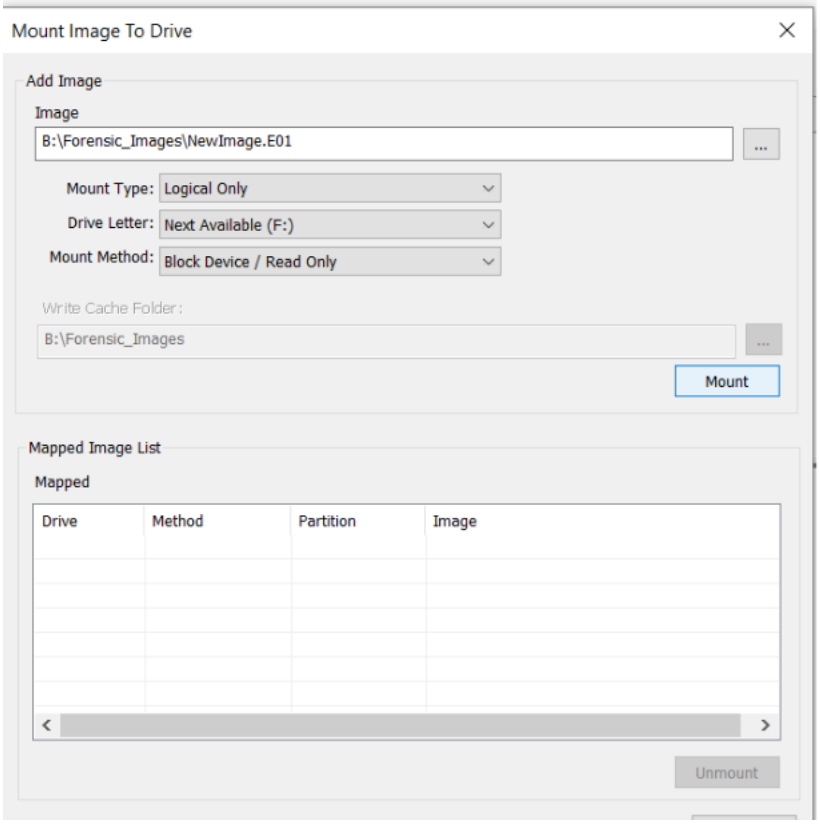
- Browsed the contents of the mounted drive through File Explorer.
- Verified files matched the original USB contents.



Step 7: Dismount the Image

- Returned to File → Image Mounting in FTK.
- Selected the mounted image and clicked Dismount.
- Confirmed the dismount; the drive letter disappeared from File Explorer.


Mount Image To Drive
Add Image
Image
B:\Forensic_Images\NewImage.E01
...
Mount Type: Logical Only
Drive Letter: Next Available (G:)
Mount Method: Block Device / Read Only
Write Cache Folder:
B:\Forensic_Images
...
Mount
Mapped Image List
Mapped
Drive
Method
Partition
Image
F:
Block Device/Re...
Hard Drive [NT...
B:\Forensic_Images\NewImage.E01
Unmount
Close


Mount Image To Drive
Add Image
Image
B:\Forensic_Images\NewImage.E01
...
Mount Type: Logical Only
Drive Letter: Next Available (F:)
Mount Method: Block Device / Read Only
Write Cache Folder:
B:\Forensic_Images
...
Mount
Mapped Image List
Mapped
Drive
Method
Partition
Image
Unmount

5. Conclusion

This lab demonstrated the use of FTK Imager to create, mount, and dismount forensic images. The process confirmed that FTK produces verified forensic copies using hashing to maintain evidence integrity. Mounting the image allowed access in read-only mode, ensuring the original evidence was not altered. Dismounting completed the process safely, simulating real-world forensic handling of digital evidence.

6. Challenges and Observations

- The progress bar sometimes did not update immediately; this was resolved by waiting for FTK to initialize.
- Remembering to choose a fragment size under 4 GB was important for compatibility.
- Running FTK as Administrator was required to access the drive fully.
- Logical imaging of small files (3 GB) was extremely fast compared to physical imaging of an entire disk.

8. References

- AccessData FTK Imager User Guide
- NIST Computer Forensics Tool Testing Program
- Microsoft Docs: File Systems and Hashing

Q&A (based on lab)

1. **What image format did you use?**
   E01 (Expert Witness Format).
2. **What hash algorithms were used to verify the image?**
   MD5 and SHA1.
3. **What drive letter was assigned when mounting the image?**
   F: (Read-Only).
4. **Why is FTK Imager important in digital forensics? Provide an example.**
   FTK Imager is important because it allows forensic investigators to create verified, tamper-proof copies of digital evidence. For example, if investigators seize a suspect's USB drive, they can create an E01 image with FTK, verify it with hashes,

and analyze the mounted copy while leaving the original untouched for court evidence.

Course: NWIT 263 – Digital Forensics

Lab Number: 05

Lab Title: Enabling and Disabling USB Write Protection in Windows

Professor: Reza Mirabrishami

Date: 9/26/2025

1. Introduction

This lab focused on enabling and disabling USB write protection in Windows using the Registry Editor. The objective was to simulate the functionality of a write blocker, preventing data from being altered on a USB drive. Write blocking is an essential forensic practice that ensures evidence cannot be tampered with while being accessed.

2. Lab Objectives

- By completing this lab, I was able to:
- Enable write protection on a USB drive via the Windows Registry.
- Confirm that write attempts were blocked.
- Disable write protection to restore normal write access.

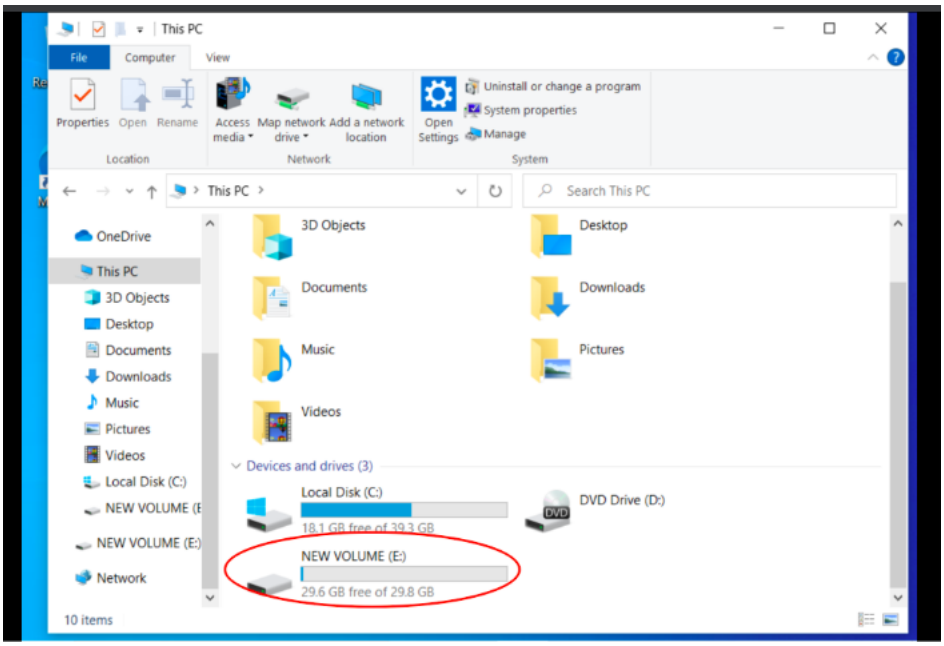
3. Tools and Requirements

- **Operating System:** Windows 10
- **Hardware:** USB flash drive
- **Software:** Windows Registry Editor (regedit)

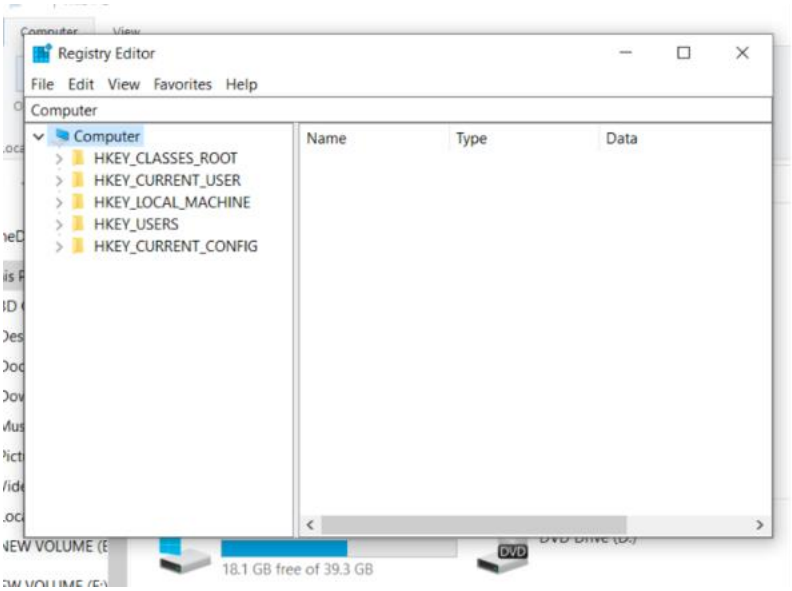
4. Lab Steps and Execution

Part 1 – Enable USB Write Protection

1. Inserted USB flash drive.

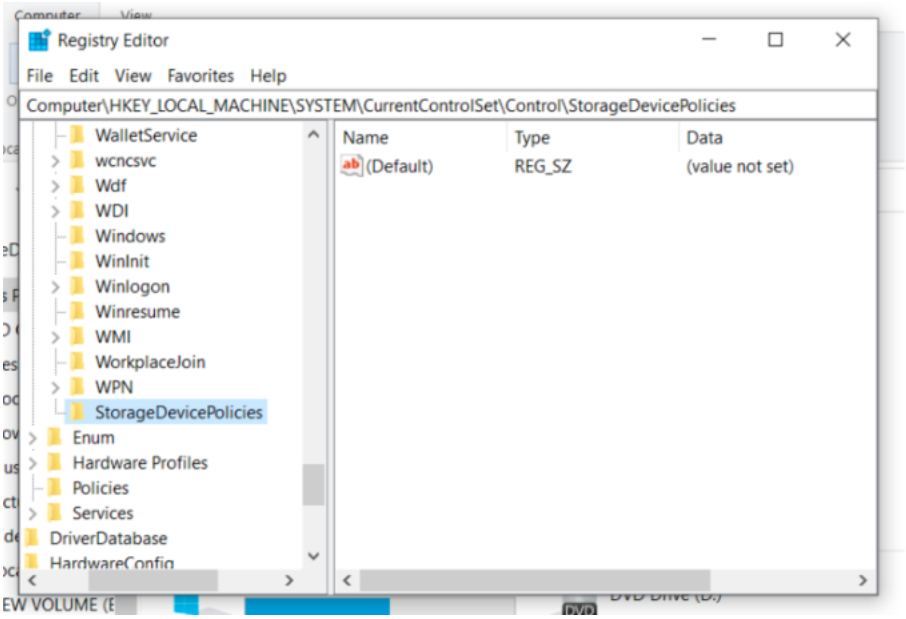


2. Opened **Registry Editor** (regedit).

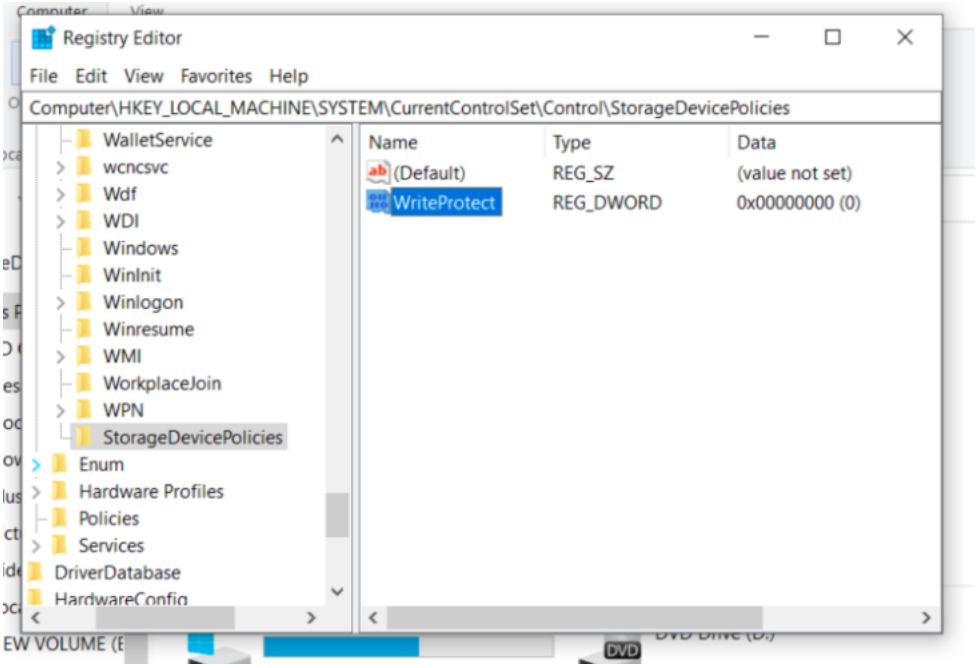


3. Navigated to:

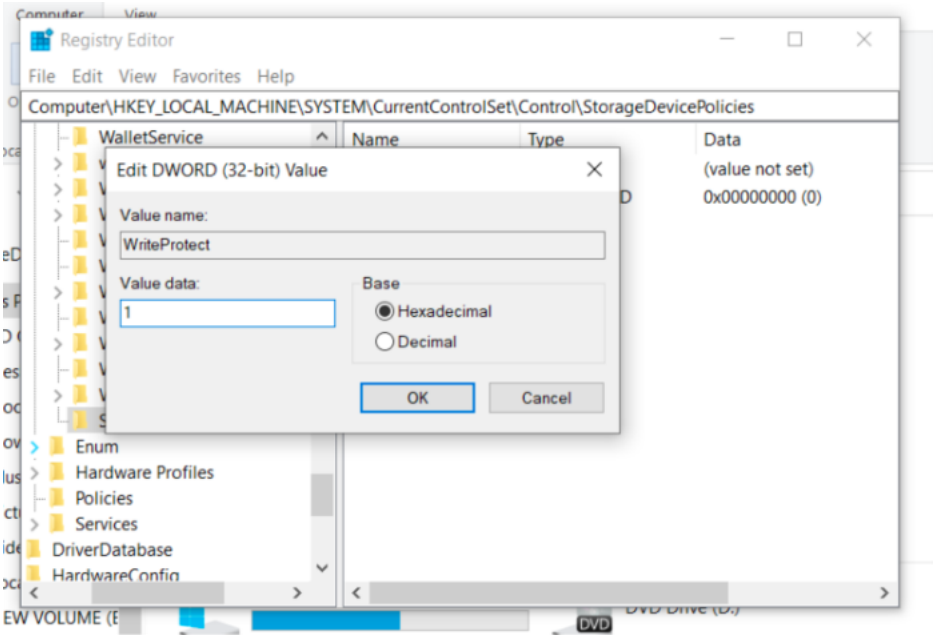
   HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\StorageDevicePolicies



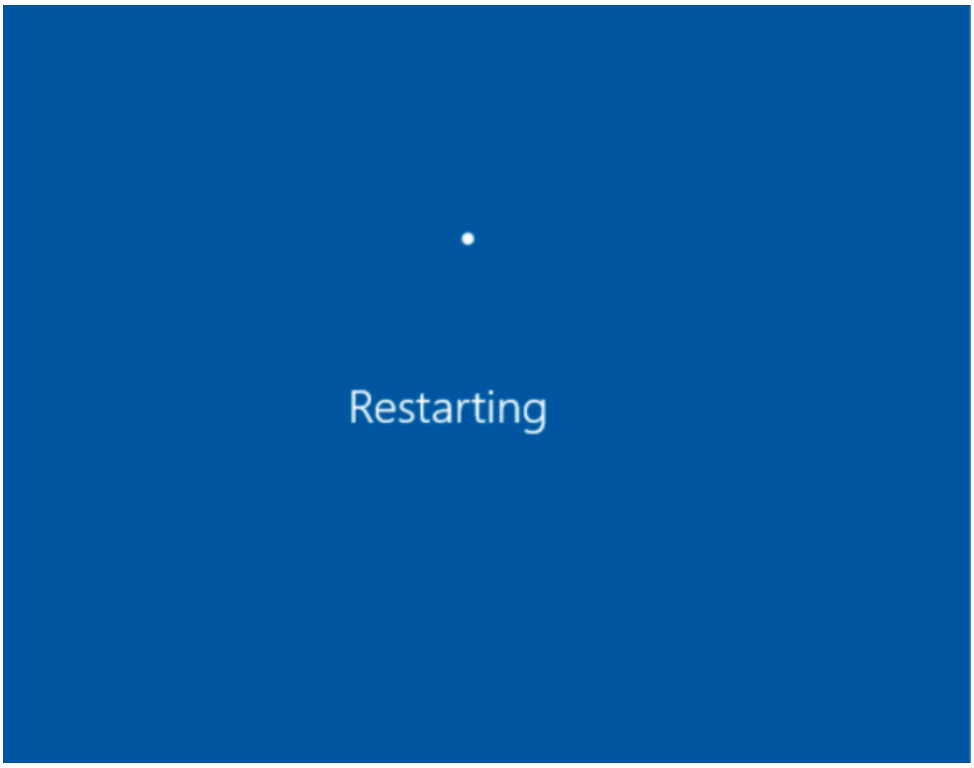
4. Created a new key (StorageDevicePolicies) and DWORD value WriteProtect if not already present.



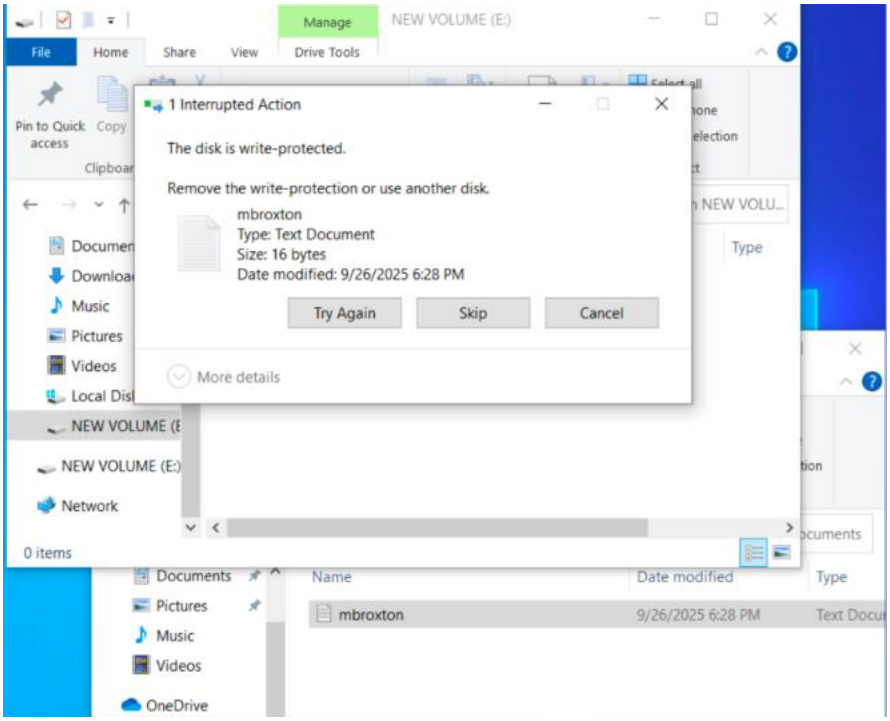
5. Set WriteProtect = 1 to enable write protection.



6. Restarted the computer.

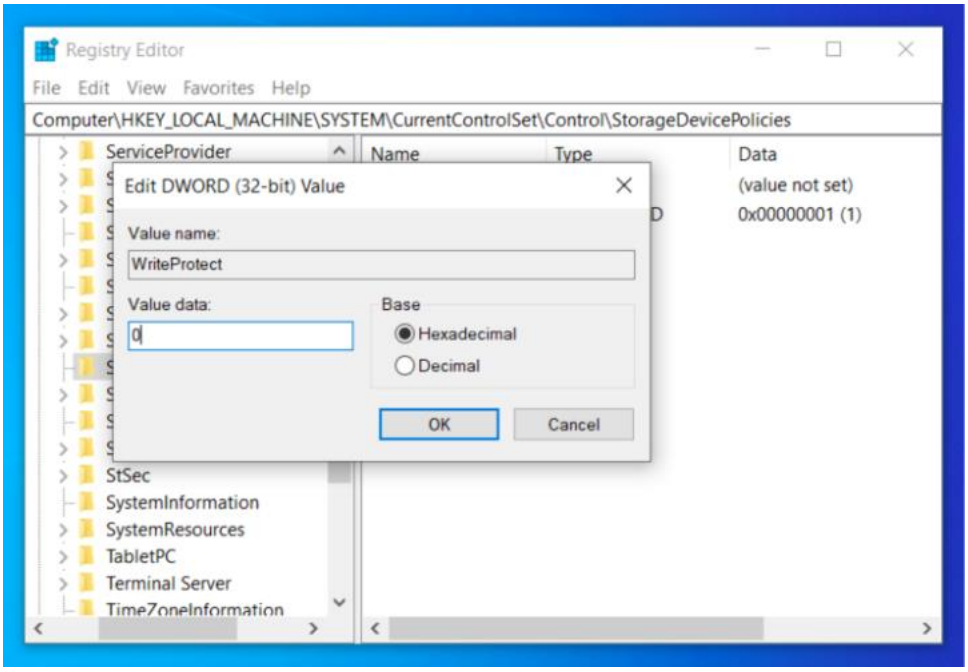


7. Attempted to copy a file to the USB drive. An error appeared indicating the disk was write-protected.

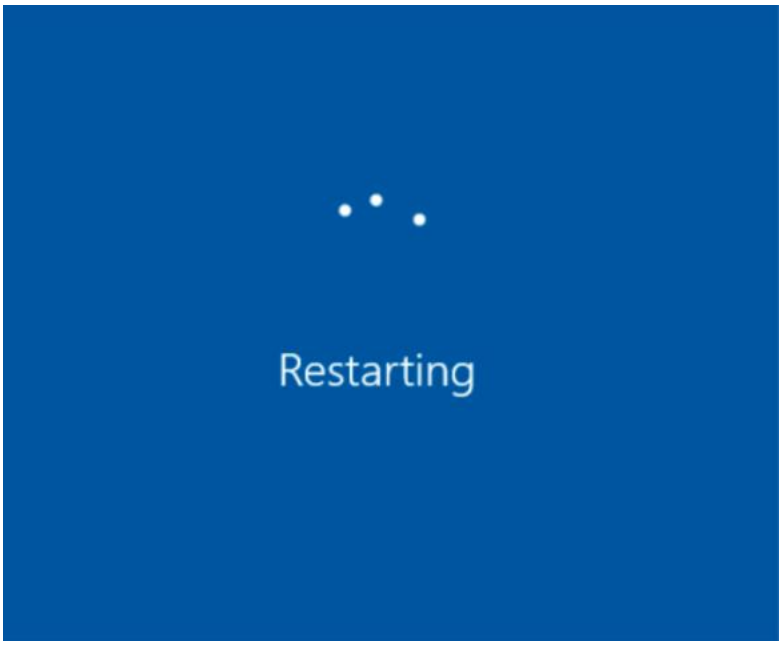


Part 2 – Disable USB Write Protection

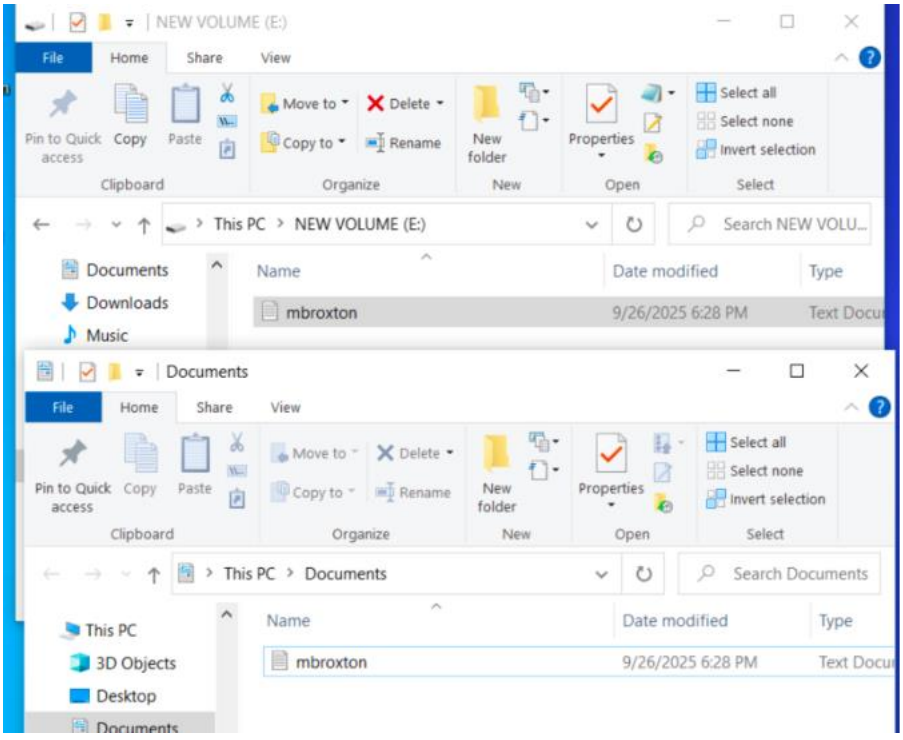
1. Reopened **Registry Editor**.
2. Navigated to the same location.
3. Set WriteProtect = 0 to disable write protection.



4. Restarted the computer.



5. Attempted to copy a file to the USB drive again. This time the file copied successfully.



5. Conclusion

This lab demonstrated how to use the Windows Registry Editor to simulate a write blocker. Enabling write protection prevented new files from being written to the USB device, while disabling it restored normal operation. It also showed how software-based write protection works at the system level.

6. Challenges and Observations

- The **StorageDevicePolicies** key did not exist initially and had to be created.
- A system restart was necessary for changes to take effect.
- The error message when attempting to write clearly indicated protection was working.
- Unlike hardware write blockers, this method can be bypassed if someone modifies the registry again.

7. References

- Microsoft Documentation: Registry Editor Overview
- Course Handout: Write Blocker Lab Instructions

Course: NWIT 263 – Digital Forensics
Lab Number: 06

Lab Title: Extracting and Mapping GPS Data
Professor: Reza Mirabrishami
Date: 10/03/2025

1. Introduction

This lab focused on extracting GPS metadata from digital photographs using ExifTool. Metadata embedded in photos can include location, time, and device details. Forensic investigators often use this information to determine where and when a photo was taken.

2. Lab Objectives

- Use ExifTool to extract metadata, including GPS coordinates, from an image.
- Convert GPS coordinates from Degrees-Minutes-Seconds (DMS) to decimal format.
- Use Google Maps to locate where the photo was taken.
- Demonstrate privacy awareness by removing EXIF metadata.

3. Tools and Requirements

- **Device Used:** Smartphone (photo source)
- **Computer OS:** Windows 10
- **Software:** ExifTool
- **Additional Tool:** Google Maps

4. Lab Steps and Execution

Step 1 – Enable Location Services and Take Photo

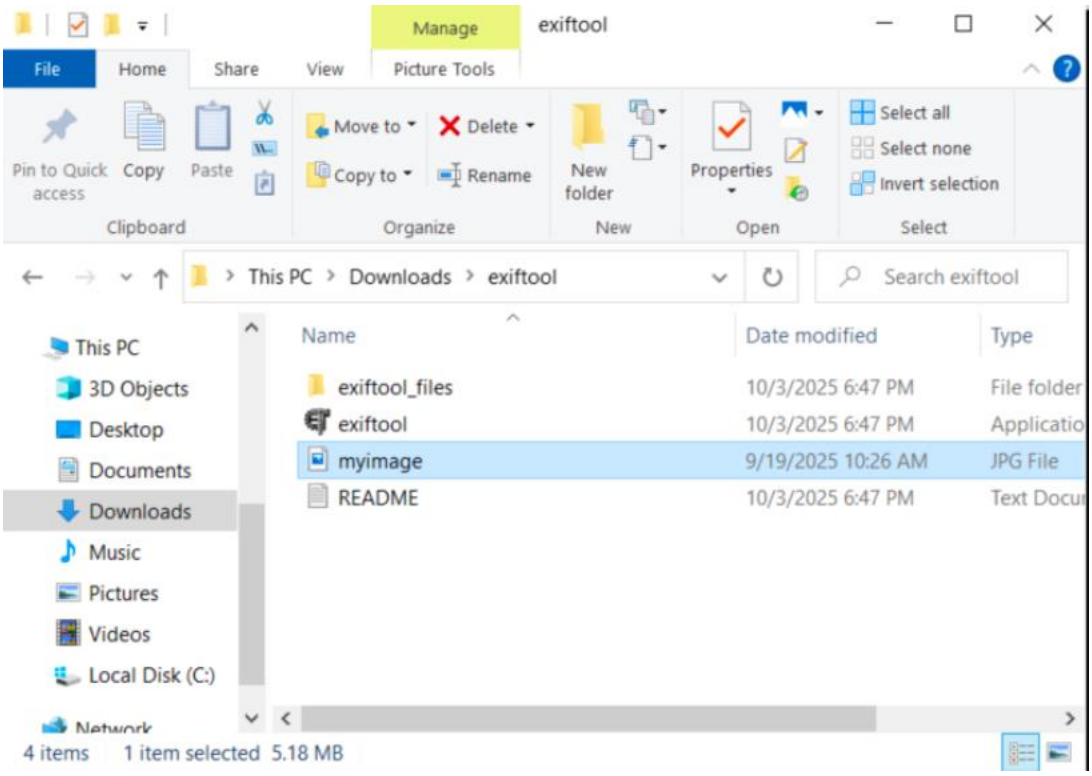
- Enabled Location Services on smartphone camera.
- Took a photo

Step 2 – Transfer Photo

- Transferred the photo to the computer via USB.

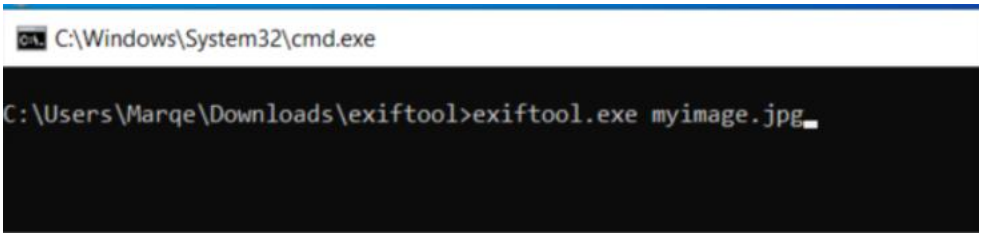
Step 3 – Install ExifTool

- Downloaded ExifTool for Windows.
- Renamed exiftool(-k).exe to exiftool.exe for easier use.
- Placed both ExifTool and the photo in the same folder.

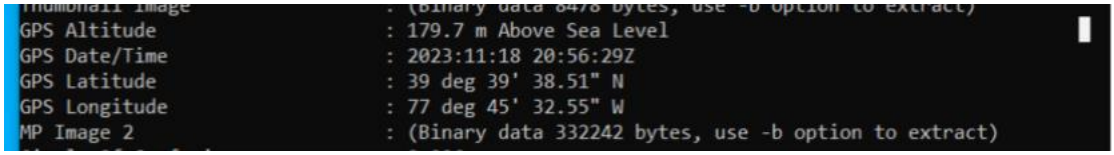


Step 4 – Extract Metadata

- Opened Command Prompt, navigated to the photo folder.
- Ran: exiftool.exe myimage.jpg



- Located **GPS Latitude** and **GPS Longitude** in the output.



Step 5 – Convert Coordinates

- GPS data was in DMS format.
- Converted to decimal degrees using:

Decimal = Degrees + (Minutes / 60) + (Seconds / 3600)

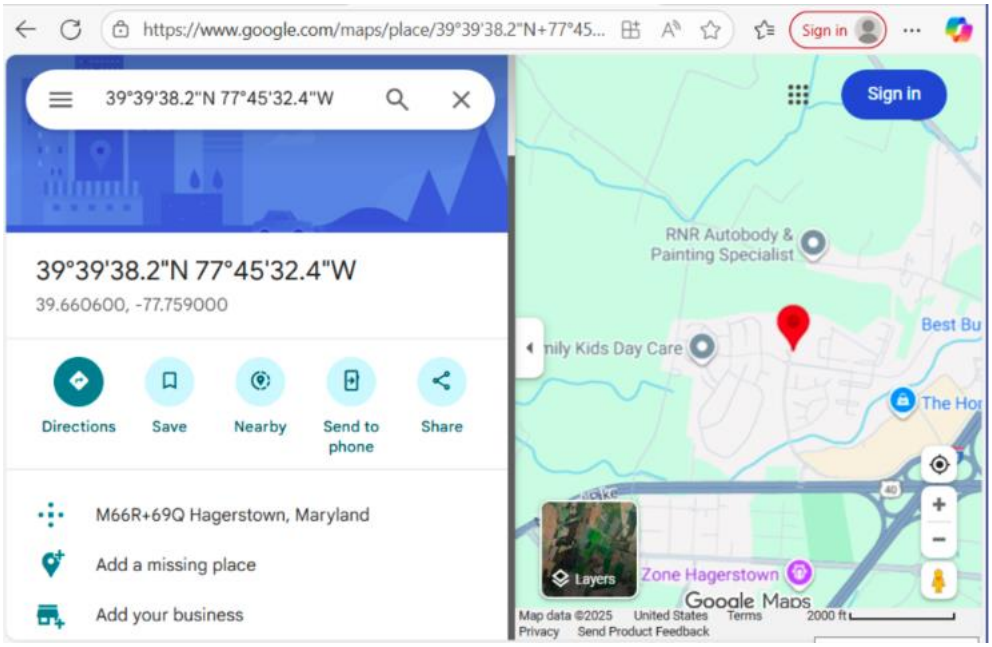
Latitude: 39 + (39 / 60) + (38.51 / 3600) = 39.6606972 N

Longitude: 77 + (45 / 60) + (32.55 / 3600) = -77.7590417 W

Changed the Longitude to a negative decimal since it is West.

Step 6 – Map the Coordinates

- Opened Google Maps.
- Entered converted coordinates: 39.6606972, -77.7590417
- Pin dropped on the correct location.

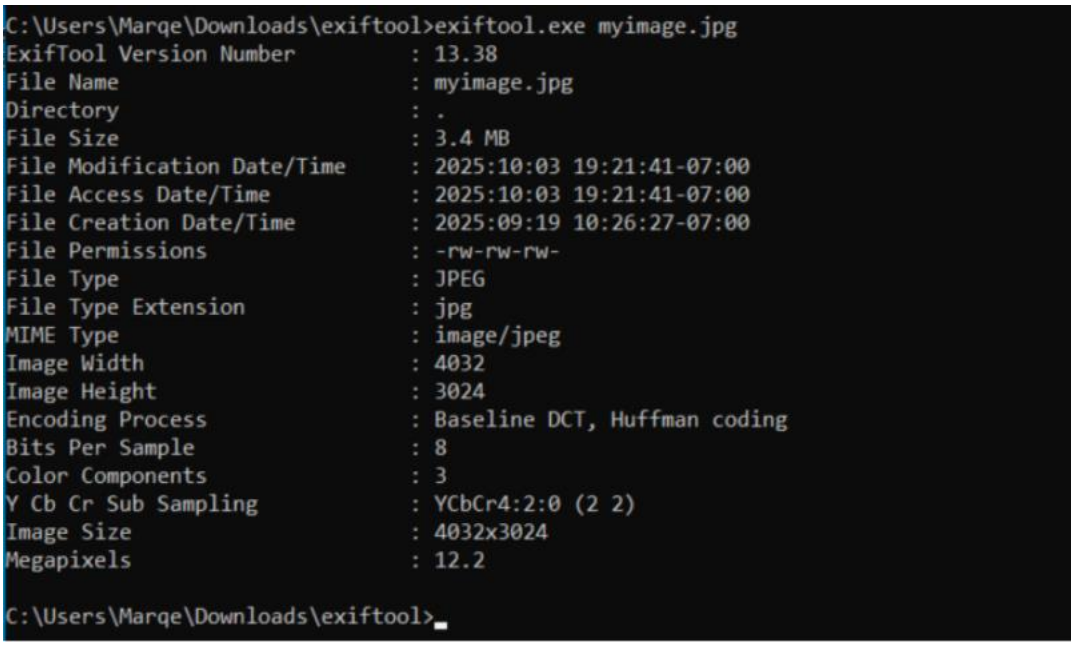


Step 7 – Privacy Awareness

- Removed all EXIF metadata by running:

  exiftool.exe -all= -overwrite_original myimage.jpg

  Re-ran ExifTool and confirmed GPS data was gone.



5. Conclusion

This lab demonstrated how ExifTool can be used to extract GPS metadata from images, convert coordinates to decimal format, and map locations in Google Maps. This process is useful in digital forensics, where photo metadata can provide crucial evidence of where and when a picture was taken. Additionally, the optional step highlighted privacy risks and showed how metadata can be stripped to protect personal information.

6. Challenges and Observations

- Initially, remembering the exact ExifTool command syntax required careful attention.
- Conversion from DMS to decimal format had to account for negative longitude values (west of the Prime Meridian).
- Google Maps accurately pinpointed the location using converted coordinates.
- The metadata removal step reinforced the importance of privacy awareness.

7. References

- ExifTool Official Website: https://exiftool.org/
- Google Maps: https://maps.google.com

Course: NWIT 263 – Digital Forensics
Lab Number: 07
Lab Title: Hashing Files on Windows Using Febooti Hash & CRC
Professor: Reza Mirabrishami
Date: 10/16/2025

1. Introduction

This lab focused on using the Febooti Hash & CRC utility to generate and compare hash values for files in Windows. The objective was to understand how file integrity can be verified by hashing and how even a small change in a file result in a completely different hash value.

2. Lab Objectives

- Generate MD5 and SHA-256 hash values for files using Febooti Hash & CRC.
- Verify file integrity by comparing original and modified file hashes.

- Demonstrate that any change to a file produces a new hash, confirming tamper detection in digital forensics.

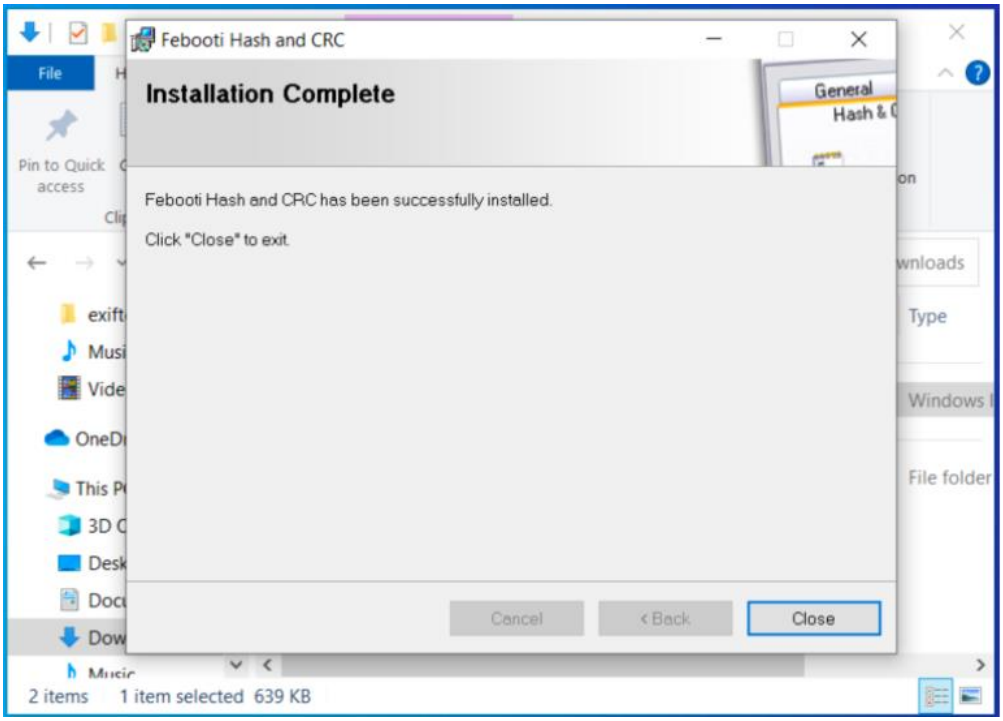
3. Tools and Requirements

- Operating System: Windows 10
- Software: Febooti Hash & CRC
- File Used: hash_test.txt

4. Lab Steps and Execution

Step 1 – Install Febooti Hash & CRC

1. Downloaded Febooti Hash & CRC from the official site: https://www.febooti.com/products/filetweak/members/hash-and-crc/.
2. Installed using the default setup wizard.
3. Confirmed installation by right-clicking on a file and verifying the “File Hash & CRC” context menu option appeared.

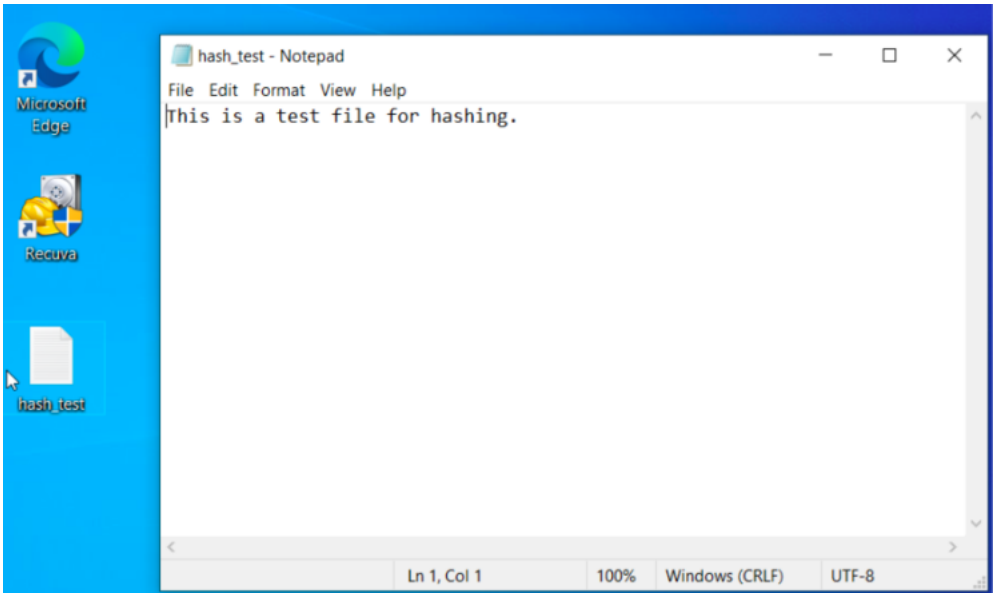


Step 2 – Generate Hash for Original File

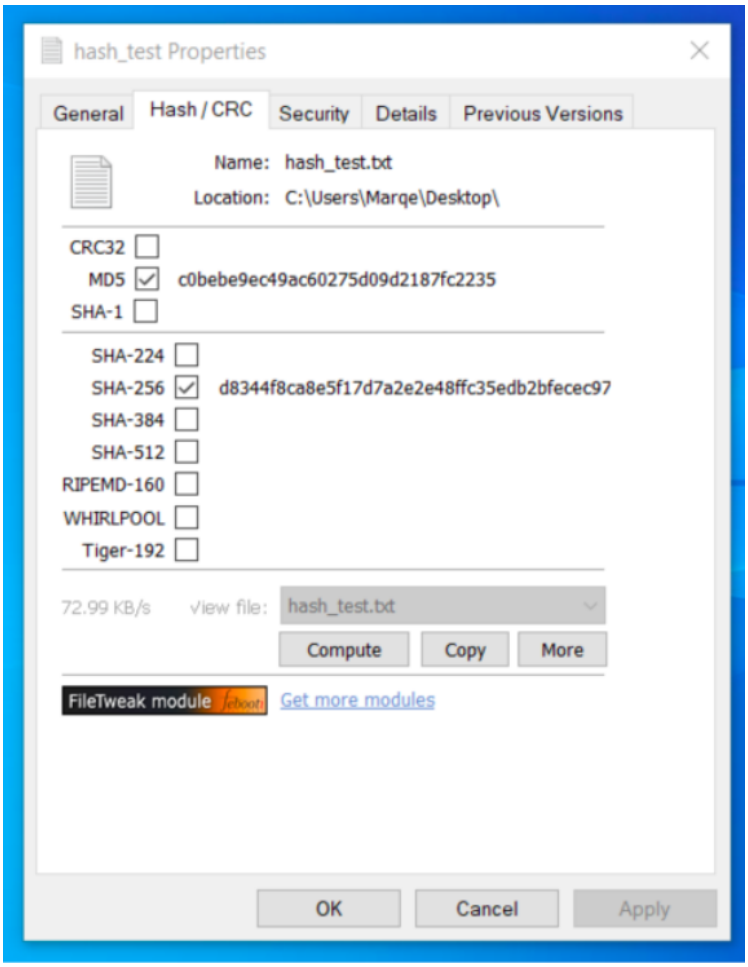
1. Opened Notepad and created a file with the text:

This is a test file for hashing.

2. Saved it as hash_test.txt on the Desktop.



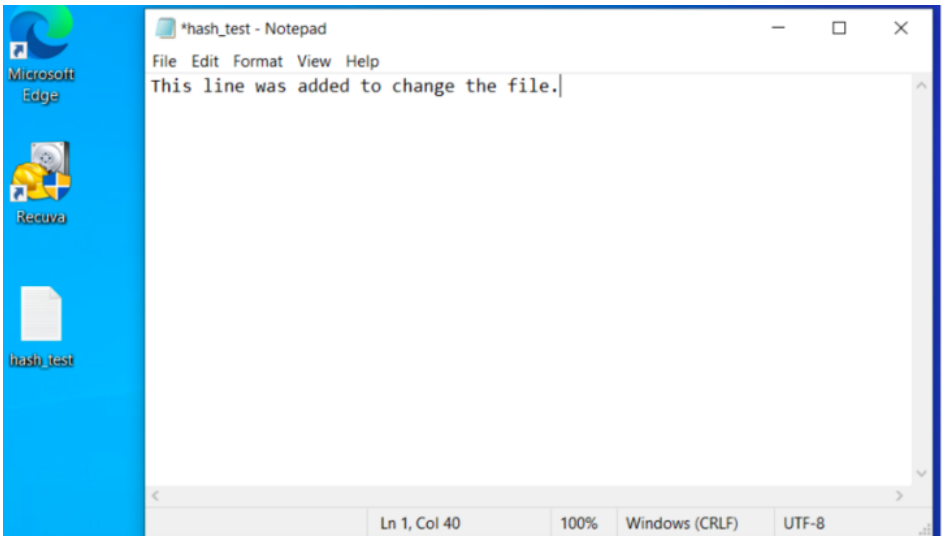
3. Right-clicked the file → Properties “Hash/CRC” → generated MD5 and SHA-256 hashes.



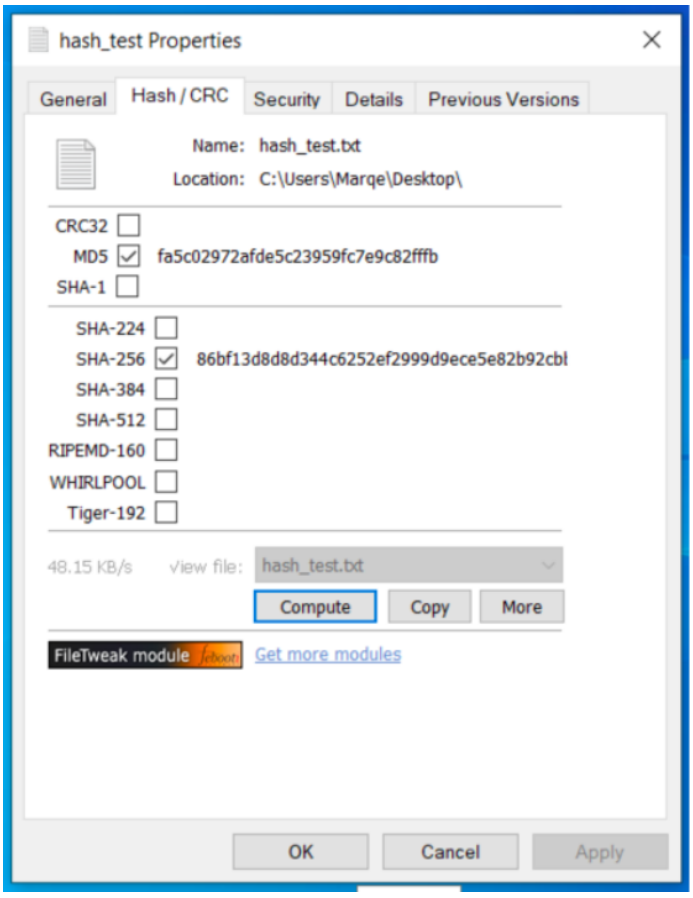
Step 3 – Modify File and Compare Hashes

1. Reopened hash_test.txt in Notepad.
2. Added the line:

   This line was added to change the file.



3. Saved the file again.
4. Right-clicked → Properties “Hash/CRC” → regenerated MD5 and SHA-256 hashes.



Step 4 – Analyze Results

- Compared the two sets of hashes from before and after modification.
- Both MD5 and SHA-256 values were completely different after changing one line.
- This property is known as the avalanche effect, meaning small modifications in the input result in large, unpredictable changes in the hash output.

5. Conclusion

This lab demonstrated how hashing ensures data integrity and authenticity in digital forensics. Using Febooti Hash & CRC, I verified that even any character change alters the entire hash value. This proves that hashing is a reliable way to detect tampering or corruption in digital evidence.

6. Challenges and Observations

- Remembering to select both MD5 and SHA-256 required double-checking the settings.
- The difference in hashes clearly illustrated the sensitivity of hashing algorithms.
- Febooti's context menu integration made it easy to hash files directly without command-line tools.
- The lab reinforced that hashing is fundamental to forensic validation.

8. References

- Febooti Hash & CRC: https://www.febooti.com/products/filetweak/members/hash-and-crc/
- NIST Hash Function Validation Documentation

Course: NWIT 263 – Digital Forensics

Lab Number: 08

Lab Title: Extracting Metadata from an MS Office File

Professor: Reza Mirabrishami

Date: 10/23/2025

1. Introduction

This lab focused on using file properties and metadata to identify hidden information in Microsoft Office documents. Metadata can reveal when a file was created, who created it, when it was last modified, and other embedded information useful in forensic analysis. The objective was to analyze how metadata changes when a document is edited, saved, or downloaded again, and to understand how timestamps can provide insight into user activity.

2. Lab Objectives

By completing this lab, I was able to:

- Identify file metadata within Microsoft Word documents.
- Record and compare Date Created, Date Modified, and Author fields.
- Observe how metadata changes after modifying and saving a file.
- Explain why file creation and modification dates differ when downloading versus saving locally.
- Understand the forensic importance of file metadata in investigations.

3. Tools and Requirements

- **Operating System:** Windows 10
- **Software:** Microsoft Word or Word Online
- **Alternative Tools:** DOCX Editor (for local editing)
- **File Used:** `metadata_test.docx`

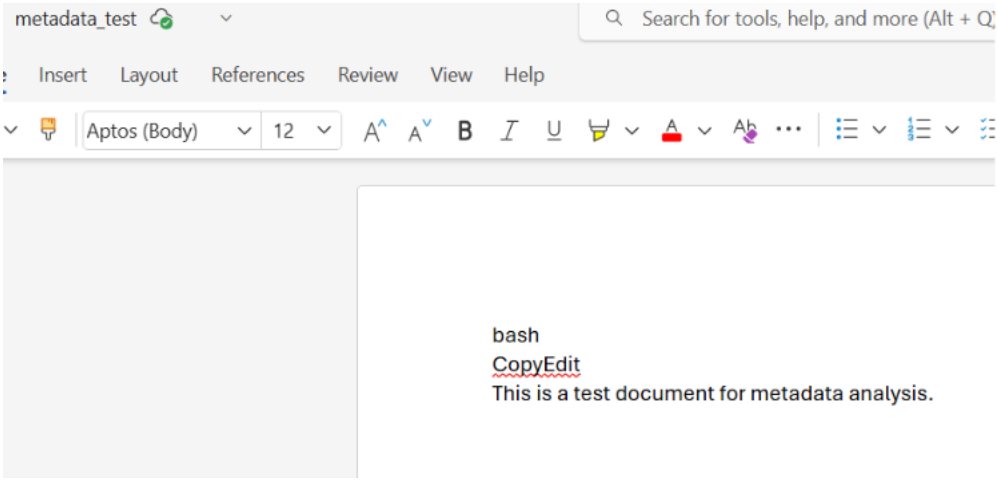
4. Lab Steps and Execution

Step 1 – Create a Test Document

1. Opened Microsoft Word and created a new document.
2. Typed:

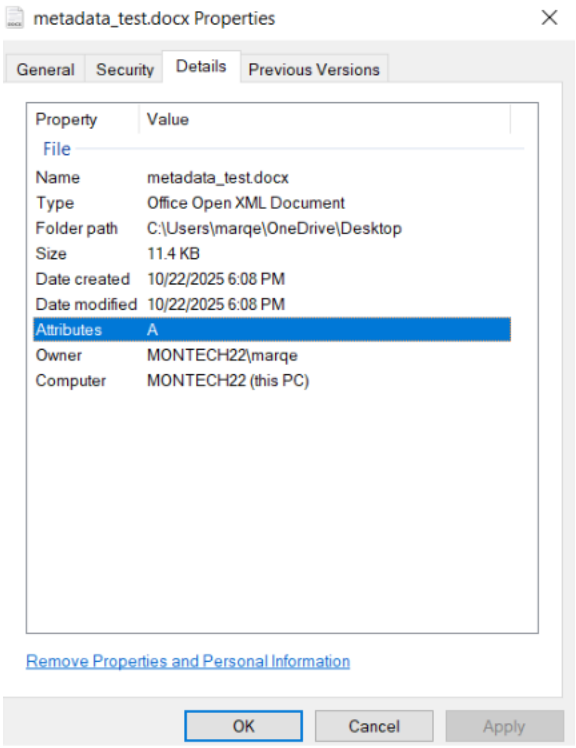
```
bash
CopyEdit
This is a test document for metadata analysis.
```

3. Saved the file as `metadata_test.docx`.



Step 2 – View Metadata in File Properties

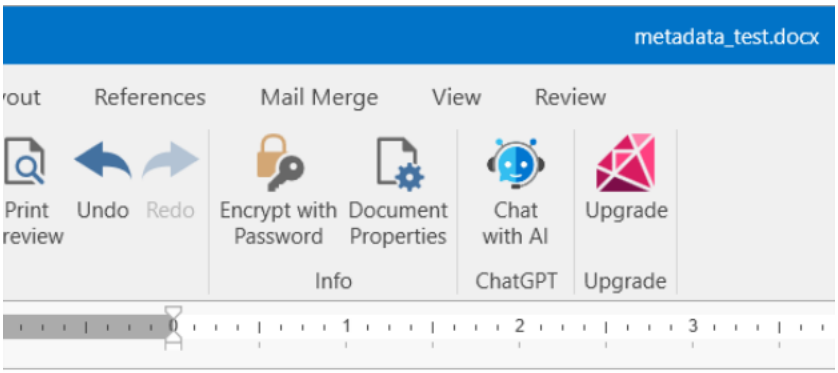
1. Checked file properties → Right-click → Properties → Details tab.
2. Recorded the following metadata values:
    a. Date Created
    b. Date Modified
    c. Author



Step 3 – Modify the Document and Check Metadata Again

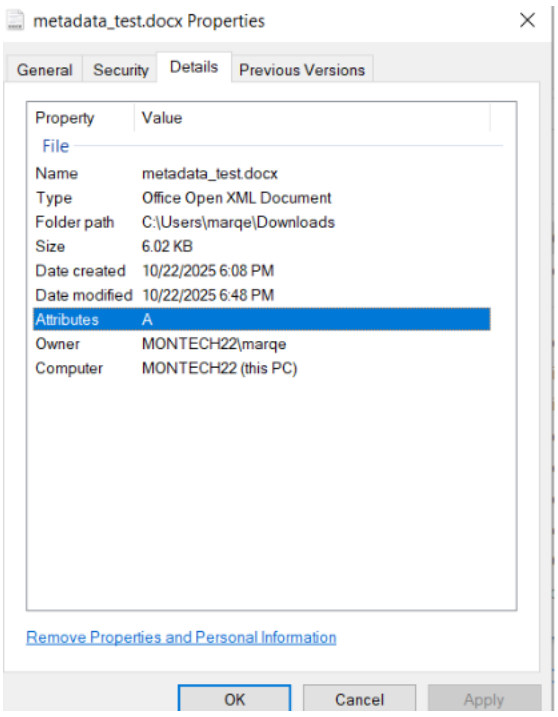
1. Opened the same file directly from the local drive.
2. Added a new line of text:

```
pgsql
CopyEdit
This is an updated version of the document.
```




```
pgsql
CopyEdit
This is an updated version of the document.
```

3. Saved the file.
4. Rechecked properties.
    a. The **Date Created** remained the same.
    b. The **Date Modified** updated to the current time.



Step 4 – Analyze Results

After creating and editing the Word document, the Date Modified field updated to the exact time the file was saved again, while the Date Created stayed the same. This confirms that Microsoft Word updates the modification timestamp every time a file is changed, but the original creation date is preserved as long as the file remains in the same location.

If the file is downloaded or saved to a different location, Windows assigns a new creation date, because the system treats it as a new instance of the file being written to the disk.

In forensic, this demonstrates the importance of understanding what each timestamp means:

Date Created – When the file first appeared on the storage device.

Date Modified – When the file content was last altered.

Date Accessed – When the file was last opened or viewed.

Knowing how these timestamps behave helps investigators determine when a file was first created, altered, or accessed, making it crucial for building a timeline in an investigation.

5. Conclusion

This lab demonstrated how Microsoft Office documents contain valuable metadata that records authorship, creation, and modification details. In forensic analysis, these timestamps can reveal when and how a file was created or tampered with. Maintaining proper chain of custody and avoiding unnecessary re-downloads ensures the original metadata remains intact for evidentiary purposes.

6. Challenges and Observations

- Using Word Online causes automatic file duplication (e.g., `metadata_test (1).docx`).
- The Date Created changed every time the document was re-downloaded.
- To keep timestamps consistent, the file must be edited and saved on the same system instead of re-downloaded.

- Understanding how metadata behaves helps distinguish between original and copied evidence.

7. References

- Microsoft Documentation: View or Change the Properties for an Office File
- NIST: Guide to Integrating Forensic Techniques into Incident Response

Course: NWIT 263 – Digital Forensics
Lab Number: 09

Lab Title: Windows Forensic Artifacts
Professor: Reza Mirabrishami
Date: 10/23/2025

1. Introduction

This lab focused on identifying and analyzing Windows forensic artifacts that attackers may leave behind on a compromised system. These artifacts can provide valuable evidence of persistence, program execution, and user activity. The goal was to use built-in Windows tools to locate potential malicious entries, such as startup items, recent file activity, and prefetch data, which together can help investigators reconstruct what occurred on a system after an intrusion.

2. Lab Objectives

By completing this lab, I was able to:

- Examine Windows Registry entries that show programs configured to run automatically at startup.
- Analyze the Recent Files folder to identify files that were recently accessed or executed.
- Review Prefetch files to determine which programs were run and when.

- Interpret what each artifact reveals about system or attacker activity.

3. Tools and Requirements

- **Operating System:** Windows 10
- **Tools Used:**
    - Registry Editor (`regedit`)
    - File Explorer
    - Notepad

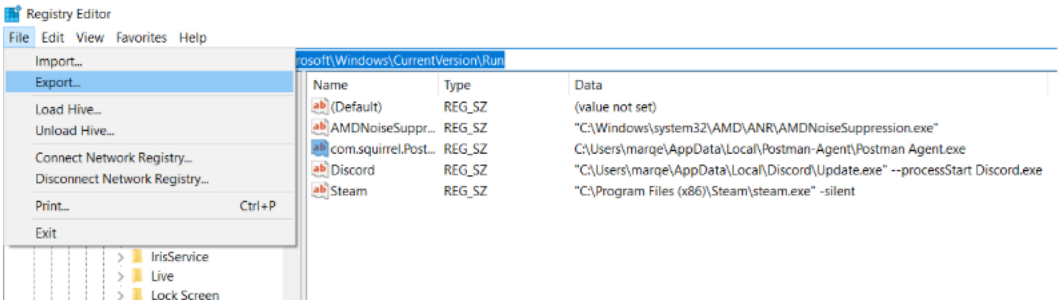
4. Lab Steps and Execution

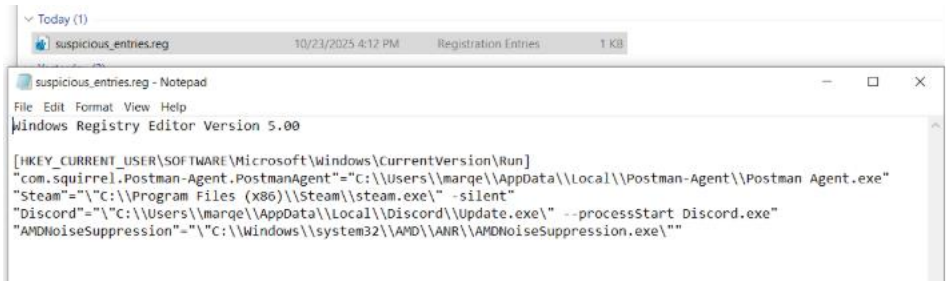
Step 1 – Investigate Auto-Start Entries (Registry)

1. Opened Registry Editor (regedit).
2. Navigated to the following key:

`HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run`

3. Reviewed each entry for unusual program names or file paths, such as those running from AppData, Temp, or user folders.
4. Found one suspicious entry:
    a. **Name:** `Postman Agent`
    b. **Path:** `C:\Users\marqe\AppData\Local\Postman-Agent\Postman Agent.exe`
5. Exported the entry and opened it in Notepad to confirm the executable path.



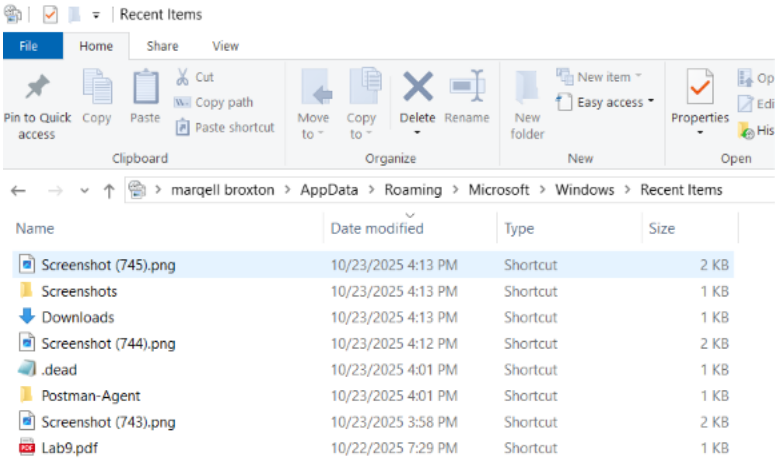


Step 2 – Check Recently Accessed Files

1. Opened File Explorer and navigated to:

`C:\Users\marqe\AppData\Roaming\Microsoft\Windows\Recent`

2. Reviewed the contents for recently accessed files.
3. Identified several items referencing temporary or user-created folders, suggesting potential script or tool execution.



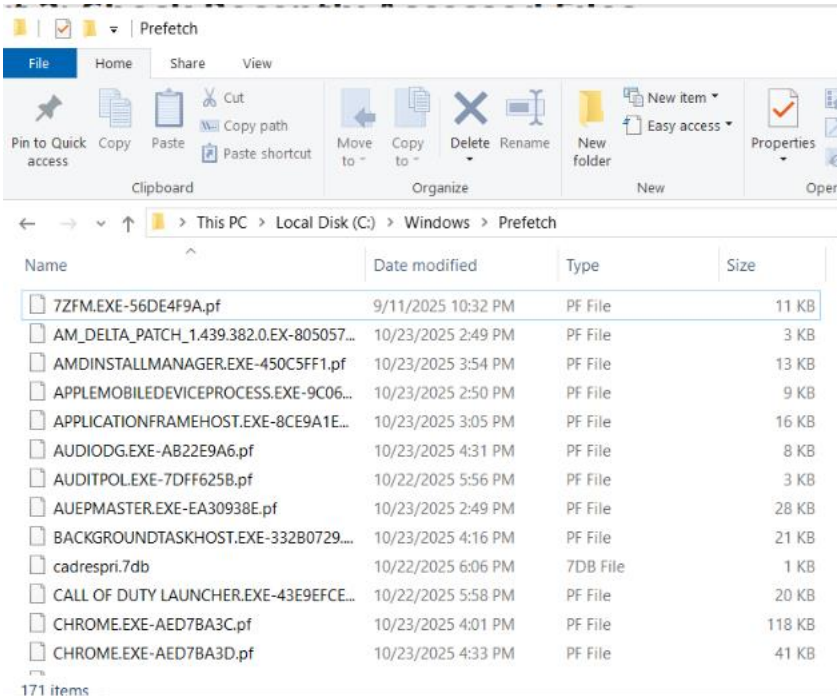
Step 3 – Examine Prefetch Files

1. Navigated to the Prefetch directory:

   `C:\Windows\Prefetch`

2. Observed multiple `.pf` files indicating programs that had run recently.
3. Identified one unusual file:

ADMININSTALLMANAGER.EXE-450C5FF1.pf



5. Analyze Results

The artifacts collected from the registry, recent file list, and prefetch directory all pointed to potentially malicious activity.

- The Registry Run key can show an attacker configured in their own malware to automatically start each time the user logged in.
- The Recent Files folder may reveal traces of files accessed by the user, or malicious scripts executed by an attacker.
- The Prefetch file will confirm actual program execution, proving that a .exe file ran on the system and was not simply placed there.

Together, these artifacts provide a clear timeline of compromise:

1. The malicious file was introduced and executed.
2. The program created persistence via the Run key.
3. Prefetch and recent file data confirmed repeated or recent execution.

6. Conclusion

This lab showed how forensic investigators can identify traces of malicious activity using native Windows artifacts. By reviewing the Registry, Recent folder, and Prefetch files, it's possible to reconstruct evidence of persistence, program execution, and user interaction. These data sources are essential for timeline reconstruction and attribution during digital investigations.

7. Challenges and Observations

- Some legitimate programs also run from AppData or Temp, so context and path verification are important.
- Prefetch files can only be viewed on systems with Prefetch enabled.
- Autoruns could be used as an additional verification tool for startup persistence.

8. References

- Microsoft Docs – Windows Registry and Autostart Locations
- SANS DFIR Poster: Windows Forensic Artifacts
- NIST SP 800-86: Guide to Integrating Forensic Techniques into Incident Response

Course: NWIT 263 – Digital Forensics
Lab Number: 10

Lab Title: Hard Links vs. Soft (Symbolic) Links

Professor: Reza Mirabrishami
Date: 10/31/2025

1. Introduction

This lab explores how Linux uses **hard links** and **soft (symbolic) links** to reference files. Understanding link behavior is important in digital forensics because link structures can

hide, reference, or preserve data even after the original file appears deleted. By working with inodes and link counts, I learned how Linux tracks file objects and how links behave when files are modified or removed.

2. Objectives

By completing this lab, I:

- Created and verified **hard links** and **soft links** in Linux
- Observed the inode values and link counts using ls -li
- Demonstrated how hard links preserve data after file deletion
- Showed how symbolic links fail once their target files are deleted
- Explained forensic relevance of link behavior

3. Tools Used

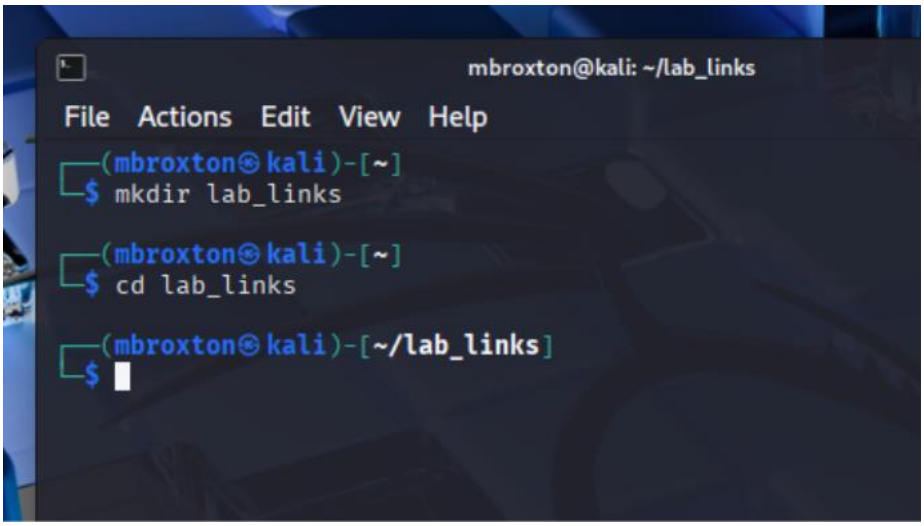
- **Operating System:** Linux (Ubuntu)
- **Terminal / Bash Shell**
- **Commands Used:**
    - ln (create hard & soft links)
    - ls -li (list files w/ inode info)
    - cat (display file contents)
    - rm (remove files)
- **Filesystem:** ext4 (default Linux filesystem)

4. Lab Procedure & Results

Part 1 – Hard Links

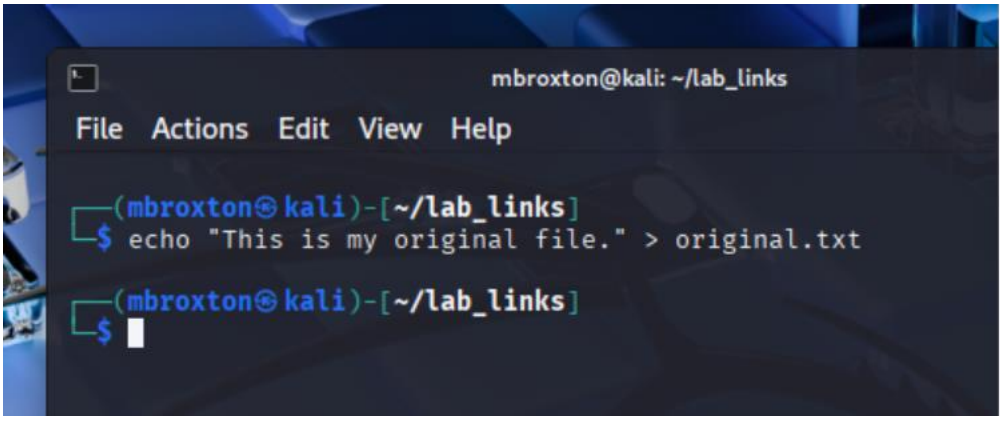
Step 1: Create directory

```
mkdir ~/lab_links && cd ~/lab_links
```



Step 2: Create original file

echo "This is my original file." > original.txt

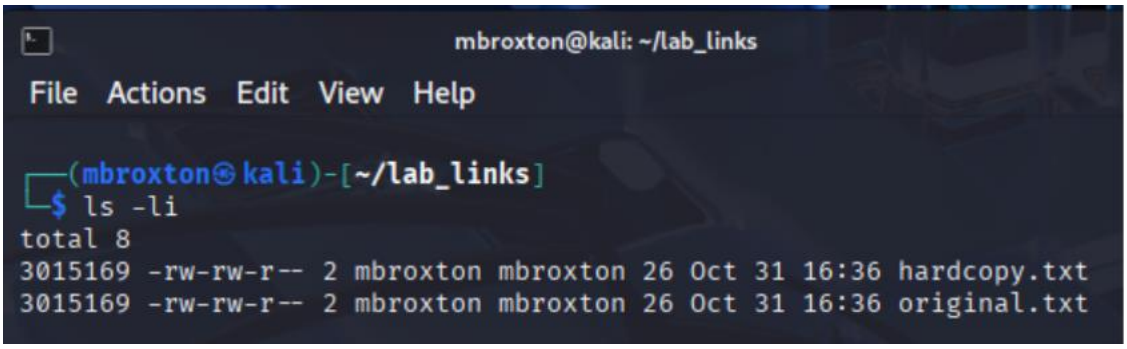


Step 3: Create hard link

ln original.txt hardcopy.txt

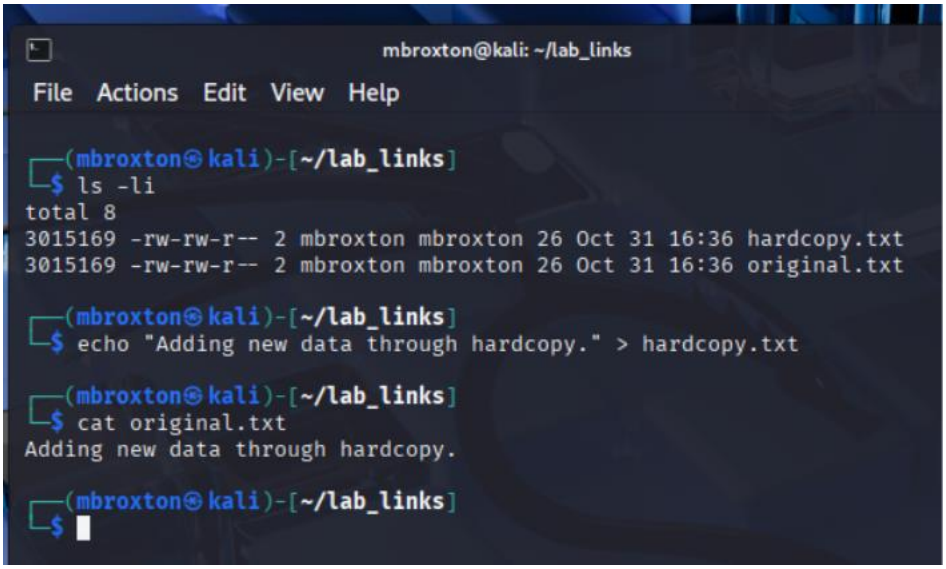
Step 4: Verify inodes

ls –li



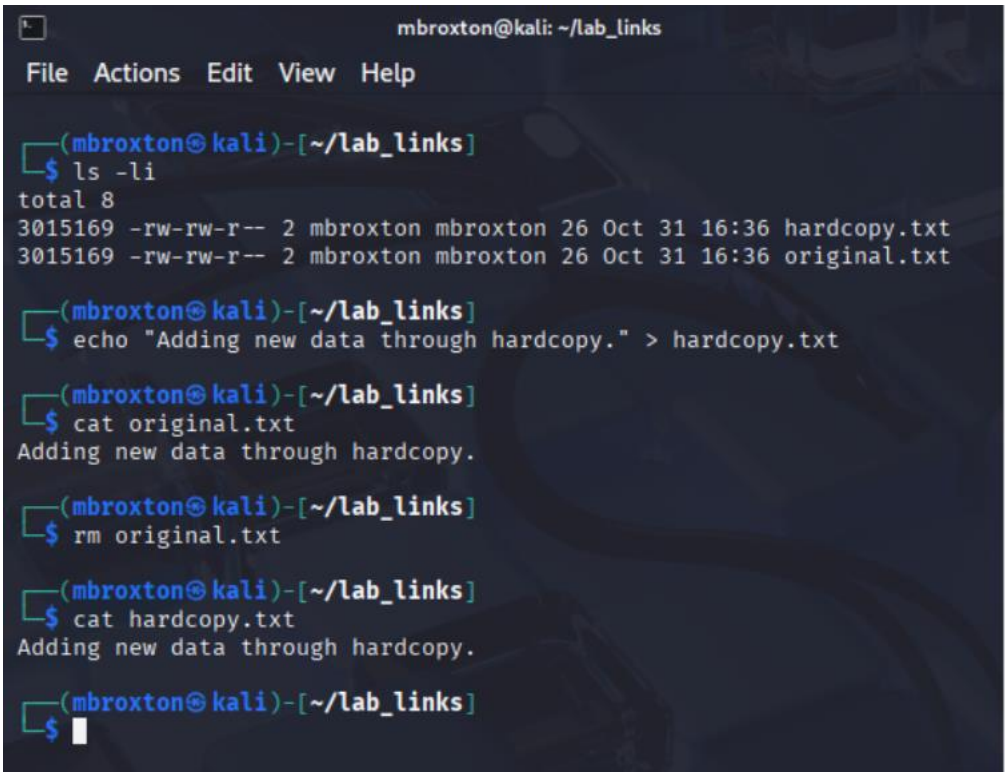
Step 5: Modify through hard link

echo "Adding new data through hardcopy." >> hardcopy.txt
cat original.txt



Step 6: Delete original file

rm original.txt
cat hardcopy.txt

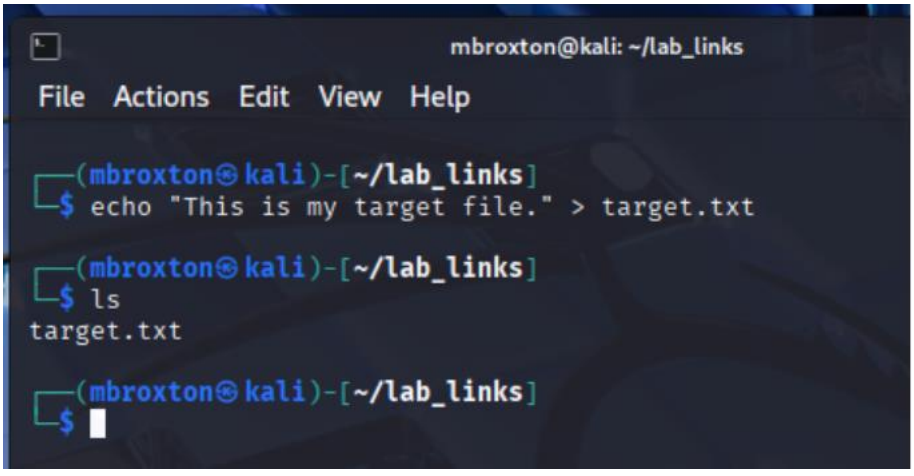


hardcopy.txt retained full content — data survived deletion.

Part 2 – Soft (Symbolic) Links

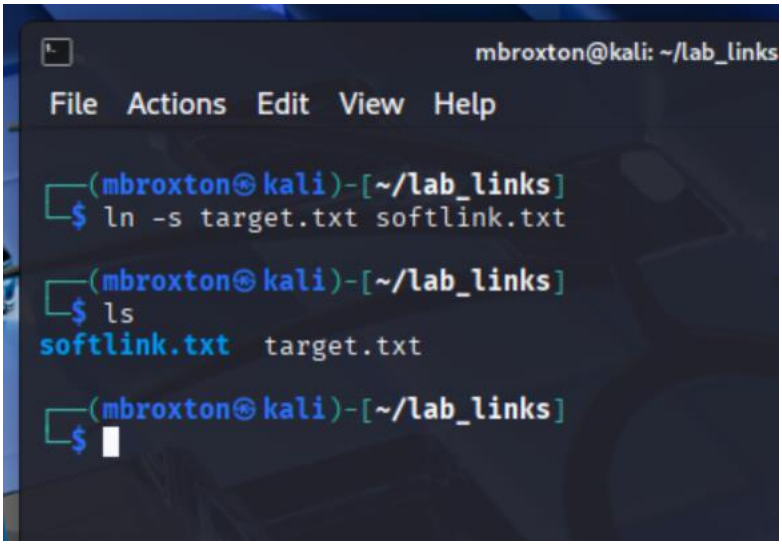
Step 1: Create new file

echo "This is my target file." > target.txt



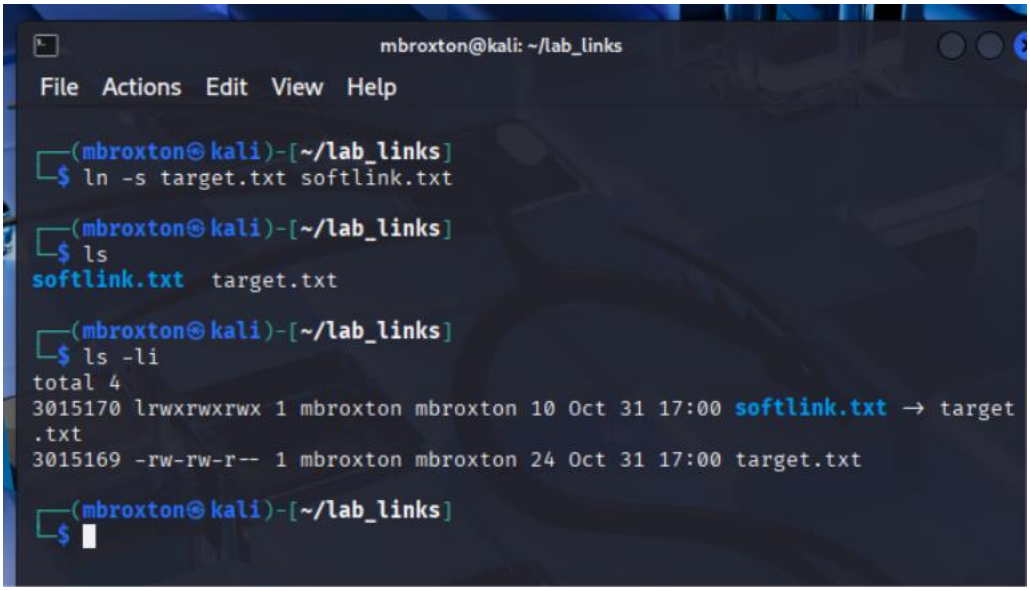
Step 2: Create symbolic link

ln -s target.txt softlink.txt



Step 3: Verify link

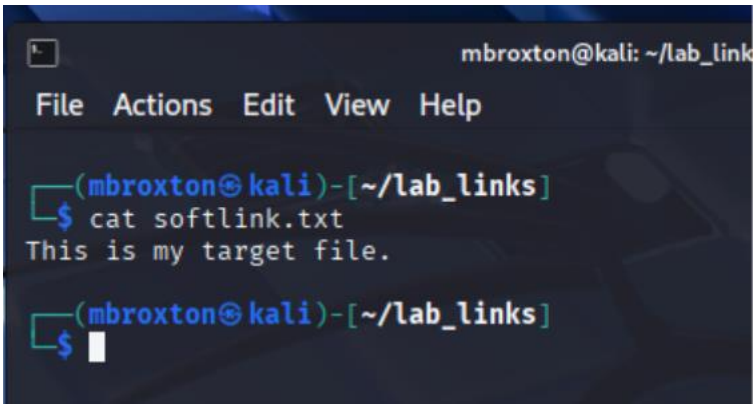
ls -li



Symbolic link displayed different inode and arrow → target.txt.

Step 4: Access through symlink

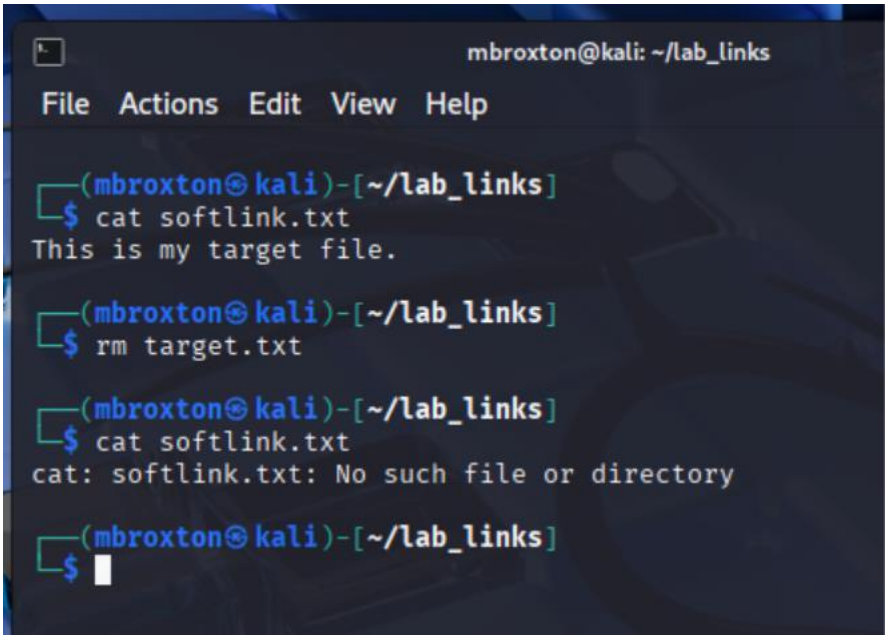
cat softlink.txt



Displayed same contents as target.txt.

Step 5: Break link

rm target.txt
cat softlink.txt



Output:

cat: softlink.txt: No such file or directory

5. Analyze Results

Hard links and symbolic links behave differently due to inode structure. Hard links share the same inode as the original file, meaning both names point to the same data on disk. When the original file was deleted, the data remained accessible through the hard link. This behavior makes hard links important in digital forensics, as a file may appear deleted but still exist through a hard link.

Symbolic links point to the file name and not the inode. When the original file was removed, the symbolic link broke because its path target no longer existed. Symbolic links are like shortcuts in Windows and do not preserve access once the original file is removed.

Understanding link behavior helps forensic analysts detect hidden or persistent files and recover data that may still exist even after deletion.

6. Conclusion

This lab demonstrated how Linux links work at the filesystem level. Hard links reference the **same inode**, meaning they remain valid even after the original file is deleted, which can preserve data and matter in forensic recovery. Symbolic links act as shortcuts to paths and break when targets are removed. Understanding link behavior helps investigators detect file-hiding techniques, data persistence, and filesystem artifacts.

7. Challenges & Observations

- Understanding inode behavior was critical for observing differences.
- Hard links successfully preserved data after file deletion.
- Symbolic links behaved as expected and failed once the target was removed.
- Recognizing how attackers might hide files using hard links is an important forensic skill.

8. References

- Linux ln and ls manual pages
- NIST SP 800-86: Guide to Integrating Forensics into Incident Response
- Course materials provided by instructor

Course: NWIT 263 – Digital Forensics
Lab Number: 11

Lab Title: Hidden Message Extraction with Steghide
Professor: Reza Mirabrishami
Date: 10/31/2025

1. Introduction

This lab focused on using steganography tools to hide and extract data within an image file. Steganography is the practice of concealing messages or information within other non-secret files, such as images. The objective of this lab was to embed a hidden text file inside a JPEG image using the Steghide tool in Linux, verify the changes using file hashes and metadata, and then extract the hidden message. This technique is relevant to digital forensics, as malicious actors often use steganography to secretly transfer data or exfiltrate information.

2. Objectives

- Verify Steghide installation on a Linux system
- Inspect the original image file metadata and hash value
- Embed hidden data inside an image file using Steghide
- Verify the difference between the original and stego image using hash comparison
- Extract the hidden file and confirm successful recovery of the secret message
- Understand forensic implications of steganography

3. Tools Used

- Kali Linux / Parrot OS
- Steghide
- file command
- exiftool

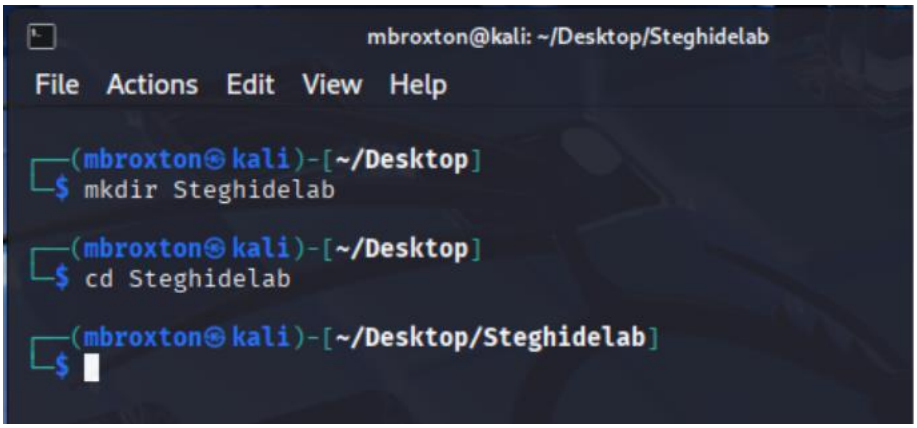
- sha256sum
- Terminal

4. Lab Procedure and Results

Step 1 – Setup and Verification

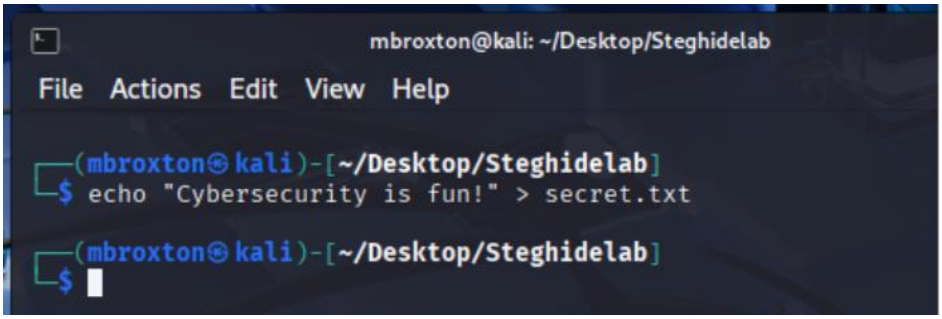
1. Created working directory:

   mkdir ~/Desktop/SteghideLab
   cd ~/Desktop/SteghideLab



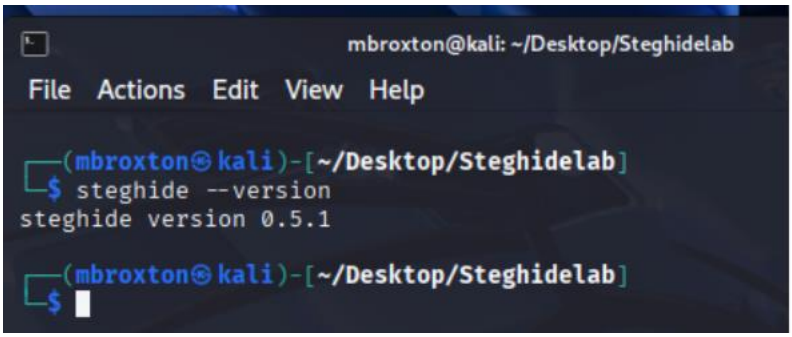
2. Created a text file containing the secret message:

   echo "Cybersecurity is fun!" > secret.txt



3. Verified Steghide installation:

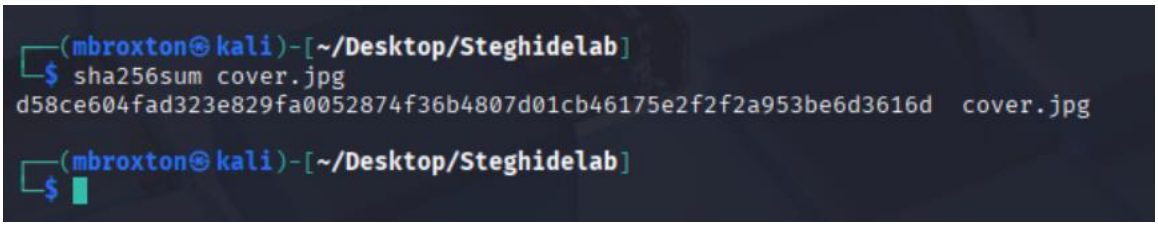
   steghide --version

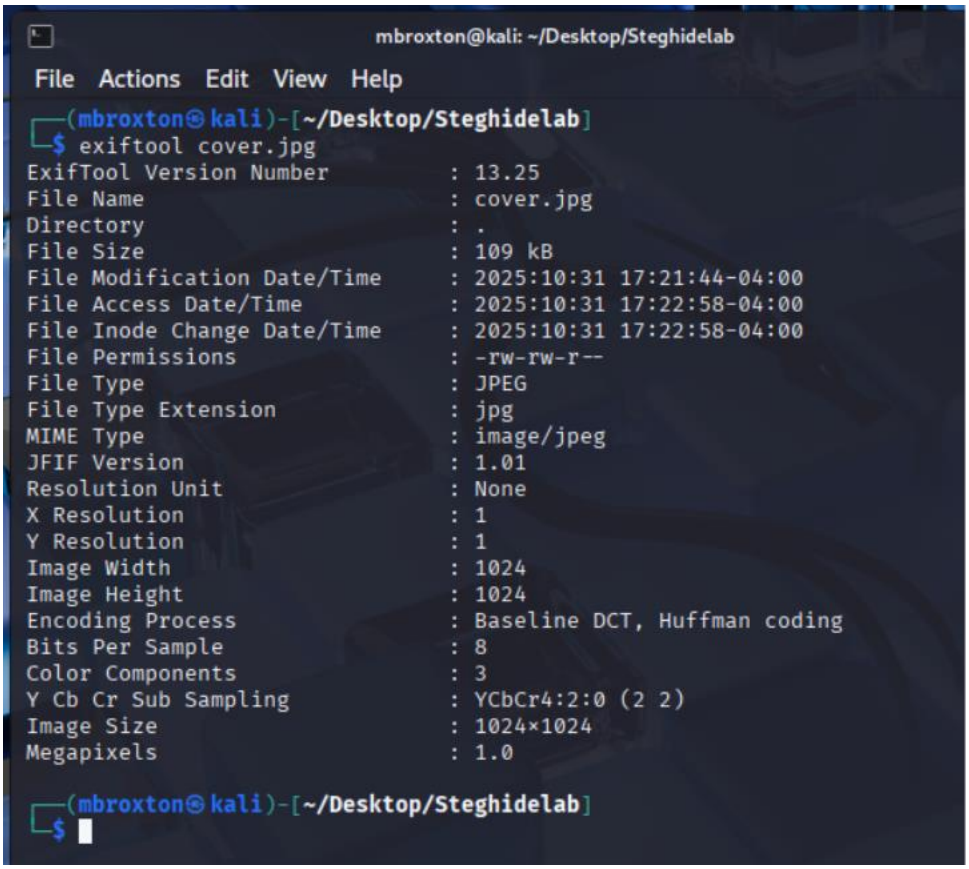


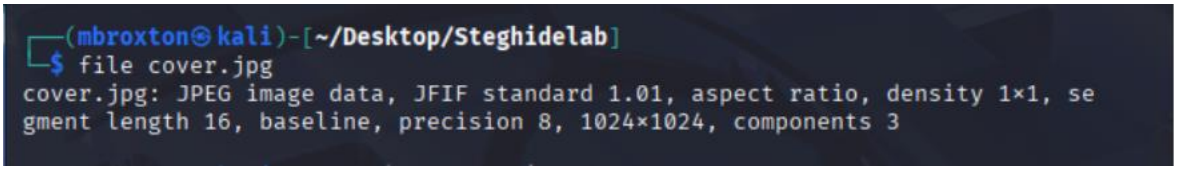
Step 2 – Analyze Original Image

1. Verified file type and metadata:

   file cover.jpg
   exiftool cover.jpg
   sha256sum cover.jpg







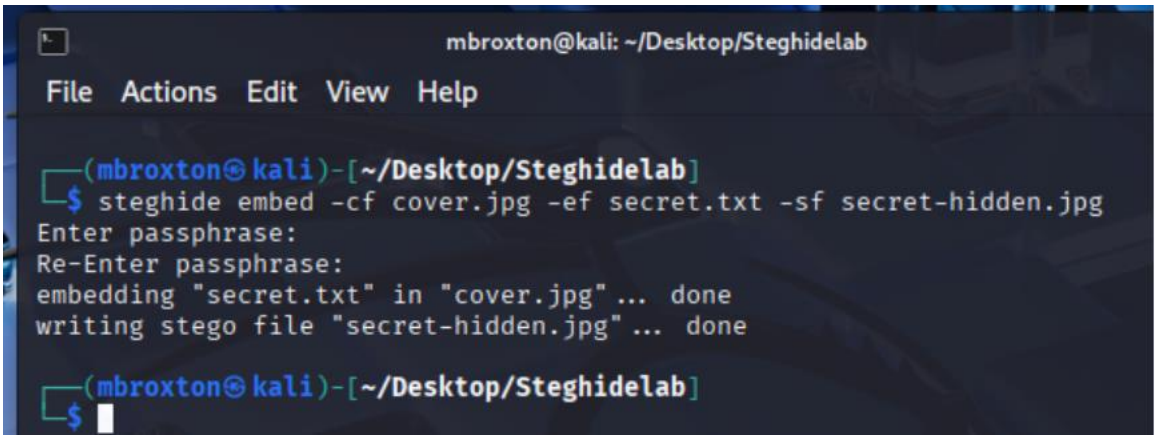
2. Recorded information and hash value for the original image.

Step 3 – Hide Secret Data in the Image

1. Embedded the secret text file into the image:

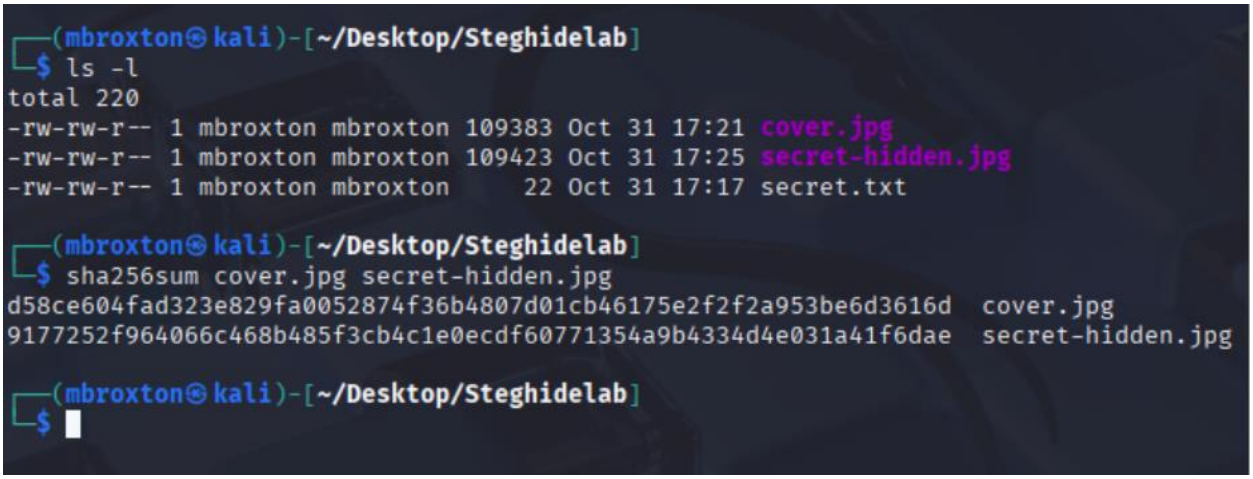
   steghide embed -cf cover.jpg -ef secret.txt -sf secret-hidden.jpg

2. Entered password when prompted.



3. Listed files and checked hashes:

   ls -l
   sha256sum cover.jpg secret-hidden.jpg



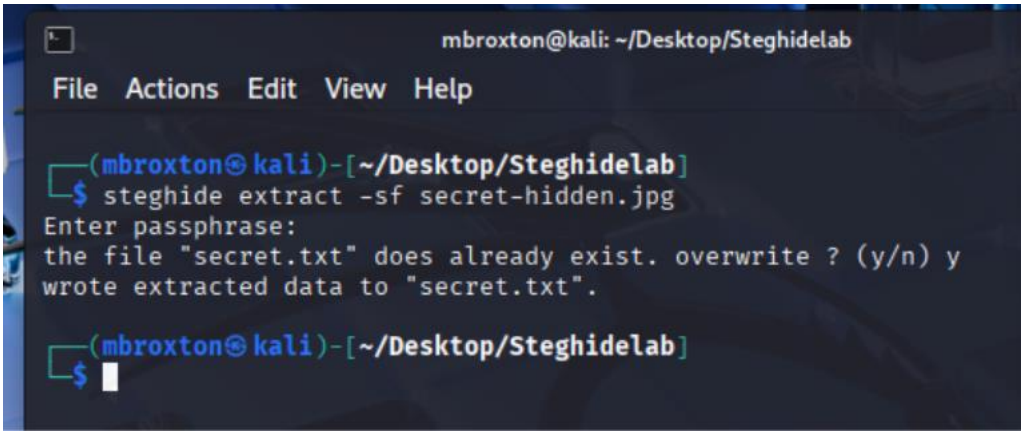
4. Observed that the hash of the new image differed from the original, proving the file was modified to include hidden data.

Step 4 – Extract Hidden Message

1. Extracted the hidden file:

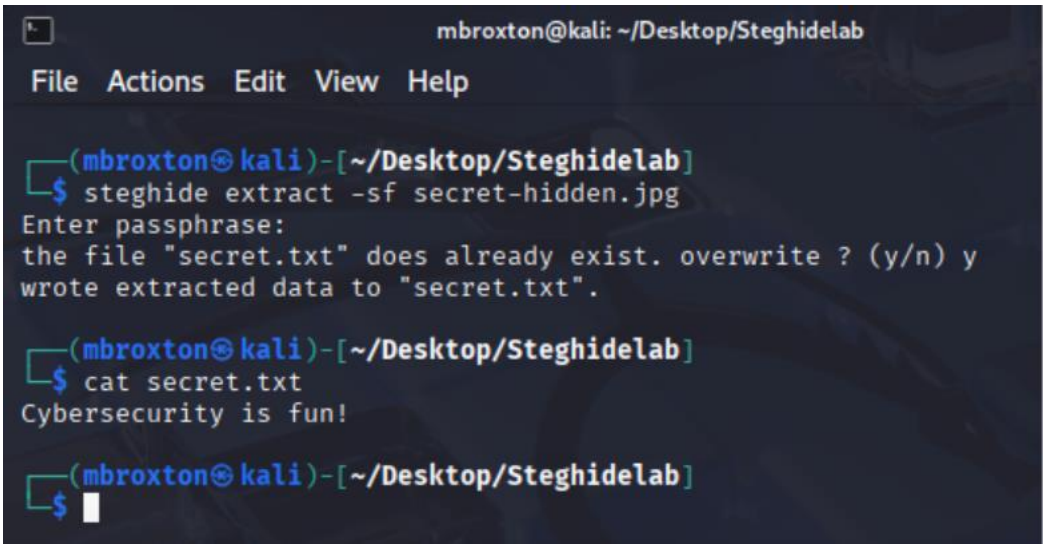
steghide extract -sf secret-hidden.jpg

2. Entered password.



3. Viewed extracted file content:

   cat secret.txt



   The extracted text matched the original secret message, confirming successful steganography and extraction.

5. Analyze Results

Steghide successfully embedded a text file into a JPEG image and later extracted it when the correct password was provided. Metadata analysis and hashing verified that the stego file differed from the original image, even though visually the files appeared identical. This demonstrates how steganography allows hidden communication without obvious evidence in the file's appearance. Hash changes confirmed the file was altered, supporting forensic validation.

6. Conclusion

This lab demonstrated the use of Steghide to conceal and extract hidden information within image files. Steganography is a covert communication method often used for malicious purposes, such as data exfiltration or hidden instruction delivery. Forensic analysts must be able to detect suspicious files, analyze metadata, compare cryptographic hashes, and extract hidden content during investigations.

7. Challenges and Observations

- Steghide required a password to extract the data, showing the importance of password-protected steganography.
- The visual appearance of the file did not change, proving that steganography is difficult to detect without forensic analysis.
- Hash comparison was critical to confirm the file was altered.

8. References

- Steghide documentation and man page
- NIST SP 800-86, Guide to Integrating Forensic Techniques
- Course materials provided by instructor

Course: NWIT 263 – Digital Forensics

Lab Number: 12

Lab Title: Network Traffic and Analysis Using Wireshark
Professor: Reza Mirabrishami
Date: 11/13/2025

1. Introduction

This lab focused on analyzing captured network traffic using Wireshark to identify suspicious activities such as port scanning, cleartext communication, and directory or user enumeration. Using targeted Wireshark filters, I examined packets within a provided PCAP file to detect abnormal behaviors that may indicate the early stages of an attack. This exercise demonstrated how packet inspection and filtering techniques support threat identification within digital forensics and incident response.

2. Objectives

- Open and analyze a packet capture (PCAP) file in Wireshark
- Apply specific filters to isolate suspicious traffic
- Identify scanning behavior using SYN flag analysis
- Detect HTTP requests that could include reconnaissance or data exfiltration
- Identify LDAP-related traffic indicating user or directory queries
- Document findings based on filtered network traffic

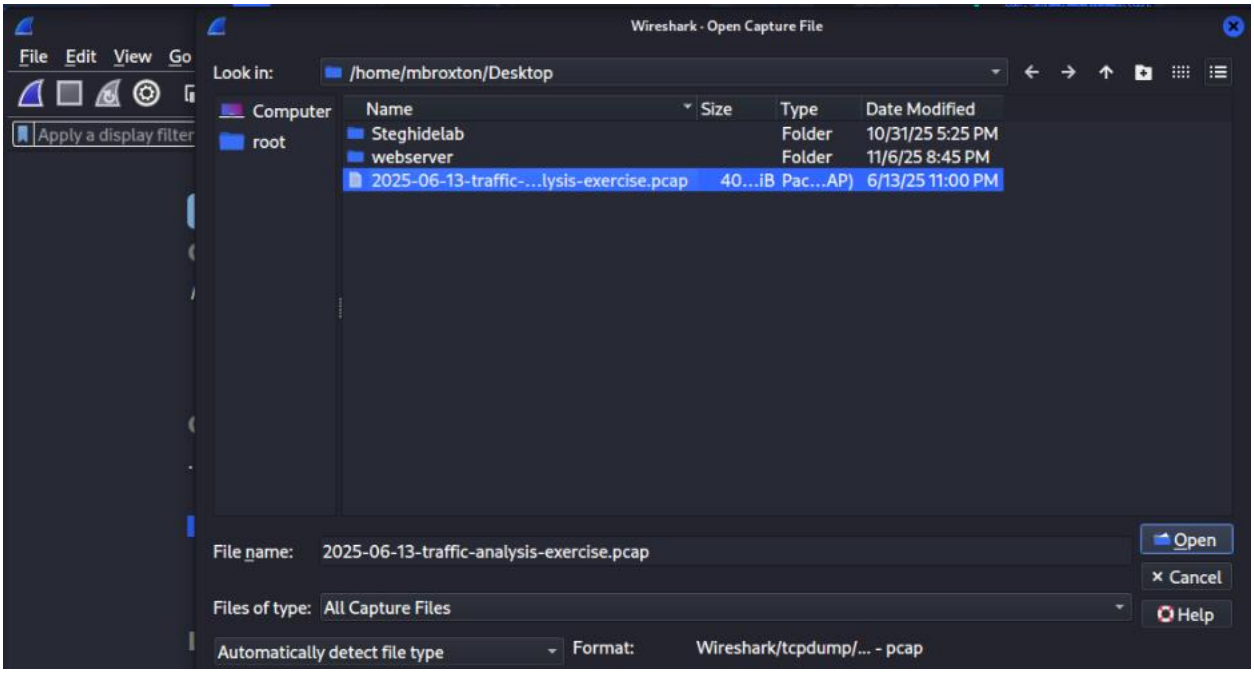
3. Tools Used

- Wireshark
- PCAP file provided by instructor
- Wireshark filters:
    - http.request
    - tcp.flags.syn == 1
    - ldap

4. Lab Procedure and Results

Step 1 – Loading and Reviewing the PCAP

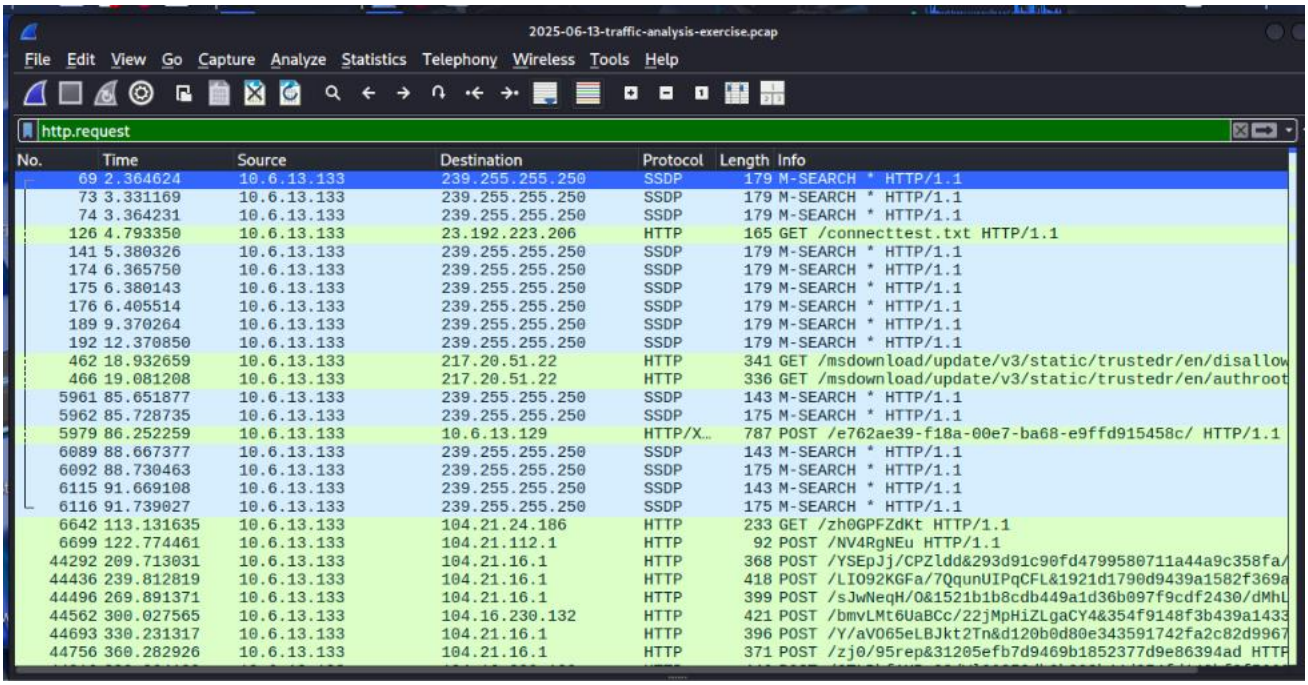
- Opened Wireshark and loaded the PCAP file.
- Briefly reviewed the unfiltered traffic to observe baseline activity.



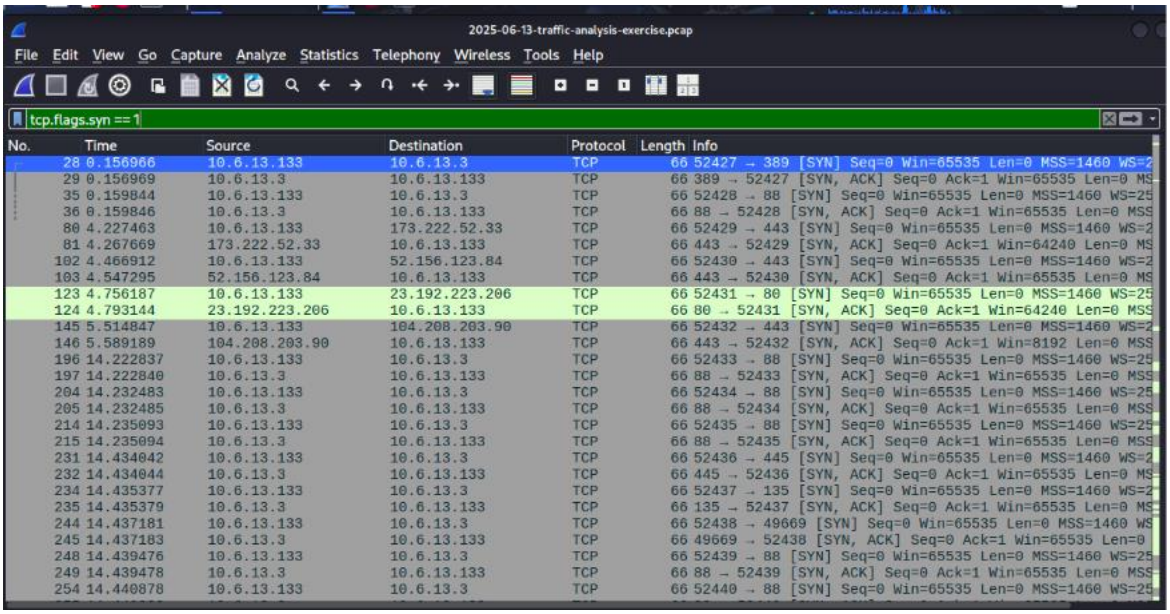
Step 2 – Apply Filters

Filter Used:

- Http.request - This filter displayed all HTTP requests sent from clients to servers.
  - Findings: Multiple unencrypted HTTP GET and POST requests were visible.

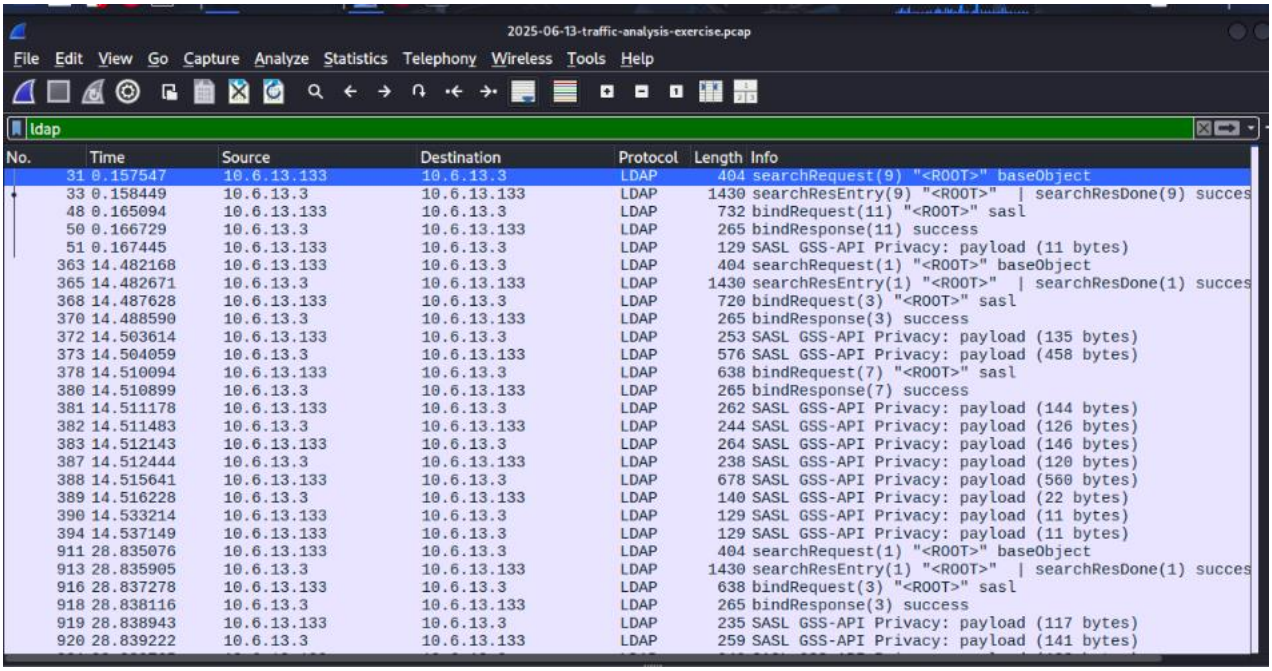


- tcp.flags.syn == 1 - This filter shows TCP SYN packets, typically used in connection initiation.
  - Findings: A high number of SYN packets were observed between the same source and multiple destination ports.



- ldap - LDAP traffic is commonly used for authentication, directory queries, and user enumeration.

- ♣ Findings: The PCAP showed LDAP query traffic between a client and a server.



Step 3 – Follow a Suspicious TCP Stream

- Right-click any suspicious packet → “Follow” → “TCP Stream”

The TCP stream captured shows an **HTTP POST request** that is highly suspicious and indicative of **malware Command and Control (C2) communication** or **data exfiltration**.

5. Analysis

By applying each filter separately, distinct types of suspicious activity were identified. The SYN traffic suggested port scanning behavior, which is commonly part of the reconnaissance phase of an attack. The HTTP requests revealed cleartext web communication that could include credential submission or unauthorized access

attempts. The LDAP traffic may indicate attempts to gather directory information, often used by attackers for privilege escalation planning.

The combination of SYN packets, LDAP queries, and HTTP requests fits a typical intrusion kill chain sequence:

- Reconnaissance (SYN scan)
- Enumeration (LDAP queries)
- Potential access or interaction (HTTP requests)

Together, these findings demonstrate how packet-level analysis can reveal the intent and behavior of potential attackers within a network.

6. Conclusion

This lab demonstrated the importance of filtering and analyzing network traffic within Wireshark to detect abnormal or malicious behavior. By applying specific filters, I was able to isolate scanning attempts, cleartext HTTP communication, and LDAP activity that could serve as part of an attack chain. Understanding how to examine network traffic provides valuable skills in digital forensics, incident response, and threat detection.

7. Challenges and Observations

- Applying precise filters helped quickly isolate suspicious packets.
- The SYN filter proved especially effective for identifying scanning.
- LDAP traffic is uncommon in typical PCAPs, so recognizing enumeration characteristics was useful.
- No alert log was provided, so analysis was based solely on packet activity.

8. References

- Wireshark User Guide
- NIST SP 800-115: Technical Guide to Information Security Testing
- Course lecture materials

Course: NWIT 263 – Digital Forensics

Lab Number: 13

Lab Title: Wireshark – Capturing Cleartext Login with Local HTTP Server

Professor: Reza Mirabrishami

Date: 11/13/2025

1. Introduction

This lab demonstrated how to capture and analyze unencrypted HTTP traffic using Wireshark. By setting up a local web server with a simple login form, I was able to observe how credentials entered through an insecure connection can be intercepted in transit. The lab illustrated the dangers of transmitting sensitive information over HTTP and reinforced the importance of encryption protocols such as HTTPS for secure communication.

2. Objectives

- Set up a local Python HTTP web server with a fake login page.
- Capture network traffic using Wireshark.
- Filter and identify HTTP POST requests containing login credentials.
- Follow and analyze an HTTP stream to extract username and password data.
- Understand why unencrypted HTTP poses a significant security risk.

3. Tools Used

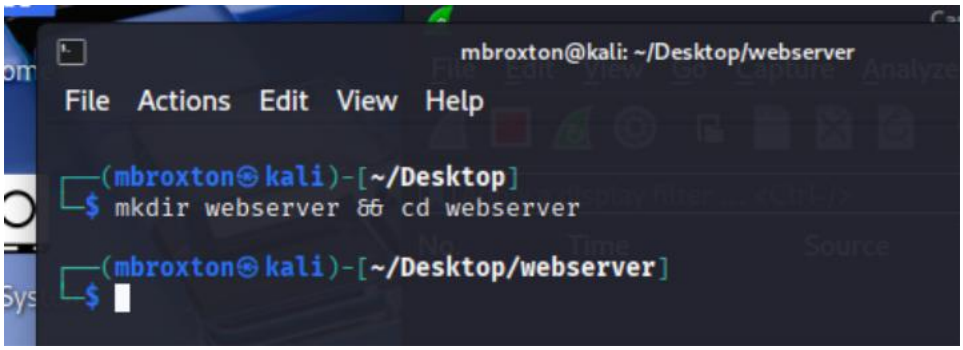
- **Operating System:** Linux (Kali)
- **Wireshark**
- **Python 3** (for local web server)
- **Web Browser**
- **Terminal**

4. Lab Procedure and Results

Part 1 – Set Up Local HTTP Server

1. Created a new project directory:

- mkdir ~/webserver
- cd ~/webserver



2. Created a fake login page:

- nano index.html
- Added HTML code containing username and password form fields.

```
<!DOCTYPE html>
<html>
<body>
        <h2>Login Page</h2>
        <form action="/login" method="POST">
                Username: <input type="text" name="username"><br>
                Password: <input type="password" name="password"><br>
                <input type="submit" value="Login">
        </form>
</body>
</html>
```

3. Created and configured a Python web server script (server.py):

- nano server.py
- This script handled GET and POST requests and printed login attempts to the terminal.

```
from http.server import BaseHTTPRequestHandler, HTTPServer
                import urllib.parse

                class SimpleHandler(BaseHTTPRequestHandler):
                        def do_GET(self):
                                if self.path == "/":
                                        with open("index.html", "rb") as f:
                                                self.send_response(200)
                                                self.send_header("Content-type", "text/html")
                                                self.end_headers()
                                                self.wfile.write(f.read())

                        def do_POST(self):
                                length = int(self.headers['Content-Length'])
                                post_data = self.rfile.read(length).decode('utf-8')
                                data = urllib.parse.parse_qs(post_data)
                                print("=== LOGIN ATTEMPT ===")
                                print(f"Username: {data.get('username', [''])[0]}")
                                print(f"Password: {data.get('password', [''])[0]}")
                                self.send_response(200)
                                self.end_headers()
                                self.wfile.write(b"Login submitted. Check server console.")

                server = HTTPServer(('0.0.0.0', 8080), SimpleHandler)
                print("Server started on http://localhost:8080")
                server.serve_forever()
```

4. Started the web server:

- python3 server.py
- The terminal displayed:

  Server started on http://localhost:8080

Part 2 – Capture Traffic with Wireshark

1. Launched **Wireshark** and selected the **lo (loopback)** interface.
2. Started a live packet capture.

3. Opened a web browser and navigated to:

   http://localhost:8080

4. Entered the following login credentials:
   a. Username: reza
   b. Password: test@123

5. Clicked “Login.” The server terminal captured credentials in plain text.

Part 3 – Filtering HTTP Traffic

- Applied the following filters in Wireshark to narrow the capture view:

  - http.request.method == "POST"

    - frame contains "username"

- http

- tcp.port == 8080

These filters displayed HTTP POST packets containing form data.

- Followed the HTTP stream (Right-click → Follow → HTTP Stream), the raw POST request was visible:

```
POST /login HTTP/1.1
Host: localhost:8080
Content-Type: application/x-www-form-urlencoded
Content-Length: 33

username=reza&password=test%40123
```

The %40 symbol represents “@” through URL encoding.

5. Analyze Results

The Wireshark capture revealed that the login credentials were transmitted in **plain text** using the HTTP protocol. Because HTTP does not encrypt data between the client and the server, anyone with access to the network traffic could intercept sensitive information. This demonstrates the security risk associated with using unencrypted connections. In a real-world scenario, attackers could use packet sniffers to steal usernames, passwords, or other confidential data.

The captured POST request clearly displayed both the username and password fields, confirming that the data was not protected. This reinforces the necessity of using **HTTPS**, which encrypts communication using SSL/TLS to prevent interception.

6. Conclusion

This lab effectively demonstrated the vulnerability of unencrypted HTTP traffic. By setting up a local server and analyzing packets with Wireshark, I observed how user credentials can be exposed during transmission. The exercise emphasized the importance of encryption and secure network protocols to protect sensitive information. Understanding how to identify clear text transmissions equips forensic and cybersecurity professionals to detect and prevent data exposure in real-world environments.

7. Challenges and Observations

- Initially, no packets appeared in Wireshark until the correct **loopback interface (lo)** was selected.
- Using the frame containing filter made it easier to pinpoint the packets containing login data.
- The demonstration clearly showed the risk of HTTP, as the password was visible in the raw packet stream.

8. References

- Wireshark User Guide
- Python HTTP Server documentation
- NIST SP 800-115: Technical Guide to Information Security Testing and Assessment
- Course materials provided by instructor

Course: NWIT 263 – Digital Forensics

Lab Number: 14

Lab Title: Diskpart and File Hiding Techniques

Professor: Reza Mirabrishami

Date: 11/13/2025

1. Introduction

This lab focused on hiding and unhiding storage partitions and files on both Windows and Linux systems. These techniques are commonly used by attackers to conceal data, making them relevant to digital forensics. By working with Diskpart, Windows attributes, and Linux hidden file naming conventions, this lab demonstrated how data can be intentionally hidden and how forensic investigators may uncover it.

2. Objectives

- Use Windows Diskpart to hide and unhide a disk partition.
- Apply Windows attribute commands to hide and unhide individual files.
- Hide and reveal files in Linux using dot-prefixed filenames.
- Observe system behavior before and after hiding operations.
- Understand the forensic importance of detecting hidden data.

3. Tools Used

- Windows 10 Diskpart
- Windows Command Prompt
- Linux Terminal (Kali Linux)
- Basic file-system commands (attrib, mv, ls -a)

4. Lab Procedure and Results

Part 1 – Hiding a Partition (Diskpart – Windows)

- Opened Diskpart from Windows command line.
- Displayed volumes:

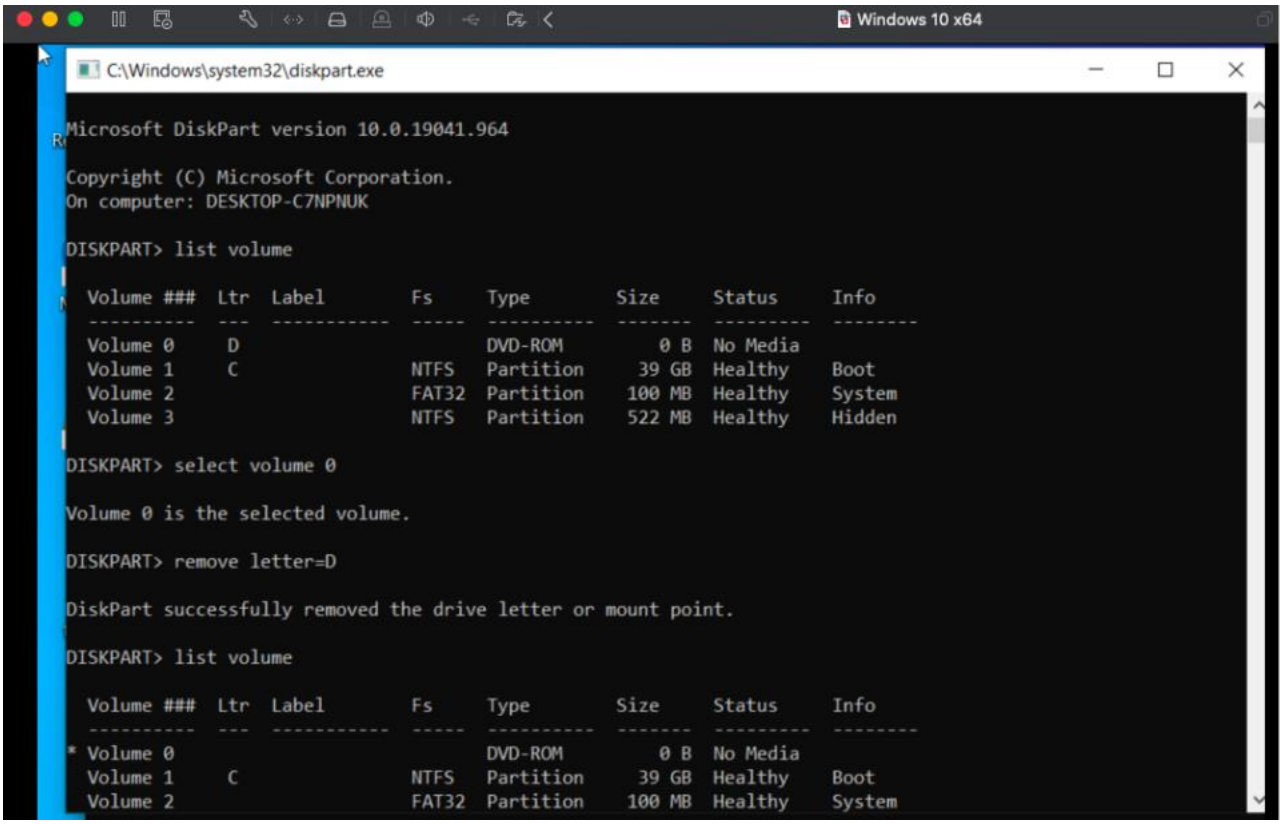
list volume

- Selected target volume (example):

  select volume 0

- Hid the partition by removing its drive letter:

  remove letter=D



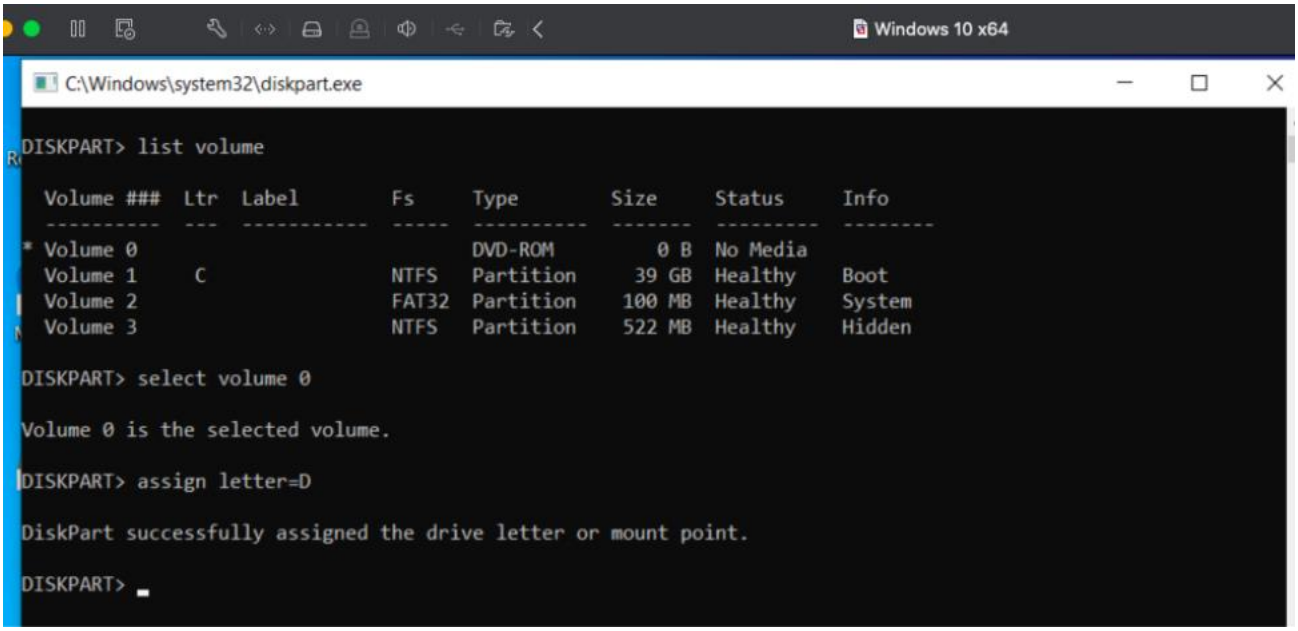
The letter was successfully removed.

Part 2 – Unhiding a Partition

- Repeated volume listing and selection:
- list volume
  select volume 3

- Restored the partition by assigning its drive letter:

- assign letter=D

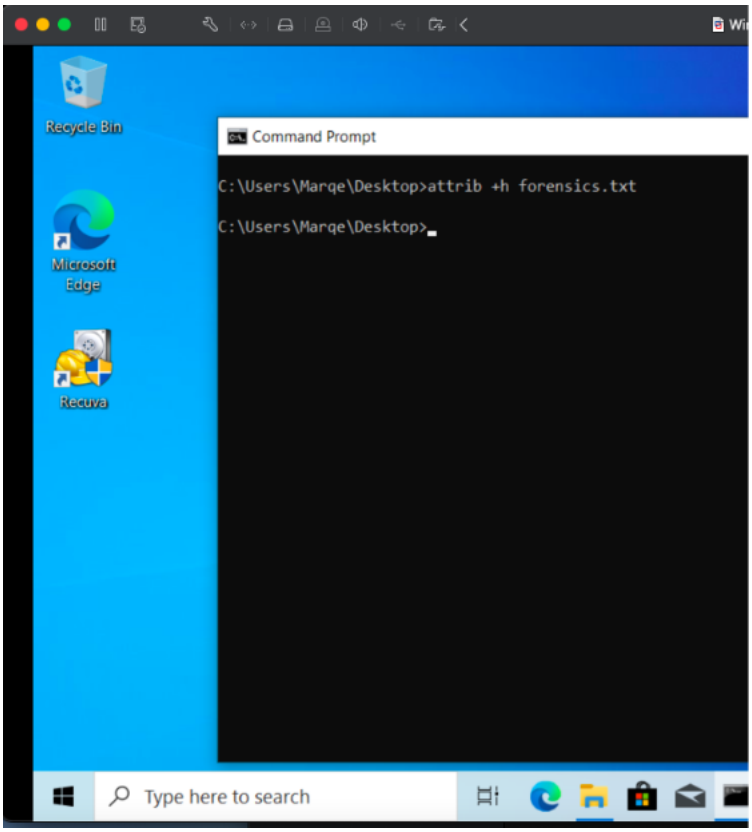


The letter was successfully added back.

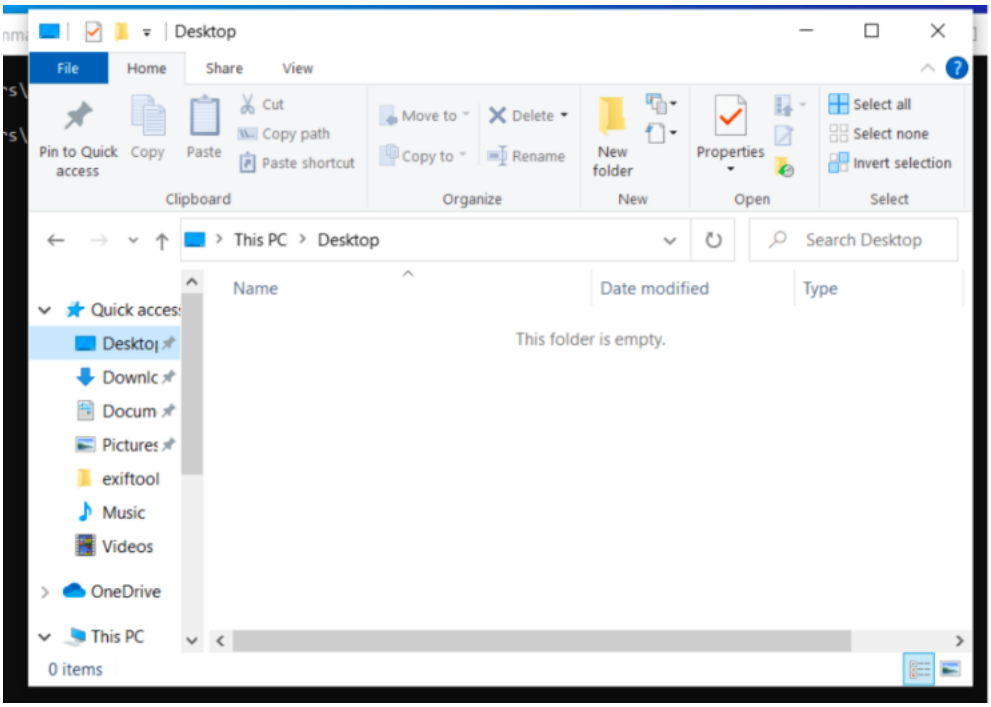
Part 3 – Hiding Files in Windows

- Created a new text file on the desktop

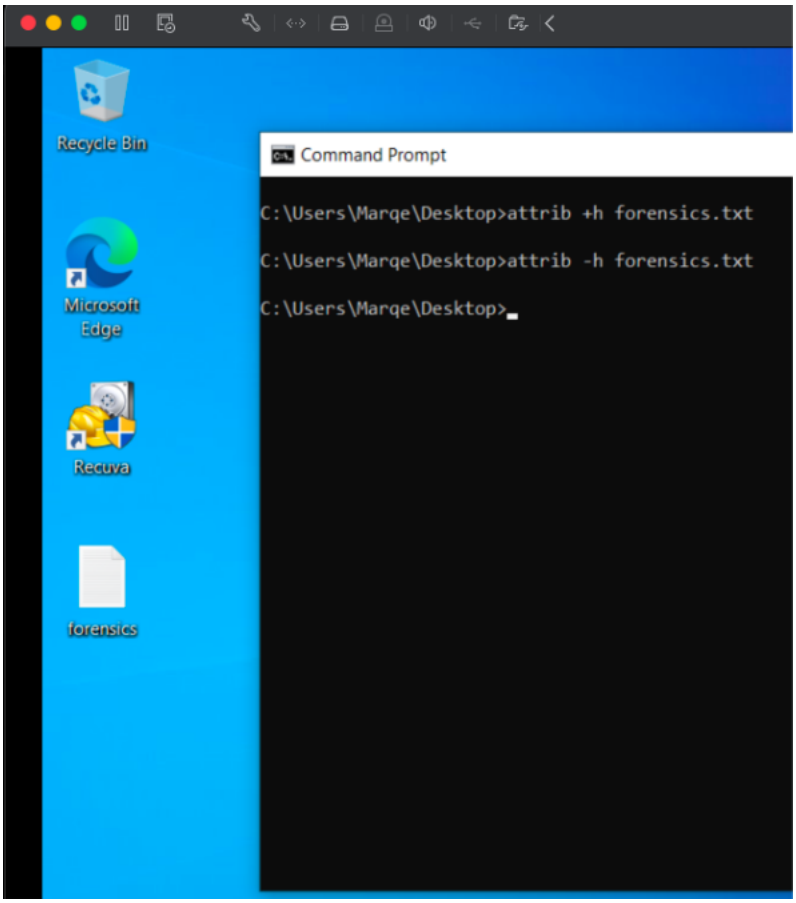
- Opened Command Prompt in Desktop
- Hid the file using:
- attrib +h forensics.txt



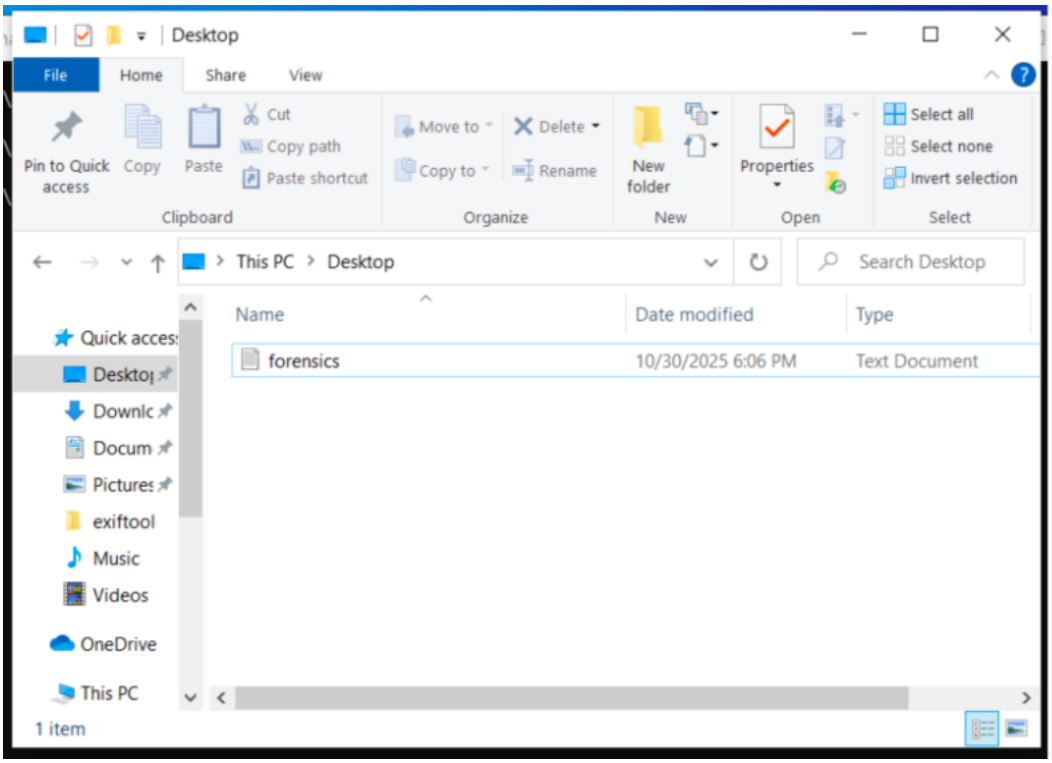
The file no longer appeared in File Explorer unless hidden items were enabled.



- Unhid the file:
- attrib -h filename.txt

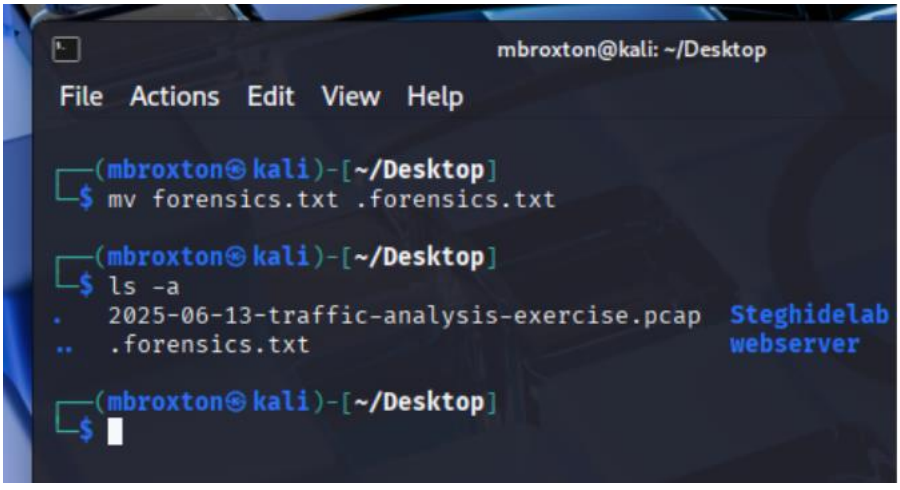


The file became visible again.

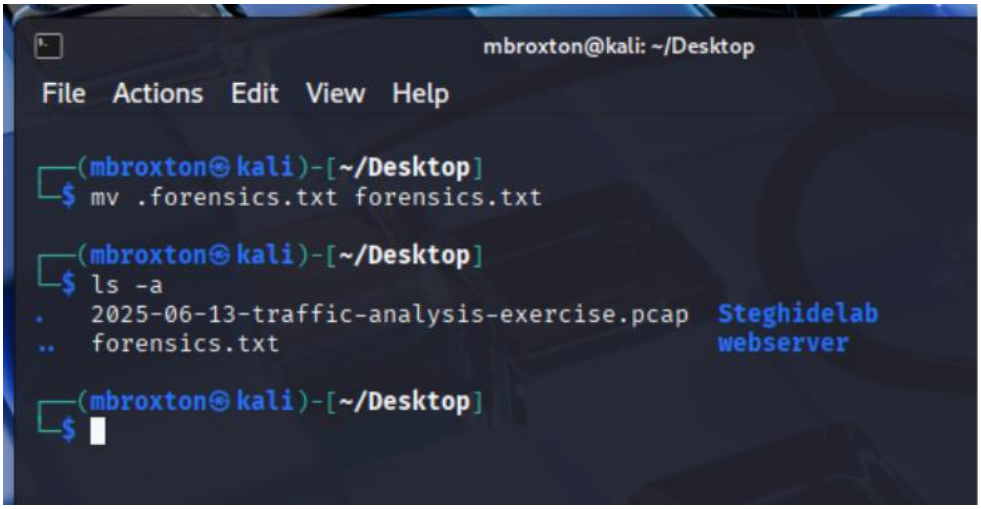


Part 5 – Hiding Files in Linux

- Created a text file on the Kali desktop.
- Used ls -a to show all files before hiding.
- Hid the file by renaming it with a dot prefix:
- mv forensics.txt .forensics.txt



- Ran ls -a again to confirm the file was now hidden from normal directory listings.
- Unhid the file by restoring its original name:
- mv .filename.txt filename.txt
- Verified visibility again using ls –a



Linux successfully hid the file using the dot-prefix method, and the file reappeared after the name was changed back.

5. Analyze Results

This lab demonstrated multiple methods attackers may use to conceal data. Using Diskpart, an entire partition can be hidden simply by removing the drive letter, making data invisible to average users. Windows file attributes provide a basic hiding mechanism by marking files as hidden, while Linux uses dot-prefixed filenames to hide files from standard directory listings.

From a forensic standpoint, these hiding methods are detectable. Hidden partitions can still be viewed in Disk Management, Diskpart, or forensic imaging tools. Hidden Windows files still exist in the file system and are displayed when hidden items are enabled. In Linux, running ls -a reveals hidden files. Understanding these behaviors is essential for identifying concealed data during investigations.

6. Conclusion

This lab reinforced the importance of knowing how files and partitions can be hidden across operating systems. The ability to hide entire volumes or individual files demonstrates techniques attackers may use to obscure evidence or store unauthorized data. Mastery of the tools and commands required to reveal these hidden items is critical for digital forensic analysts and incident responders.

7. Challenges and Observations

- Diskpart required administrative privileges to modify volume attributes.
- Removing the drive letter effectively hid the partition from users without altering its data.
- Windows hidden files could be easily revealed by enabling “show hidden files,” demonstrating the limitation of this method.
- Linux hiding relies on filename conventions rather than permission changes, making it simple to bypass via ls -a.

8. References

- Microsoft Diskpart Documentation
- Linux mv and ls manual pages
- NIST SP 800-86: Guide to Integrating Forensic Techniques
- Course materials provided by instructor

Course: NWIT 263 – Digital Forensics
Lab Number: 15

Lab Title: Virtual Machine Forensics – Recovering Deleted Snapshots
Professor: Reza Mirabrishami
Date: 11/27/2025

1. Introduction

This lab focused on the forensic analysis of virtual machine (VM) snapshot artifacts. Snapshots record point-in-time states of a VM and often contain valuable evidence about past user activity. When a snapshot is deleted, important artifacts may still be recoverable on the host system. The goal of this lab was to locate VM snapshot files, identify a missing snapshot, recover the deleted snapshot using forensic tools, mount it, and extract evidence from the earlier VM state.

2. Objectives

- Locate VirtualBox VM snapshot files on the host system.
- Identify a deleted snapshot by comparing the snapshot chain.
- Recover the deleted snapshot using a recovery tool.
- Mount the recovered snapshot using FTK Imager.
- Extract evidence and compare differences between the base disk and the recovered snapshot.
- Understand the forensic significance of VM snapshots during investigations.

3. Tools Used

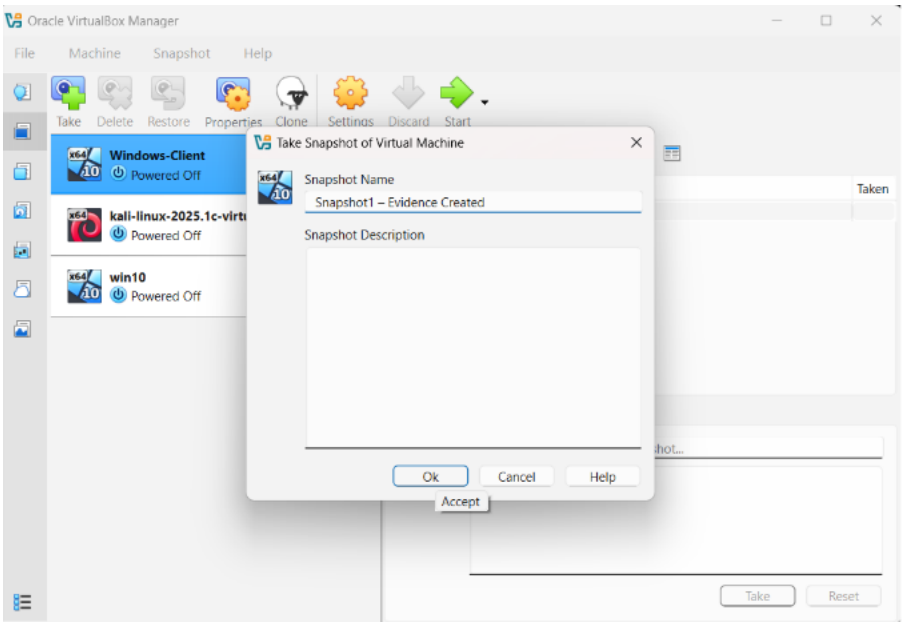
- VirtualBox
- Windows host system
- Recuva (file recovery)
- FTK Imager (mounting forensic images)
- VirtualBox VM configured with multiple snapshots

4. Lab Procedure and Results

Step 1 – Create a Snapshot Chain

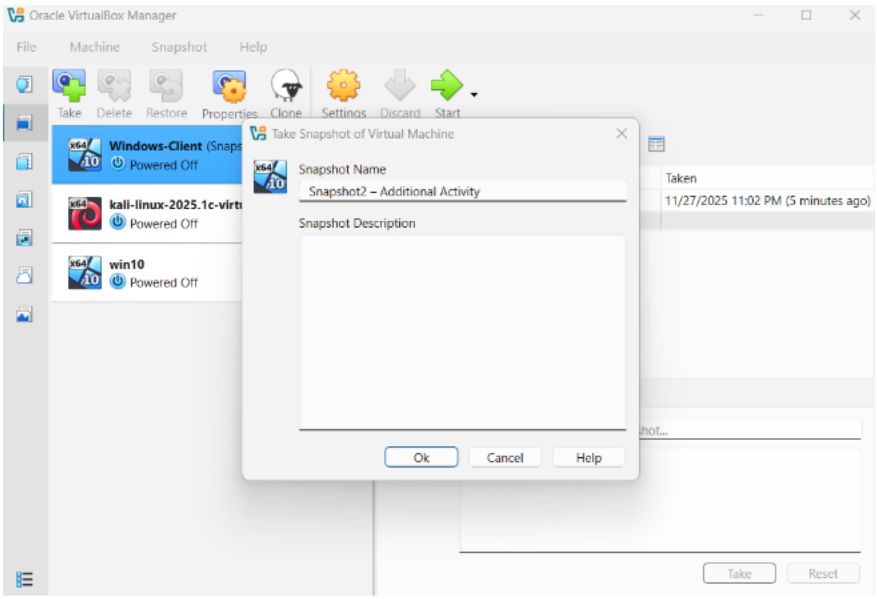
- Create a new file:

- Shut down the VM
- In VirtualBox → select VM → **Snapshots** → **Take Snapshot**
- Name: **Snapshot1 – Evidence Created**



- Start VM again → create another file:

- Shut down → Take another snapshot
- Name: **Snapshot2 – Additional Activity**



- **Delete Snapshot1**
- Right-click → **Delete Snapshot**

Step 2 — Locate VM Snapshot Files on the Host

- Navigated to the default VirtualBox directory:
- C:\Users\<Username>\VirtualBox VMs\<VM Name>\

- Located the following items:
  - Base disk .vdi
  - Snapshot differencing disks located under the *Snapshots* folder
  - Timestamps and file sizes indicating the order of snapshots
- Observed that the deleted snapshot no longer appeared in the Snapshots folder but was still referenced in historical activity.

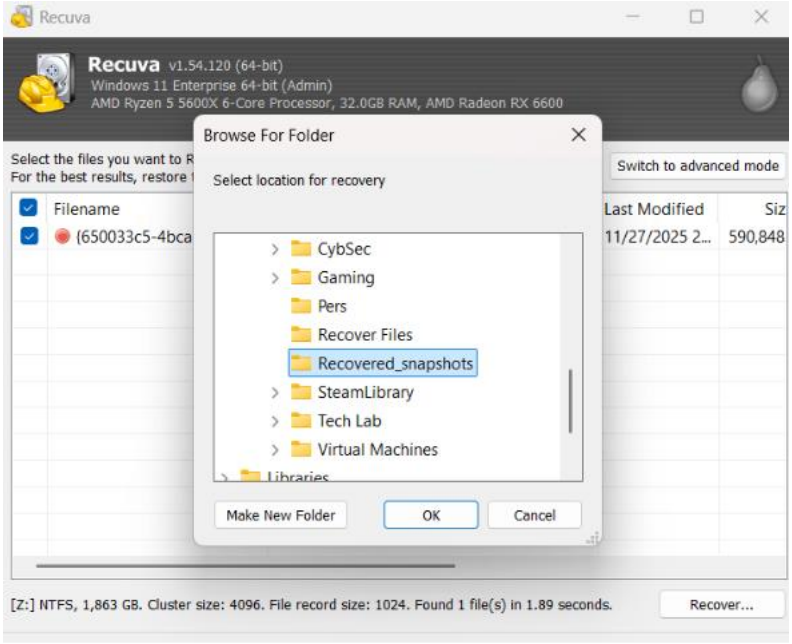
Step 3 – Identify the Missing Snapshot

- Compared the snapshots listed in VirtualBox Manager to the files present on the disk.
- Noted that one snapshot—"Snapshot1 – Evidence Created"—was missing.

- Identified the UUID associated with the missing snapshot by reviewing file names and VirtualBox Manager metadata.
- (Screenshot: VirtualBox snapshot list)

Step 4 – Recover the Deleted Snapshot

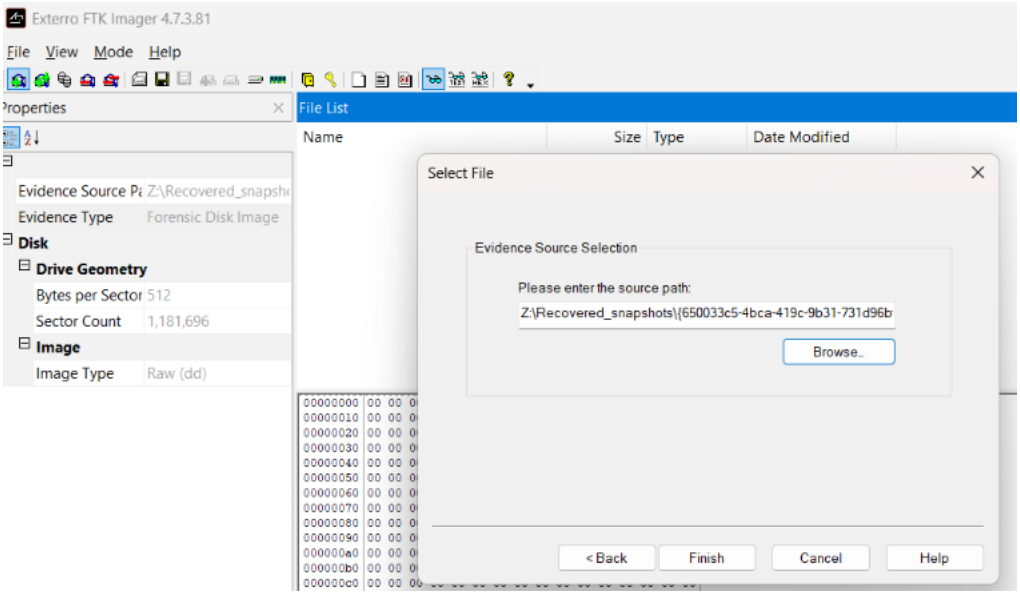
- Using Recuva:
- Launched Recuva and targeted the drive containing the VirtualBox VM directory.
- Searched for deleted .vdi files.
- Located a large deleted differencing disk associated with the missing snapshot, based on its UUID.
- Recovered the snapshot file to an external folder:
- E:\Recovered_Snapshots\



Step 5 – Mount the Recovered Snapshot in FTK Imager

- Opened FTK Imager.
- Selected:
- File → Add Evidence Item → Image File
- Imported the recovered snapshot .vdi file.
- FTK Imager successfully mounted the recovered snapshot as a readable volume.

- Navigated through the file system and identified evidence located only within the earlier machine state, such as:
    - evidence1.txt
    - User profile content
    - Older browser history



Step 6 – Compare Base Disk vs. Recovered Snapshot

- The comparison revealed the following differences:

Item	Base Disk	Recovered Snapshot
evidence1.txt	Missing	Present
evidence2.txt	Present	Missing
Browser history	Limited	More complete
System state	Current	Older/previous

- These differences demonstrated the timeline preserved by snapshots and the value of earlier states during forensic investigations.

5. Analyze Results

The recovered snapshot contained evidence that no longer existed in the current VM state. Although the suspect deleted the original snapshot, its differencing disk was still

recoverable on the host system. This confirmed that VM snapshots leave residual artifacts that can reveal historical user actions. Mounting the recovered snapshot allowed access to data that had been overwritten or deleted in the base image, supporting the importance of snapshot forensics in investigative work.

6. Conclusion

This lab demonstrated the significance of virtual machine snapshots as forensic artifacts. Even when deleted, snapshots can often be recovered and analyzed to reveal past system states. Through locating VM files, identifying a missing snapshot, recovering it, and mounting it for analysis, this lab reinforced the importance of VM forensics in modern investigations and highlighted how suspects may attempt to hide their activity through snapshot manipulation.

7. Challenges and Observations

- Identifying the correct snapshot UUID required careful comparison between files and VirtualBox Manager metadata.
- Snapshot deletion did not immediately remove the differencing disk, allowing successful recovery.
- Mounting the snapshot in FTK Imager provided a clear view of historical system data, illustrating how snapshots preserve time-based evidence.

8. References

- Oracle VirtualBox Documentation
- NIST SP 800-86: Guide to Integrating Forensic Techniques
- FTK Imager User Guide
- Course materials provided by instructor

Course: NWIT 263 – Digital Forensics

Lab Number: 16

Lab Title: Investigating macOS Artifacts on Windows

Professor: Reza Mirabrishami

Date: 11/28/2025

1. Introduction

This lab focused on investigating macOS applications and user activity artifacts from a Windows environment. macOS stores system preferences, user interface configurations, and application activity in structured files such as property list (.plist) files and SQLite databases. The objective of this lab was to examine a macOS sidebar configuration plist and Safari's browsing history database to understand how macOS records user activity and how these artifacts can be analyzed during a forensic investigation.

2. Objectives

- Extract and review macOS artifacts on a Windows machine.
- Analyze a .plist file containing user interface preferences.
- Examine Safari's History.db SQLite database.
- Identify URLs, titles, and timestamps of visited websites.
- Understand the forensic value of macOS application artifacts.

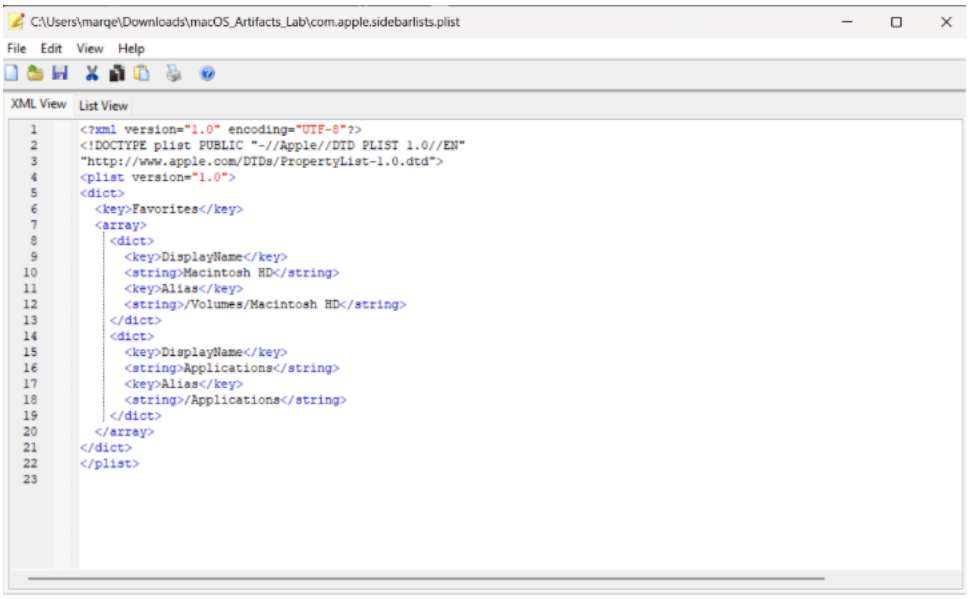
3. Tools Used

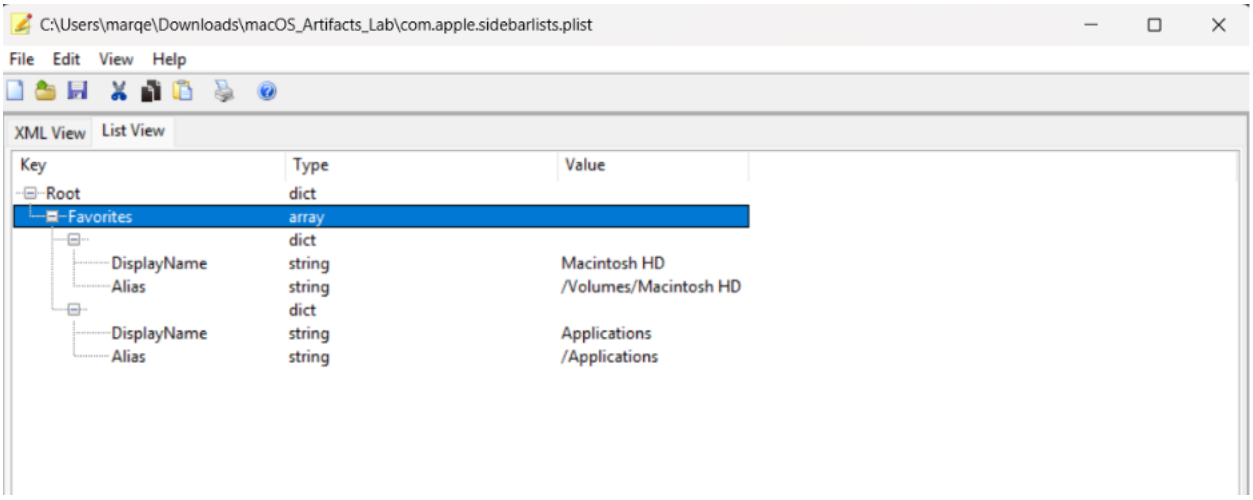
- Plist Editor for Windows
- DB Browser for SQLite
- Windows 10 computer
- macOS_Artifacts_Lab.zip (provided lab files)

4. Lab Procedure and Results

Step 1 – Examining the Sidebar .plist File

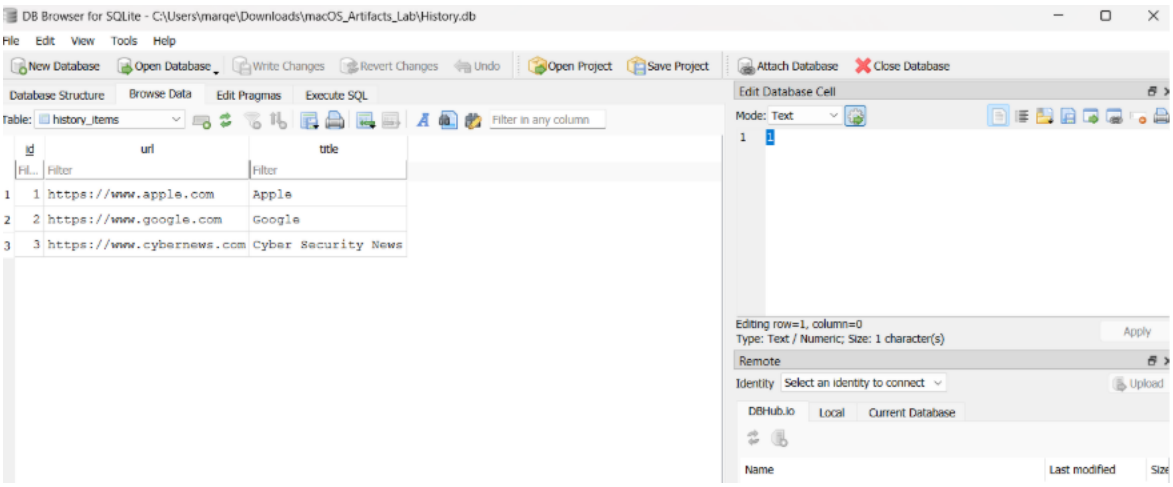
1. Unzipped the **macOS_Artifacts_Lab.zip** folder to the desktop.
2. Opened **com.apple.sidebarlists.plist** using the Plist Editor.
3. Navigated to the **Favorites** section.
4. Observed the items stored in the Finder sidebar configuration.

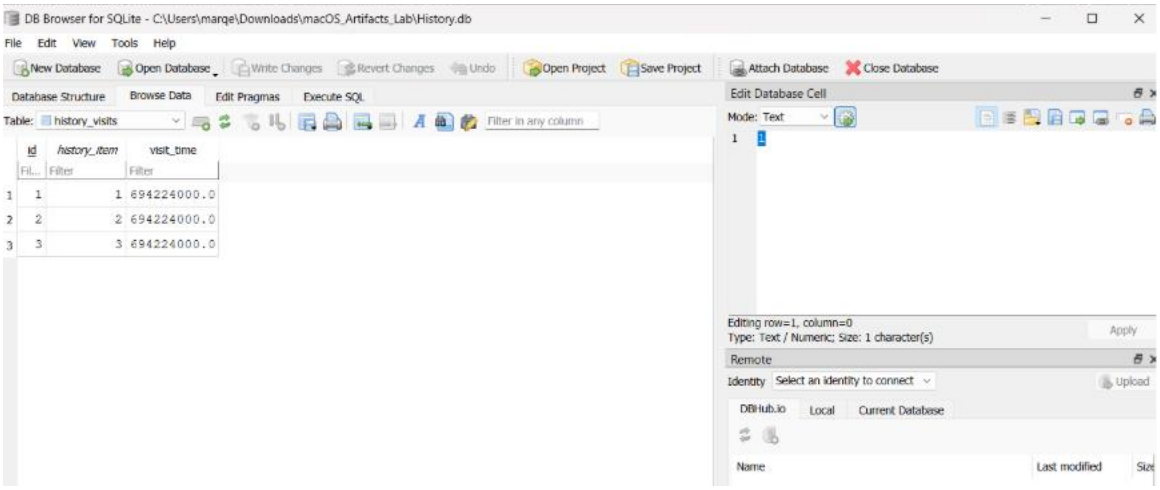




Step 2 – Analyzing Safari History Database

1. Opened **History.db** in DB Browser for SQLite.
2. Viewed **history_items** to review URLs and page titles.
3. Reviewed **history_visits** to examine Apple epoch-based timestamps.
4. Correlated URLs with visit times.





5. Analyze Results

The sidebar plist revealed Finder navigation preferences, including frequently accessed folders. The Safari History.db database provided detailed browsing activity, including visited URLs and timestamped events. Together, these artifacts demonstrated how macOS stores user activity in structured files. No suspicious browsing activity was detected and all looked normal.

6. Conclusion

This lab demonstrated how macOS artifacts, even when analyzed on Windows, provide valuable insight into user behavior and system activity. Plist files store configuration details, while Safari's SQLite database logs detailed browser history. Understanding these files is essential for macOS forensic investigations.

7. Challenges and Observations

- The plist displayed extensive configuration detail, requiring careful navigation.
- Apple epoch timestamps needed interpretation, but DB Browser allowed easy review.
- Both artifacts were readable and interpretable on Windows with the appropriate tools.

8. References

- Apple Developer Documentation
- SQLite Format Specifications
- Course lecture materials

Course: NWIT 263 – Digital Forensics
Lab Number: 17

Lab Title: Investigating Suspected Data Exfiltration with Autopsy

Professor: Reza Mirabrishami
Date: 12/14/2025

1. Introduction

The purpose of this lab was to introduce the use of Autopsy for investigating a suspected data exfiltration incident. A logical set of files was analyzed to determine whether a user copied sensitive company data to removable media or cloud services. The investigation focused on USB usage, browser activity, stored credentials, and internal project documentation.

2. Objectives

- Create a new forensic case using Autopsy
- Add logical files as a data source
- Use ingest modules to analyze evidence
- Identify indicators of data exfiltration
- Analyze artifacts related to USB usage, cloud storage, and credentials

3. Tools Used

- Autopsy (Windows version)
- Windows 10
- Provided evidence files (logical data source)

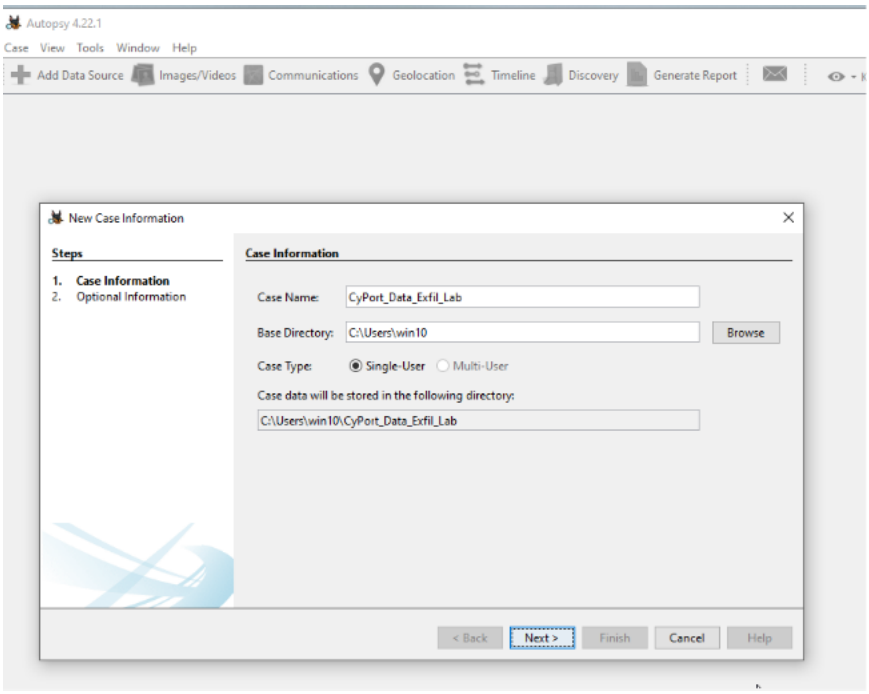
4. Evidence Description

The provided evidence consisted of a logical collection of files extracted from a suspect's workstation. The data included text documents, a browser history CSV file, a USB device log, and an image related to the CyPort project. These files were analyzed without modification to preserve integrity

5. Lab Procedure

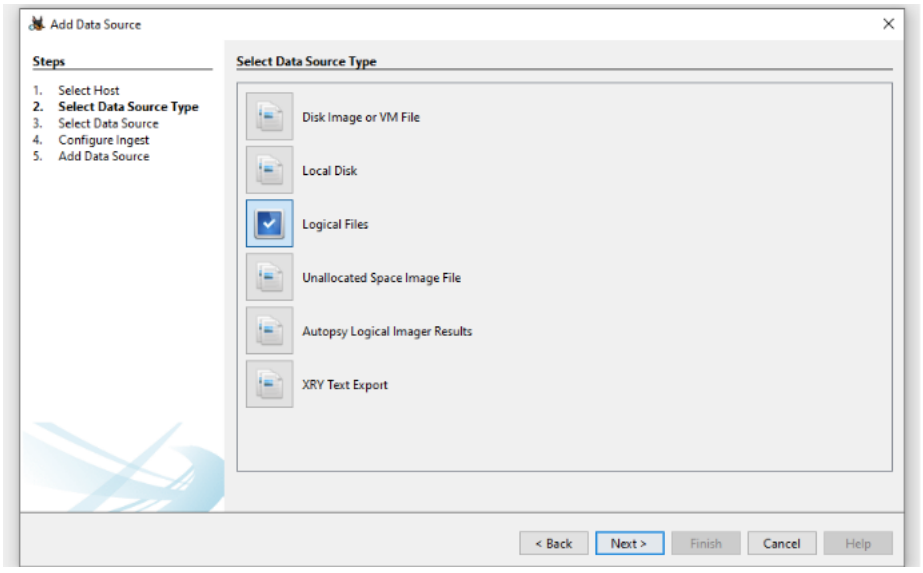
Step 1: Case Creation

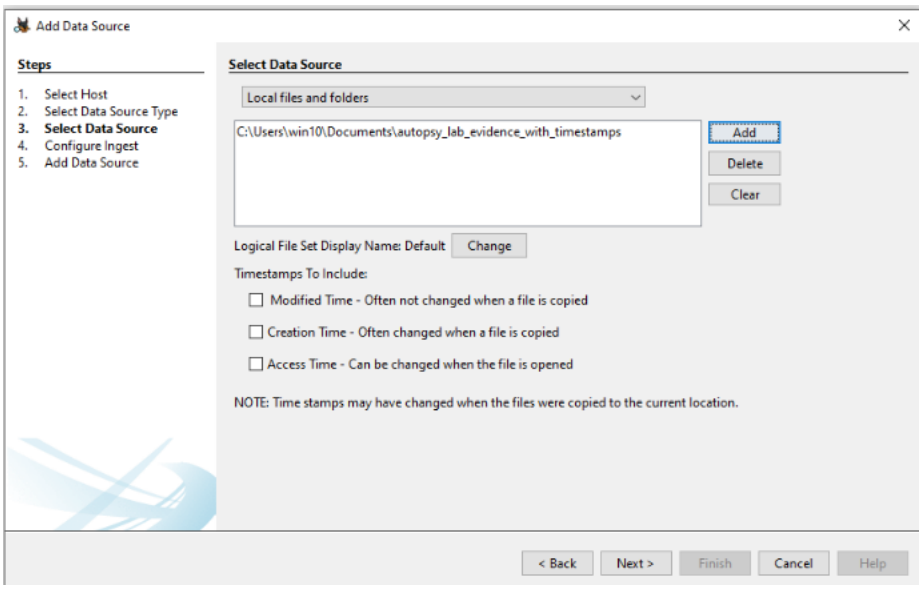
- A new case named **CyPort_Data_Exfil_Lab** was created in Autopsy, and a base directory was selected to store case files.



Step 2: Adding Data Source

- The extracted evidence folder was added as a **Logical Files** data source. This allowed Autopsy to analyze individual files without requiring a disk image.



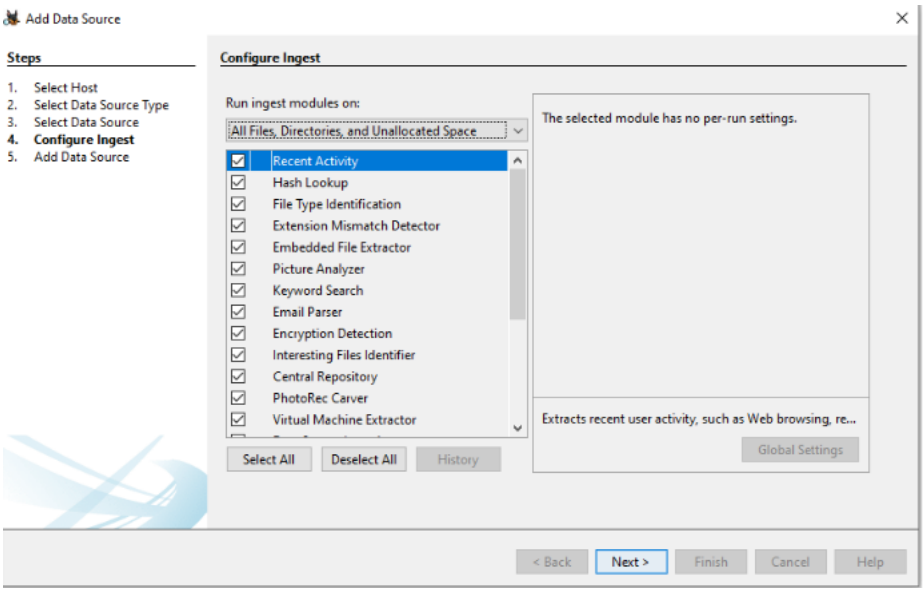


Step 3: Ingest Modules

The following ingest modules were enabled:

- File Type Identification
- Keyword Search

Autopsy processed the files and categorized them for analysis.



Step 4: Evidence Analysis

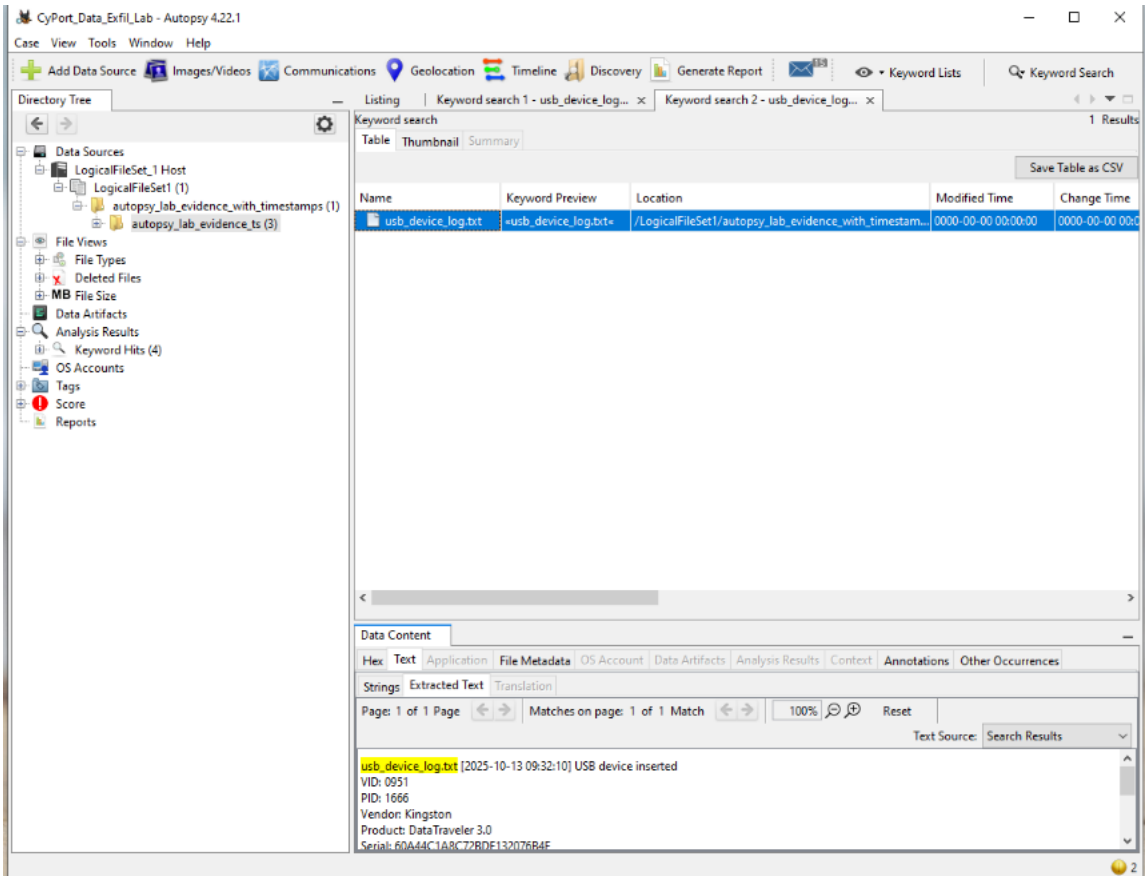
The following artifacts were examined:

- `usb_device_log.txt` to identify USB device usage
- `browser_history_mock.csv` to identify cloud and email services
- `Suspected_Notes.txt` to determine intent
- `Password_List_WARNING.txt` to identify exposed credentials
- `Project_CyPort_Overview.txt` and `cyport_diagram_mock.png` to understand the sensitivity of the data

6. Analysis and Findings

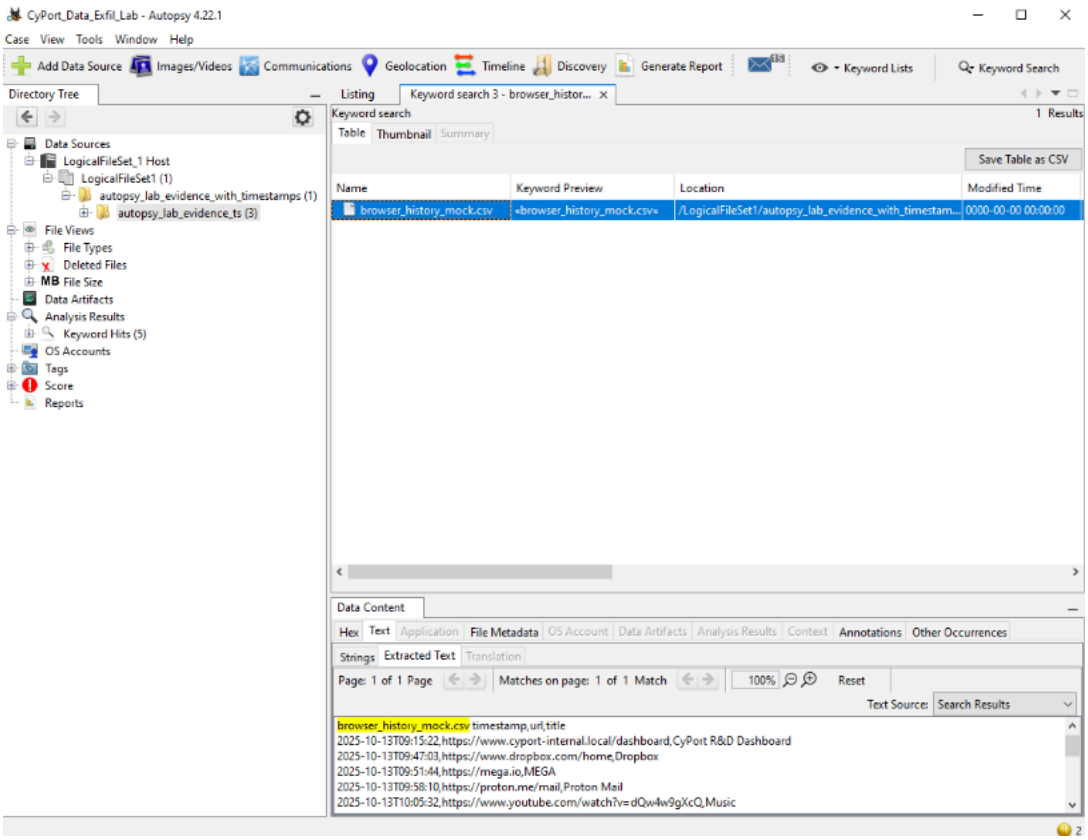
USB Device Evidence

The USB log showed that a removable USB device was connected to the system. The log included the device vendor, product name, volume label, and timestamps indicating when the device was inserted and removed.



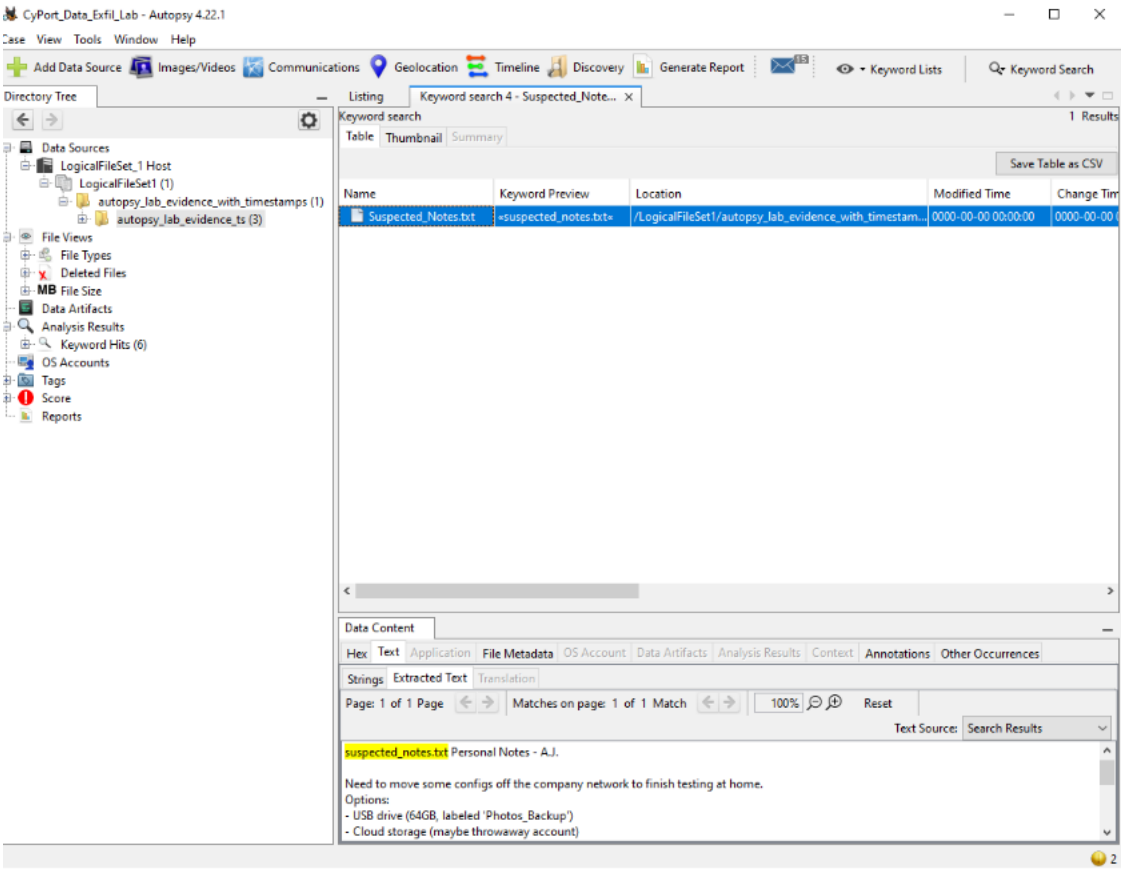
Cloud Storage and Email Activity

The browser history file contained evidence of visits to cloud storage and email services such as Dropbox, MEGA, and Proton Mail. Timestamps indicated these services were accessed around the suspected timeframe.



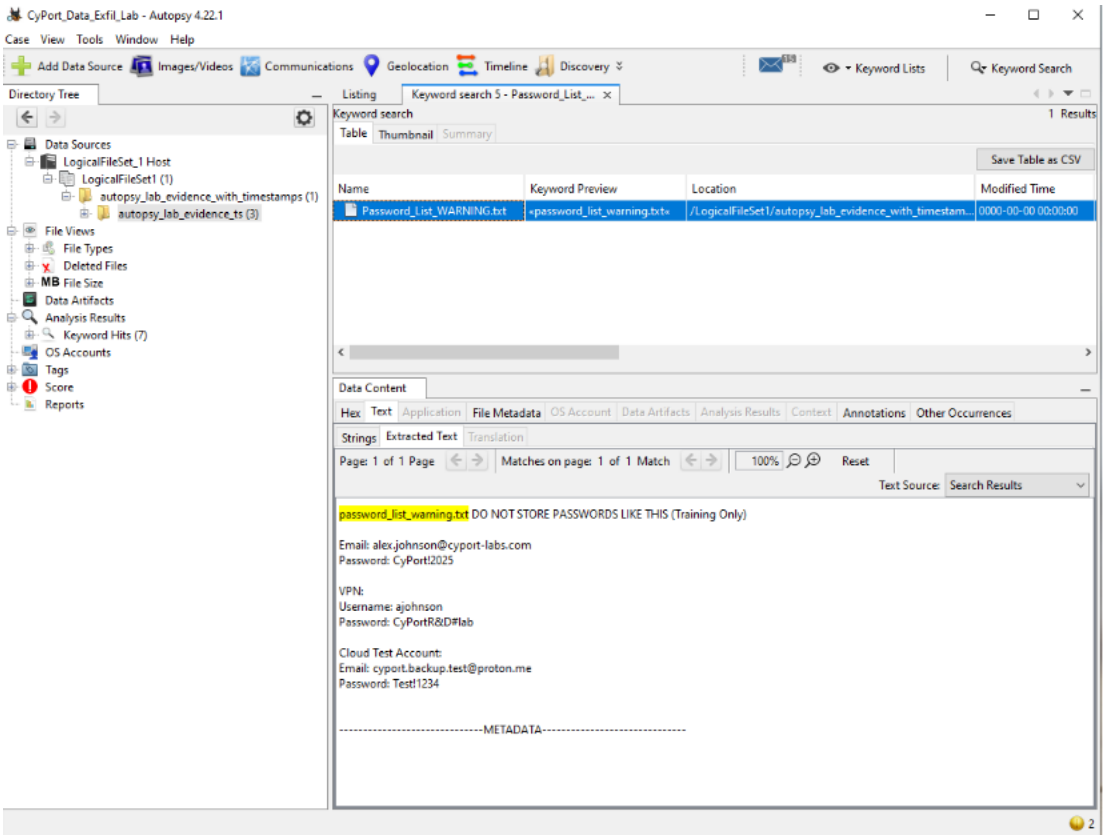
Evidence of Intent

The file **Suspected_Notes.txt** contained statements suggesting an intent to move company data off the network. References were made to using a USB drive and cloud storage while avoiding detection.



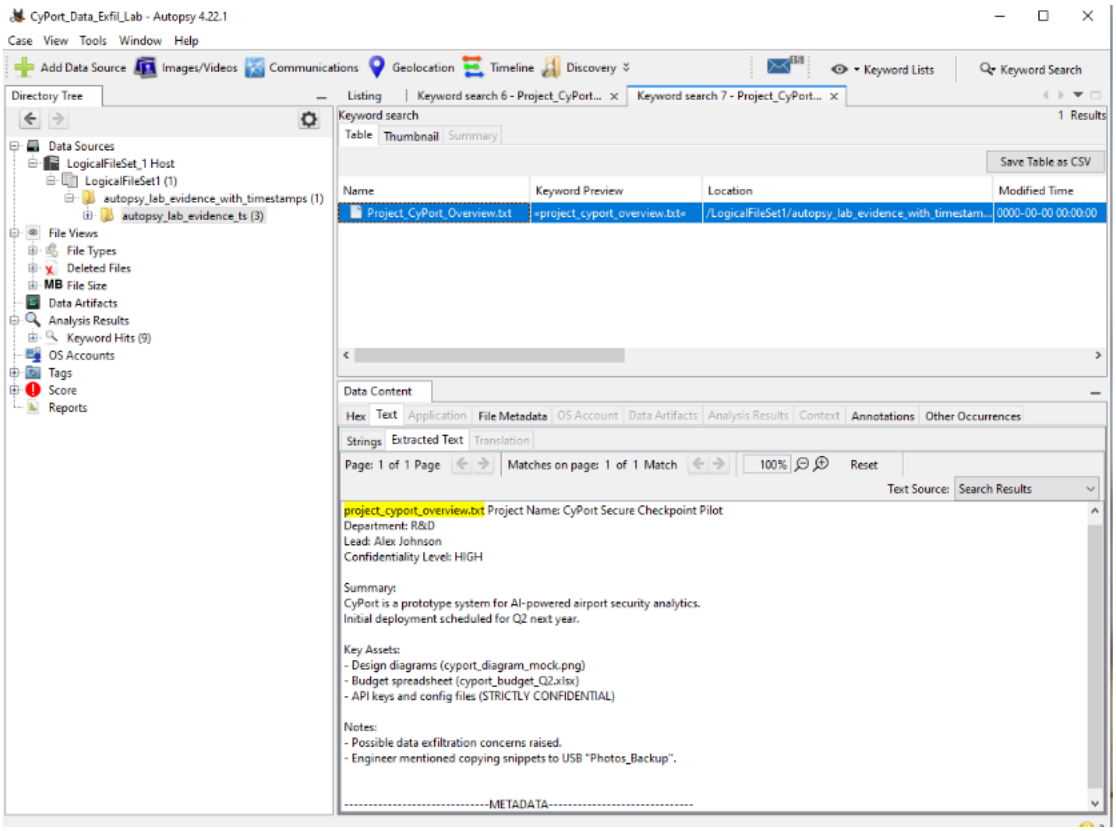
Sensitive Credentials

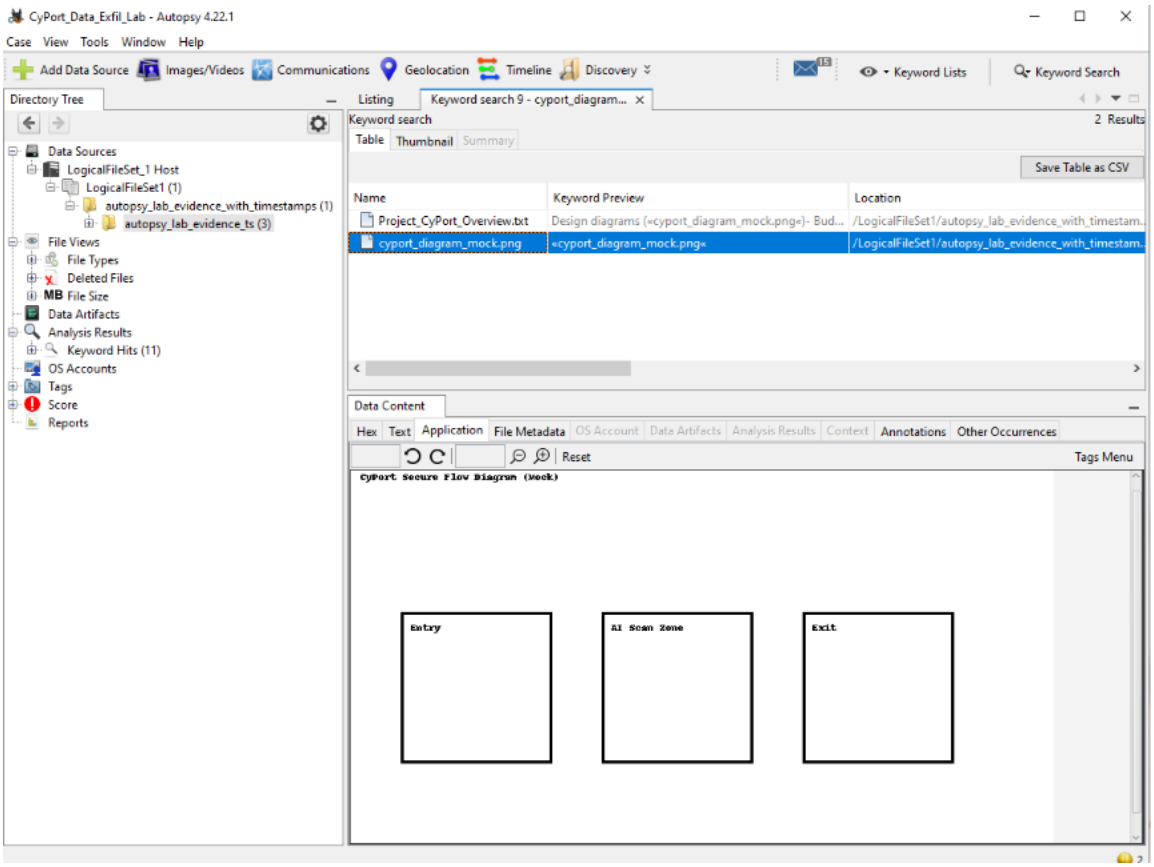
The **Password_List_WARNING.txt** file contained plaintext credentials. Storing credentials in this manner represents poor security practice and increases the risk of unauthorized access.



Project Sensitivity

The CyPort project overview and diagram revealed internal system architecture and design details. Such information would be considered sensitive intellectual property and valuable to unauthorized parties.


CyPort_Data_Exfil_Lab - Autopsy 4.22.1
Case View Tools Window Help
Add Data Source
Images/Videos
Communications
Geolocation
Timeline
Discovery
Keyword Lists
Keyword Search
Directory Tree
Listing
Keyword search 6 - Project_CyPort...
Keyword search 7 - Project_CyPort...
Keyword search
1 Results
Table
Thumbnail
Summary
Save Table as CSV
Name
Keyword Preview
Location
Modified Time
Project_CyPort_Overview.txt
«project_cyport_overview.txt«
/LogicalFileSet1/autopsy_lab_evidence_with_timestam...
0000-00-00 00:00:00
Data Sources
LogicalFileSet_1 Host
LogicalFileSet1 (1)
autopsy_lab_evidence_with_timestamps (1)
autopsy_lab_evidence_ts (3)
File Views
File Types
Deleted Files
File Size
Data Artifacts
Analysis Results
Keyword Hits (9)
OS Accounts
Tags
Score
Reports
Data Content
Hex
Text
Application
File Metadata
OS Account
Data Artifacts
Analysis Results
Context
Annotations
Other Occurrences
Strings
Extracted Text
Translation
Page: 1 of 1 Page
Matches on page: 1 of 1 Match
100%
Reset
Text Source:
Search Results
project_cyport_overview.txt Project Name: CyPort Secure Checkpoint Pilot
Department: R&D
Lead: Alex Johnson
Confidentiality Level: HIGH
Summary:
CyPort is a prototype system for AI-powered airport security analytics.
Initial deployment scheduled for Q2 next year.
Key Assets:
- Design diagrams (cyport_diagram_mock.png)
- Budget spreadsheet (cyport_budget_Q2.xlsx)
- API keys and config files (STRICTLY CONFIDENTIAL)
Notes:
- Possible data exfiltration concerns raised.
- Engineer mentioned copying snippets to USB "Photos_Backup".
----------METADATA----------



7. Conclusion

Based on the analysis of USB logs, browser history, notes, and credential files, the evidence supports the suspicion of data exfiltration. The artifacts indicate that a USB device was used, cloud services were accessed, and sensitive project information and credentials were exposed. These findings demonstrate how Autopsy can be used to identify and correlate multiple indicators of data exfiltration.

8. Security Recommendations

- Enforce encryption and access controls on removable media
- Prohibit storage of credentials in plaintext files
- Monitor and restrict access to personal cloud storage services
- Implement user activity logging and alerting for sensitive data access

9. References

- Carrier, B. (2005). *File System Forensic Analysis*. Addison-Wesley.
- The Sleuth Kit & Autopsy. (n.d.). *Autopsy Digital Forensics Tool*. https://www.sleuthkit.org/autopsy/
- NIST. (2014). *Guide to Integrating Forensic Techniques into Incident Response (SP 800-86)*. https://csrc.nist.gov/publications/detail/sp/800-86/final
- Course materials and lab instructions provided by the instructor.

Course: NWIT 263 – Digital Forensics
Lab Number: 18

Lab Title: Cloud Forensic

Professor: Reza Mirabrishami
Date: 12/14/2025

1. Introduction

This lab focused on performing a cloud forensic investigation using AWS CloudTrail logs to analyze a suspected multi-stage attack within an Amazon Web Services environment. The organization, CyberDomE Cloud Services, experienced suspicious activity involving unauthorized login attempts, abnormal S3 access patterns, IAM privilege escalation, and attempts to tamper with logging infrastructure. The objective of this lab was to analyze the CloudTrail log file and determine how the attacker gained access, what actions were performed, whether privileges were escalated, and whether anti-forensic techniques were attempted.

2. Objectives

- Analyze AWS CloudTrail logs for suspicious activity
- Identify the initial point of access
- Determine S3 reconnaissance and data access
- Detect IAM privilege escalation and persistence mechanisms
- Identify anti-forensic behavior targeting logging services
- Build an attack timeline
- Recommend incident response and security controls

3. Tools Used

- AWS CloudTrail log file (`cloudtrail_complex.json`)
- Visual Studio Code / JSON Viewer
- Windows 10 system

4. Evidence Description

The primary evidence examined was a CloudTrail log file provided in the lab dataset. CloudTrail records AWS API calls, including authentication events, S3 object access, IAM changes, and logging configuration actions. This log served as the authoritative source for reconstructing the attacker's actions during the incident.

5. Analysis and Findings

5.1 Initial Access

- **Q1.1 – First Suspicious Successful Login**
- **Username:** billing-support
- **Timestamp (UTC):** 2025-01-16T03:41:05Z
- **Source IP Address:** 185.243.112.17
- **MFA Status:** MFA not used
- **Normal Behavior:** This login was not normal behavior due to the foreign IP location, lack of MFA, and time of access outside normal operating hours

5.2 S3 Reconnaissance and Data Exfiltration

- **Q2.1 – Enumerated S3 Buckets**
  The attacker enumerated the following S3 buckets in the ListBuckets event at 2025-01-16T03:42:10Z:
  - logs-cd-org
  - cd-sensitive-records
  - public-assets
- **Q2.2 – Sensitive Bucket Identified**
  The bucket that appears to contain sensitive files is cd-sensitive-records, as indicated by the file names enumerated in the ListBucket event at 2025-01-16T03:43:02Z (e.g., contract_2025.pdf, finance_summary.xlsx, clients_list.csv).
- **Q2.3 – Accessed or Downloaded Files**
  The attacker downloaded or accessed the following five files (via GetObject events between 03:43:30Z and 03:47:30Z):
  - contract_2025.pdf
  - finance_summary.xlsx
  - clients_list.csv
  - internal_notes.docx
  - network_diagram.png

```
{
    "eventVersion": "1.08",
    "userIdentity": {
        "type": "IAMUser",
        "userName": "billing-support"
    },
    "eventTime": "2025-01-16T03:42:10Z",
    "eventSource": "s3.amazonaws.com",
    "eventName": "ListBuckets",
    "sourceIPAddress": "185.243.112.17",
    "awsRegion": "us-east-1",
    "requestParameters": null,
    "responseElements": {
        "buckets": [
            {
                "name": "logs-cd-org"
            },
            {
                "name": "cd-sensitive-records"
            },
            {
                "name": "public-assets"
            }
        ]
```

```
{
    "eventVersion": "1.08",
    "userIdentity": {
        "type": "IAMUser",
        "userName": "billing-support"
    },
    "eventTime": "2025-01-16T03:43:02Z",
    "eventSource": "s3.amazonaws.com",
    "eventName": "ListBucket",
    "sourceIPAddress": "185.243.112.17",
    "awsRegion": "us-east-1",
    "requestParameters": {
        "bucketName": "cd-sensitive-records"
    },
    "responseElements": {
        "contents": [
            {
                "key": "contract_2025.pdf"
            },
            {
                "key": "finance_summary.xlsx"
            },
            {
                "key": "clients_list.csv"
            },
            {
                "key": "internal_notes.docx"
            },
            {
                "key": "network_diagram.png"
            }
        ]
```

5.3 Privilege Escalation

- **Q3.1 – Did the attacker create a new IAM user? If yes:**

  Yes, via the CreateUser event at 2025-01-16T03:47:41Z.

  Username: sys-admin2

  Timestamp of creation: 2025-01-16T03:47:41Z

- **Q3.2 – Which AWS policy was attached to this new user? Was it an admin-level policy?**
  AWS policy attached: arn:aws:iam::aws:policy/AdministratorAccess (event AttachUserPolicy at 2025-01-16T03:48:12Z).

  Yes, AdministratorAccess grants full access to all AWS resources and services.

- **Q3.3 – Did the attacker create any access keys for persistence?**

  Yes, a CreateAccessKey event was logged at 2025-01-16T03:50:03Z for the new user sys-admin2. Creating access keys allows non-console, programmatic access that can persist even if the console password is changed.

```
{
    "eventVersion": "1.08",
    "userIdentity": {
        "type": "IAMUser",
        "userName": "billing-support"
    },
    "eventTime": "2025-01-16T03:47:41Z",
    "eventSource": "iam.amazonaws.com",
    "eventName": "CreateUser",
    "sourceIPAddress": "185.243.112.17",
    "awsRegion": "us-east-1",
    "requestParameters": {
        "userName": "sys-admin2"
    }
```



```
{
    "eventVersion": "1.08",
    "userIdentity": {
        "type": "IAMUser",
        "userName": "billing-support"
    },
    "eventTime": "2025-01-16T03:48:12Z",
    "eventSource": "iam.amazonaws.com",
    "eventName": "AttachUserPolicy",
    "sourceIPAddress": "185.243.112.17",
    "awsRegion": "us-east-1",
    "requestParameters": {
        "userName": "sys-admin2",
        "policyArn": "arn:aws:iam::aws:policy/AdministratorAccess"
    }
},
{
    "eventVersion": "1.08",
    "userIdentity": {
        "type": "IAMUser",
        "userName": "billing-support"
    },
    "eventTime": "2025-01-16T03:50:03Z",
    "eventSource": "iam.amazonaws.com",
    "eventName": "CreateAccessKey",
    "sourceIPAddress": "185.243.112.17",
    "awsRegion": "us-east-1",
    "requestParameters": {
        "userName": "sys-admin2"
```


5.4 Persistence and Role Abuse

- **Q4.1 – Did the attacker assume any privileged IAM roles?**

  Yes. A successful AssumeRole event is logged at 2025-01-16T03:51:27Z.

  Role ARN: arn:aws:iam::123456789012:role/AdminRole

Timestamp: 2025-01-16T03:51:27Z

This is dangerous because a privileged role like AdminRole grants the attacker temporary, high-level administrative permissions, often bypassing the explicit permissions of the original compromised user (billing-support). This allows the attacker to execute actions like the subsequent anti-forensics attempts and further exploit the environment under a new identity (sys-admin2 being the session name of the assumed role).

5.5 Anti-Forensics Activity

- **Q5.1 – Did the attacker attempt to delete or disable CloudTrail logs?**
  Look for: DeleteTrail, StopLogging.

  DeleteTrail attempt: Yes, at 2025-01-16T03:53:09Z for trail "OrgTrail".

  StopLogging attempt: Yes, at 2025-01-16T03:53:45Z for trail "OrgTrail".

  The attacker attempted to disable or delete logging using `StopLogging` and `DeleteTrail` API calls.

- **Q5.2 – Did they attempt to delete the S3 bucket containing logs?**
  Yes, a DeleteBucket attempt for "logs-cd-org" occurred at 2025-01-16T03:54:30Z. Based on the name logs-cd-org from the ListBuckets

event, this is likely the bucket used for CloudTrail logging. An attempt was made to delete the S3 bucket containing CloudTrail logs.

- **Q5.3 – Did these anti-forensic attempts succeed or fail? Justify your answer using event fields.**

  All anti-forensic attempts Failed.

  Justification: The events DeleteTrail (03:53:09Z), StopLogging (03:53:45Z), and DeleteBucket (03:54:30Z) all contain either an errorCode or an errorMessage.

  DeleteTrail and StopLogging have errorCode: "AccessDenied" and errorMessage: "User is not authorized to perform: cloudtrail:...".

  DeleteBucket has errorCode: "AccessDenied" and errorMessage: "Access Denied".

  The presence of these fields indicates the API calls were explicitly denied by AWS IAM permissions.

```
{
    "eventVersion": "1.08",
    "userIdentity": {
        "type": "AssumedRole",
        "arn": "arn:aws:sts::123456789012:assumed-role/AdminRole/sys-admin2"
    },
    "eventTime": "2025-01-16T03:53:09Z",
    "eventSource": "cloudtrail.amazonaws.com",
    "eventName": "DeleteTrail",
    "sourceIPAddress": "185.243.112.17",
    "awsRegion": "us-east-1",
    "requestParameters": {
        "name": "OrgTrail"
    },
    "errorCode": "AccessDenied",
    "errorMessage": "User is not authorized to perform: cloudtrail:DeleteTrail"
},
{
    "eventVersion": "1.08",
    "userIdentity": {
        "type": "AssumedRole",
        "arn": "arn:aws:sts::123456789012:assumed-role/AdminRole/sys-admin2"
    },
    "eventTime": "2025-01-16T03:53:45Z",
    "eventSource": "cloudtrail.amazonaws.com",
    "eventName": "StopLogging",
    "sourceIPAddress": "185.243.112.17",
    "awsRegion": "us-east-1",
    "requestParameters": {
        "name": "OrgTrail"
    },
    "errorCode": "AccessDenied",
    "errorMessage": "User is not authorized to perform: cloudtrail:StopLogging"
},
```



```
{
    "eventVersion": "1.08",
    "userIdentity": {
        "type": "AssumedRole",
        "arn": "arn:aws:sts::123456789012:assumed-role/AdminRole/sys-admin2"
    },
    "eventTime": "2025-01-16T03:54:30Z",
    "eventSource": "s3.amazonaws.com",
    "eventName": "DeleteBucket",
    "sourceIPAddress": "185.243.112.17",
    "awsRegion": "us-east-1",
    "requestParameters": {
        "bucketName": "logs-cd-org"
    },
    "errorCode": "AccessDenied",
    "errorMessage": "Access Denied"
}
```


6. Attack Timeline

- Suspicious IAM login from foreign IP without MFA: **2025-01-16T03:41:05Z**

- Enumeration of S3 buckets: **2025-01-16T03:42:10Z**
- Access and download of sensitive S3 objects: **2025-01-16T03:43:30Z**
- Creation of a new IAM user: **2025-01-16T03:47:41Z**
- Attachment of an administrative policy: **2025-01-16T03:48:12Z**
- Creation of access keys for persistence: **2025-01-16T03:50:03Z**
- Assumption of a privileged IAM role: **2025-01-16T03:51:27Z**
- Attempts to disable or delete CloudTrail logging: **2025-01-16T03:53:09Z**
- Failed attempts to delete log storage: **2025-01-16T03:54:30Z**

7. Incident Response and Recommendations

- **Q7.1 — What immediate actions should the Incident Response team take?**

  Containment: Immediately disable/delete the malicious IAM user sys-admin2 and its associated access keys (AKIAFAKEACCESSKEYID).

  Containment: Force a password reset and immediately revoke all active sessions for the compromised user billing-support.

  Eradication: Review all IAM actions performed by billing-support and the sys-admin2 assumed role to ensure no other backdoors (e.g., other users, roles, or configurations) were created.

  Forensics: Isolate the suspicious source IP address 185.243.112.17 and begin a deeper investigation into the scope of the compromise.

- **Q7.2 — What long-term security controls should the organization implement?**

  MFA Enforcement: Enforce Multi-Factor Authentication (MFA) for all AWS IAM users, especially those with console access.

  Least Privilege: Review and significantly restrict the IAM policies attached to the billing-support user. It should not have permissions to perform IAM actions (CreateUser, AttachUserPolicy) or broadly enumerate S3 buckets (ListBuckets) outside of its job function.

  Administrative Access: Implement break-glass procedures for administrative access. No human user should have persistent

AdministratorAccess. Instead, users should assume a time-bound role with administrative permissions that requires MFA.

Guardrails/Detective Controls: Implement IAM Access Analyzer and Service Control Policies (SCPs) to prevent users from creating new users or attaching AdministratorAccess policies outside of a dedicated administrative account/process.

Logging Protection: Implement bucket policies on the CloudTrail S3 bucket (logs-cd-org) that prevent the deletion of objects or the bucket itself, even by the root user, to improve anti-forensics resilience.

8. Indicators of Compromise (IOCs)

- **Suspicious IP Address:** 185.243.112.17
- **Abnormal Timestamps:** 2025-01-16T03:39:52Z to 03:41:05Z
- **Unexpected API Calls:**
  - `CreateUser`
  - `AttachUserPolicy`
  - `CreateAccessKey`
  - `GetObject`
  - `StopLogging`
  - `DeleteTrail`

9. Conclusion

This lab demonstrated how AWS CloudTrail logs can be used to reconstruct a complex cloud-based attack involving unauthorized access, privilege escalation, data exfiltration, persistence mechanisms, and anti-forensic behavior. The investigation highlighted the importance of continuous monitoring, strict IAM controls, and protected logging infrastructure in cloud environments.

10. References

- Carrier, B. (2005). File System Forensic Analysis. Addison-Wesley.
- The Sleuth Kit & Autopsy. (n.d.). Autopsy Digital Forensics Tool. https://www.sleuthkit.org/autopsy/

- NIST. (2014). Guide to Integrating Forensic Techniques into Incident Response (SP 800-86). https://csrc.nist.gov/publications/detail/sp/800-86/final
- Course materials and lab instructions provided by the instructor.

Course: NWIT 263 – Digital Forensics
Lab Number: 19

Lab Title: Detecting an Incident with Logs

Professor: Reza Mirabrishami
Date: 12/19/2025

1. Introduction

The purpose of this lab was to detect signs of a potential security incident by analyzing system logs. Logs provide a detailed record of authentication events, privilege usage, and system activity, making them essential during incident response investigations. In this lab, Windows Event Viewer and Linux authentication logs were examined to identify suspicious login attempts and extract Indicators of Compromise (IOCs).

2. Objectives

- Analyze Windows Security Event Logs for suspicious activity
- Identify failed and successful login attempts
- Detect abnormal login patterns and privilege usage
- Analyze Linux authentication logs for unauthorized access attempts
- Extract and document Indicators of Compromise (IOCs)

3. Tools Used

- Windows Event Viewer

- Linux Terminal
- Windows 10 system
- Kali Linux system
- Administrative privileges

4. Lab Procedure and Results

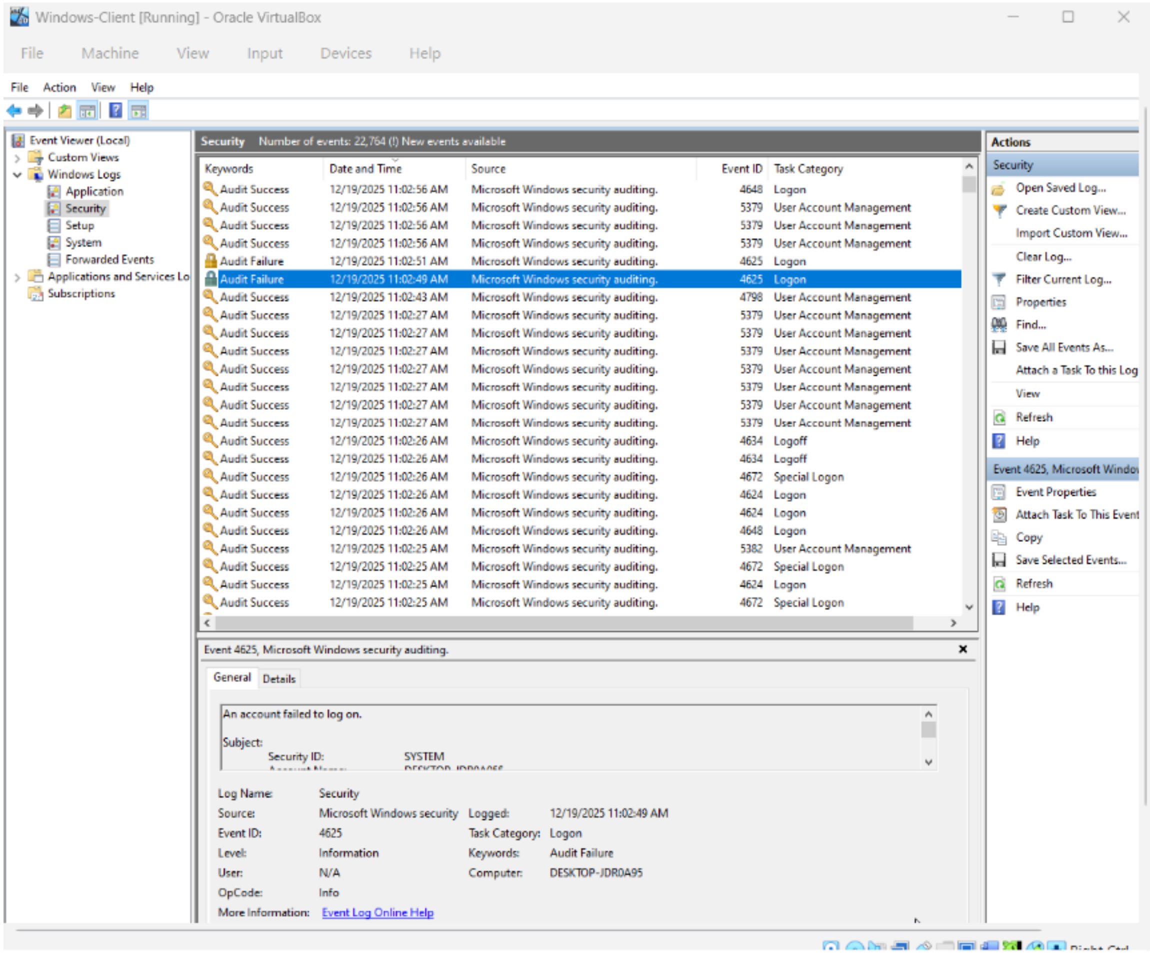
Part 1 – Windows Event Log Analysis

Step 1: Access Security Logs

- Opened **Event Viewer** on the Windows system.
- Navigated to:
- Event Viewer → Windows Logs → Security

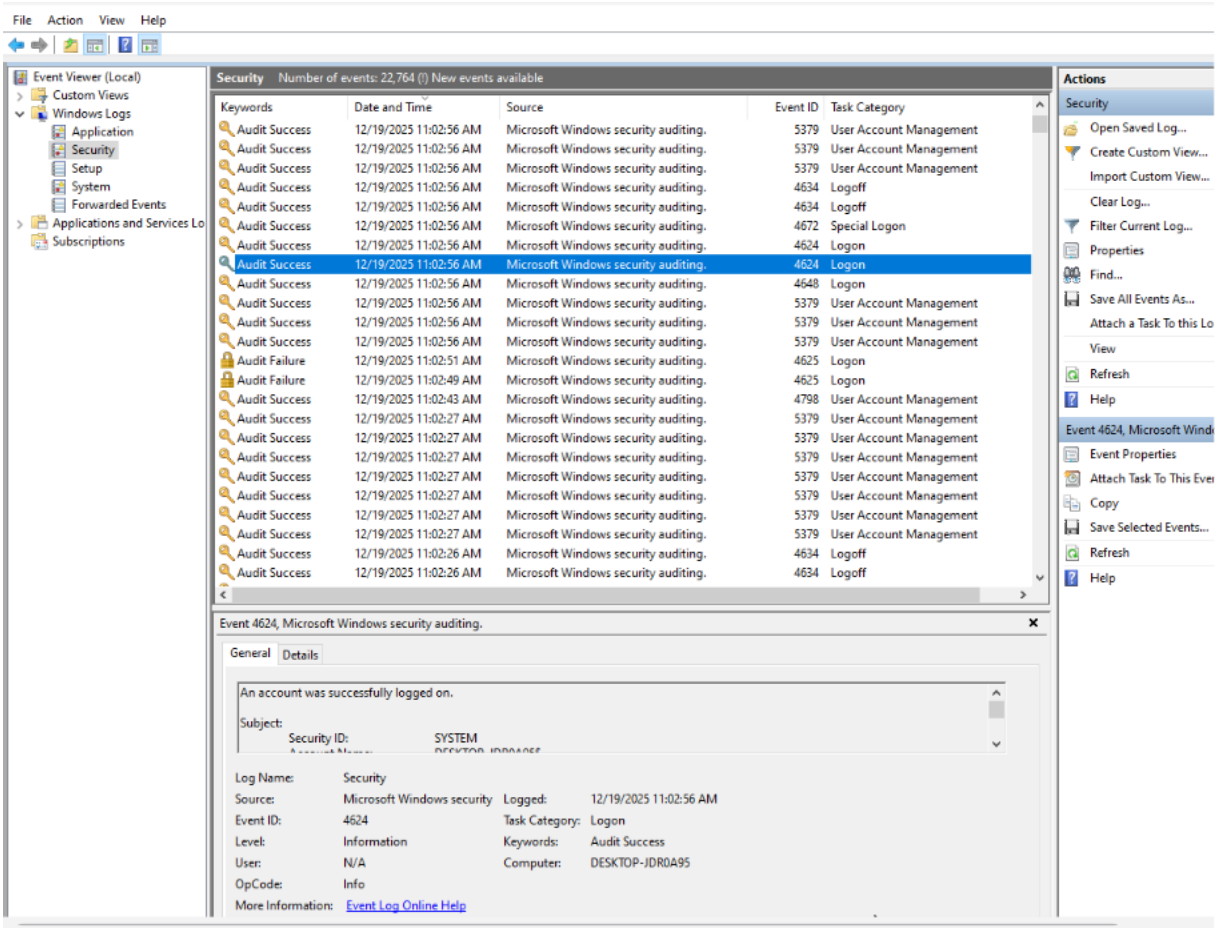
Step 2: Identify Failed Logins (Event ID 4625)

- Applied a filter for **Event ID 4625**.
- Observed multiple failed login attempts associated with the same user account.
- Noted repeated failures within a short timeframe, indicating possible brute-force or password-spraying activity.



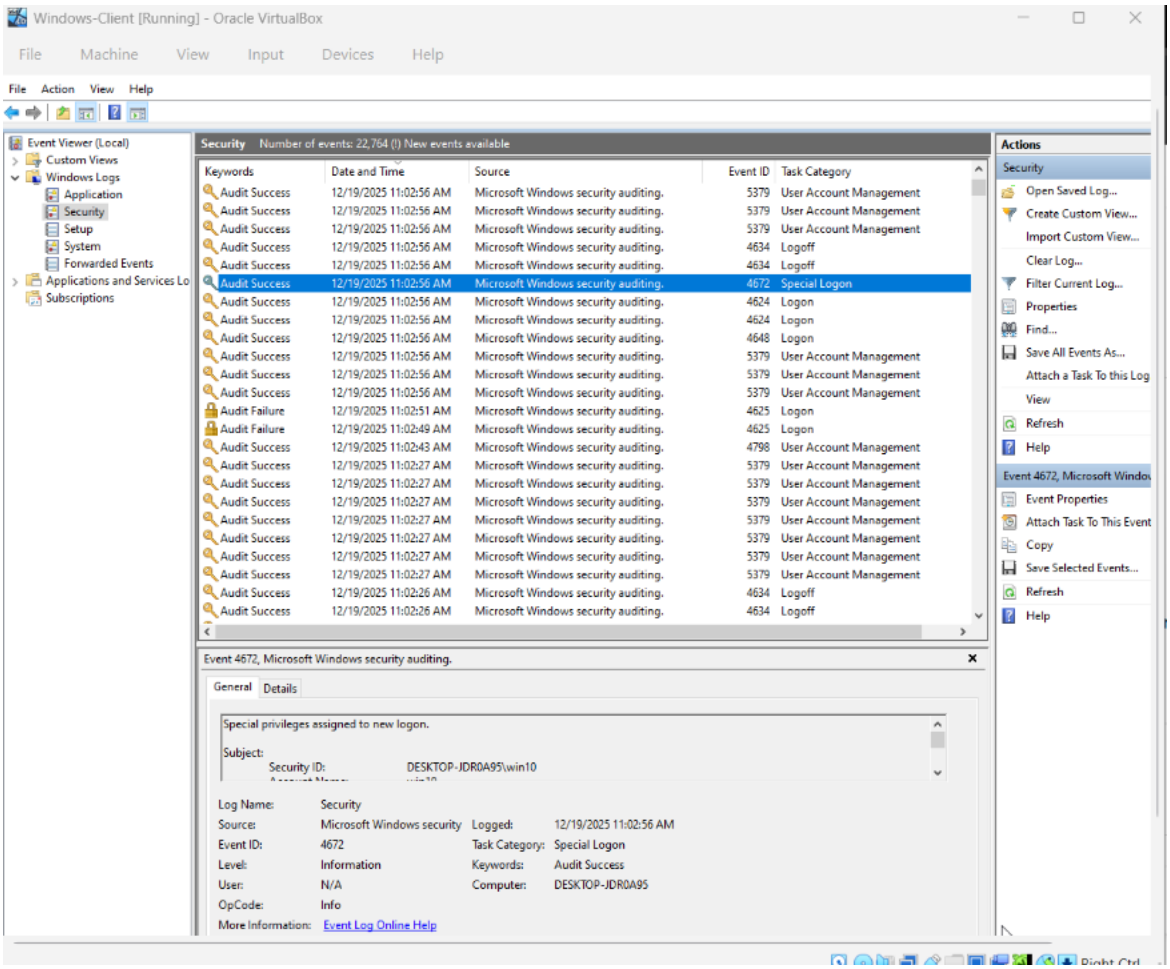
Step 3: Identify Successful Logins (Event ID 4624)

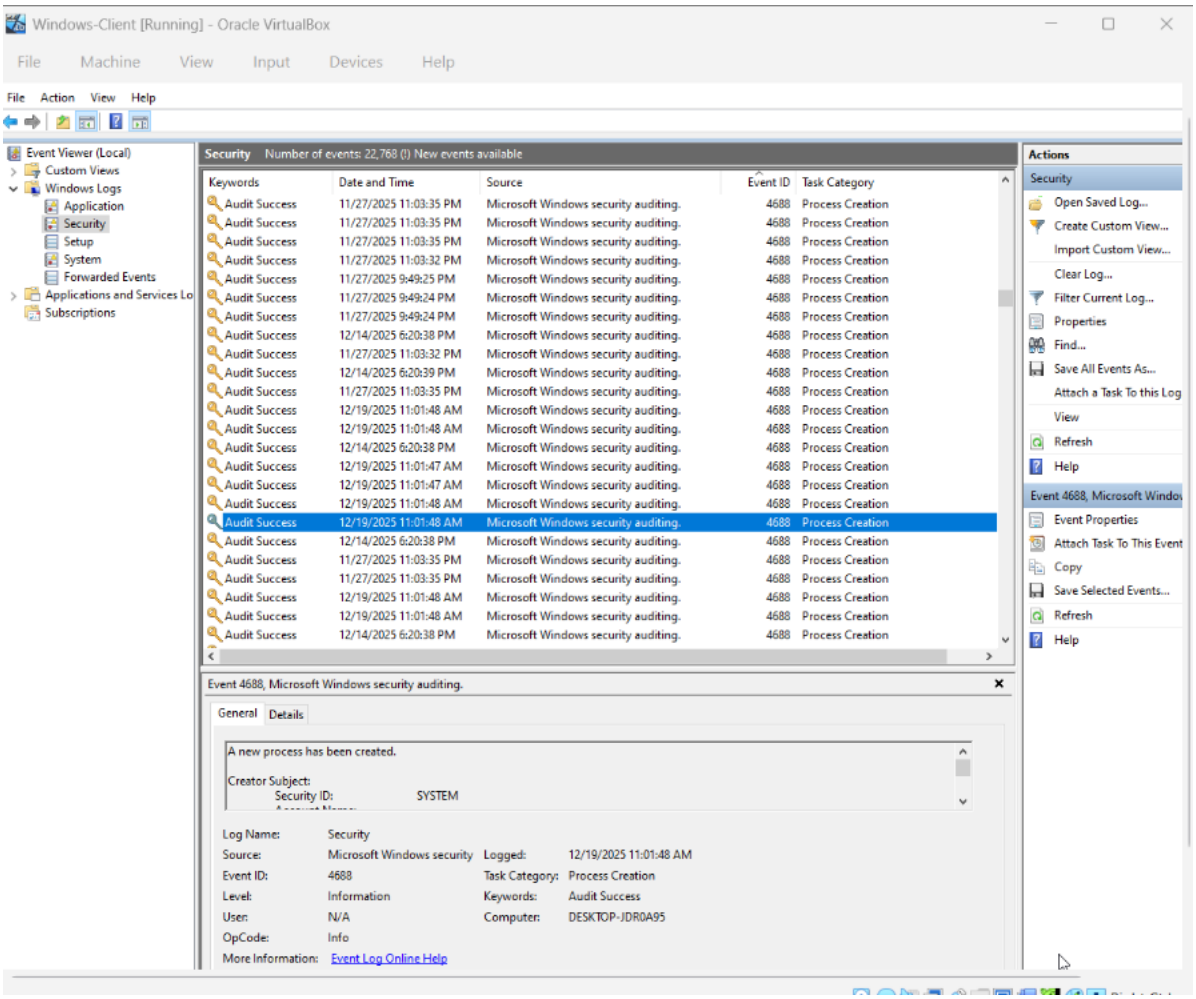
- Filtered the Security log for **Event ID 4624**.
- Identified successful login events following multiple failed attempts.
- Compared timestamps to determine whether successful access followed suspicious activity.



Step 4: Identify Additional Suspicious Events

- Additional security-relevant events were identified, including:
- **Event ID 4672:** Special logon indicating elevated privileges
- **Event ID 4688:** New process creation
- **Event ID 4740:** Account lockout
- These events suggested potential privilege escalation attempts and defensive responses by the system.


Windows-Client [Running] - Oracle VirtualBox
File Machine View Input Devices Help
File Action View Help
Event Viewer (Local)
Custom Views
Windows Logs
Application
Security
Setup
System
Forwarded Events
Applications and Services Lo
Subscriptions
Security Number of events: 22,764 (!) New events available
Keywords Date and Time Source Event ID Task Category
Audit Success 12/19/2025 11:02:56 AM Microsoft Windows security auditing. 5379 User Account Management
Audit Success 12/19/2025 11:02:56 AM Microsoft Windows security auditing. 5379 User Account Management
Audit Success 12/19/2025 11:02:56 AM Microsoft Windows security auditing. 5379 User Account Management
Audit Success 12/19/2025 11:02:56 AM Microsoft Windows security auditing. 4634 Logoff
Audit Success 12/19/2025 11:02:56 AM Microsoft Windows security auditing. 4634 Logoff
Audit Success 12/19/2025 11:02:56 AM Microsoft Windows security auditing. 4672 Special Logon
Audit Success 12/19/2025 11:02:56 AM Microsoft Windows security auditing. 4624 Logon
Audit Success 12/19/2025 11:02:56 AM Microsoft Windows security auditing. 4624 Logon
Audit Success 12/19/2025 11:02:56 AM Microsoft Windows security auditing. 4648 Logon
Audit Success 12/19/2025 11:02:56 AM Microsoft Windows security auditing. 5379 User Account Management
Audit Success 12/19/2025 11:02:56 AM Microsoft Windows security auditing. 5379 User Account Management
Audit Success 12/19/2025 11:02:56 AM Microsoft Windows security auditing. 5379 User Account Management
Audit Failure 12/19/2025 11:02:51 AM Microsoft Windows security auditing. 4625 Logon
Audit Failure 12/19/2025 11:02:49 AM Microsoft Windows security auditing. 4625 Logon
Audit Success 12/19/2025 11:02:43 AM Microsoft Windows security auditing. 4798 User Account Management
Audit Success 12/19/2025 11:02:27 AM Microsoft Windows security auditing. 5379 User Account Management
Audit Success 12/19/2025 11:02:27 AM Microsoft Windows security auditing. 5379 User Account Management
Audit Success 12/19/2025 11:02:27 AM Microsoft Windows security auditing. 5379 User Account Management
Audit Success 12/19/2025 11:02:27 AM Microsoft Windows security auditing. 5379 User Account Management
Audit Success 12/19/2025 11:02:27 AM Microsoft Windows security auditing. 5379 User Account Management
Audit Success 12/19/2025 11:02:27 AM Microsoft Windows security auditing. 5379 User Account Management
Audit Success 12/19/2025 11:02:26 AM Microsoft Windows security auditing. 4634 Logoff
Audit Success 12/19/2025 11:02:26 AM Microsoft Windows security auditing. 4634 Logoff
Event 4672, Microsoft Windows security auditing.
General Details
Special privileges assigned to new logon.
Subject:
Security ID: DESKTOP-JDR0A95\win10
Log Name: Security
Source: Microsoft Windows security Logged: 12/19/2025 11:02:56 AM
Event ID: 4672 Task Category: Special Logon
Level: Information Keywords: Audit Success
User: N/A Computer: DESKTOP-JDR0A95
OpCode: Info
More Information: Event Log Online Help
Actions
Security
Open Saved Log...
Create Custom View...
Import Custom View...
Clear Log...
Filter Current Log...
Properties
Find...
Save All Events As...
Attach a Task To this Log
View
Refresh
Help
Event 4672, Microsoft Windo
Event Properties
Attach Task To This Event
Copy
Save Selected Events...
Refresh
Help



- **Windows IOC Table**

Timestamp	Event ID	Description	IOC (Yes/No)
12/19/25 11:02:49am	4625	Multiple failed logins	Yes
12/19/25 11:02:56am	4624	Successful login after failures	Yes

12/19/25 11:02:56am	4672	Privileged logon	Yes

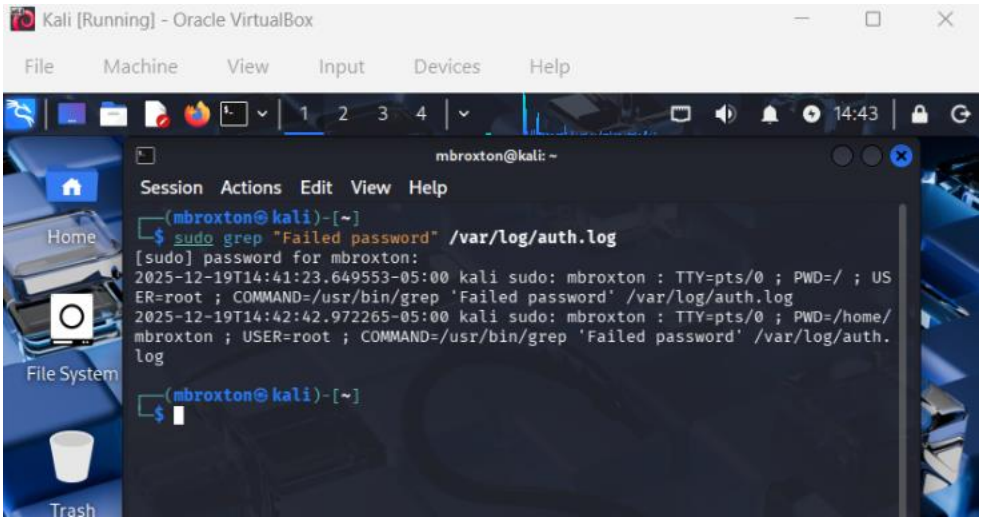
Part 2 – Linux Log Analysis

Step 1: Review Authentication Logs

- On the Linux system, the following command was used to review SSH authentication activity:
- grep "Failed password" /var/log/auth.log, grep "Accepted password" /var/log/auth.log

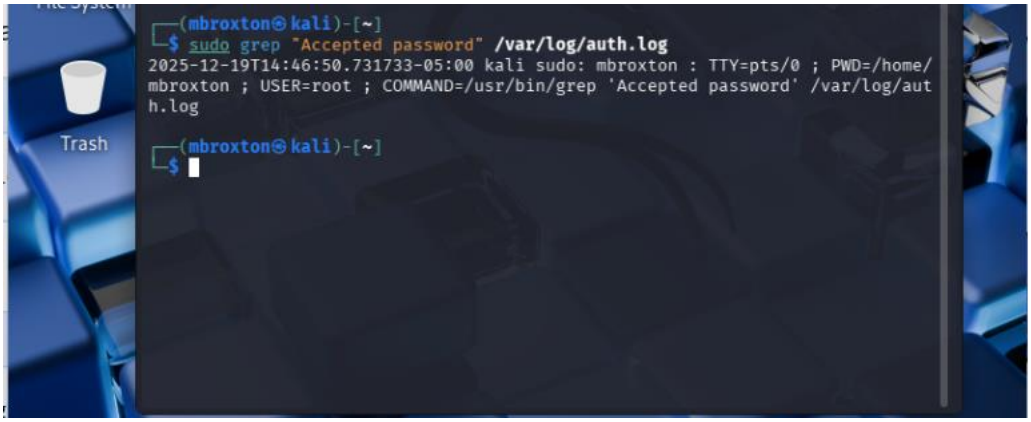
Step 2: Identify Failed Login Attempts

- Observed repeated **Failed password** entries.
- Noted attempts targeting both standard user accounts and the root account.
- Timestamps showed repeated attempts within a short period, suggesting a brute-force attempt.



Step 3: Identify Normal Login Activity

- Reviewed log entries showing successful SSH logins.
- Used these events as a baseline for comparison against suspicious attempts.



- **Linux IOC Table**

Timestamp	Username	Log Entry	IOC (Yes/No)
12/19/2025 14:41:23	root	Failed password	Yes
12/19/2025 14:42:42	mbroxton	Failed password	Yes
12/19/2025 14:46:50	root	Accepted password	Yes

5. Analysis and Findings

The Windows logs revealed multiple failed login attempts followed by a successful login and elevated privilege usage. This sequence is consistent with password-guessing activity and potential account compromise. Linux logs showed repeated failed SSH login attempts from the same source IP, indicating a likely brute-force attack. Together, these artifacts demonstrate how log analysis can uncover unauthorized access attempts and suspicious user behavior.

6. Conclusion

This lab demonstrated the importance of log analysis in detecting security incidents. By reviewing Windows Security logs and Linux authentication logs, suspicious login behavior and potential indicators of compromise were identified. Event correlation across systems provided valuable insight into attacker techniques and reinforced the role of logging in incident detection and response.

7. Challenges and Observations

- Security logs contained large volumes of data, requiring careful filtering.
- Correlating timestamps across events helped build an accurate timeline.
- Both Windows and Linux logs provided complementary evidence of suspicious activity.

8. References

- Microsoft. *Windows Security Event IDs Documentation*
- NIST SP 800-61 Rev. 2 – Computer Security Incident Handling Guide
- NIST SP 800-92 – Guide to Computer Security Log Management
- Course materials and lab instructions provided by the instructor